



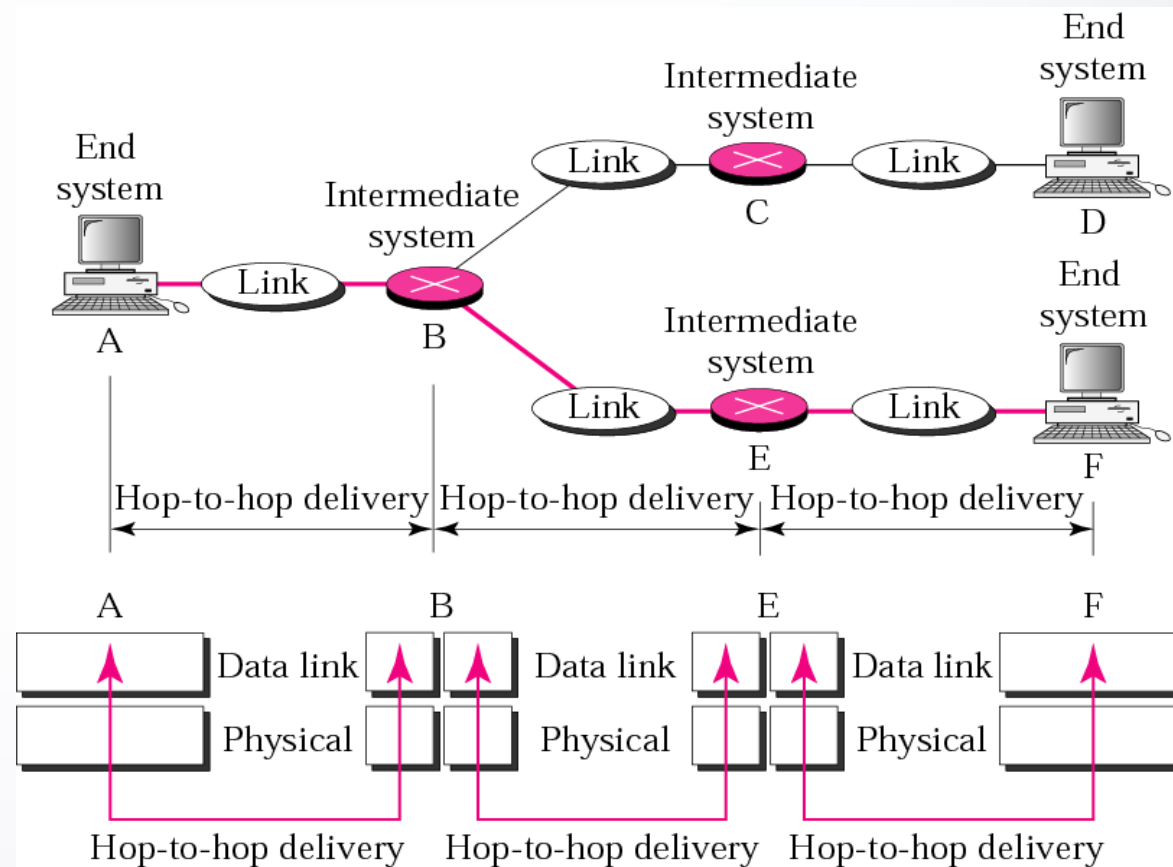
23CS/MC/CN55

Unit 3

3.1 DATA LINK LAYER

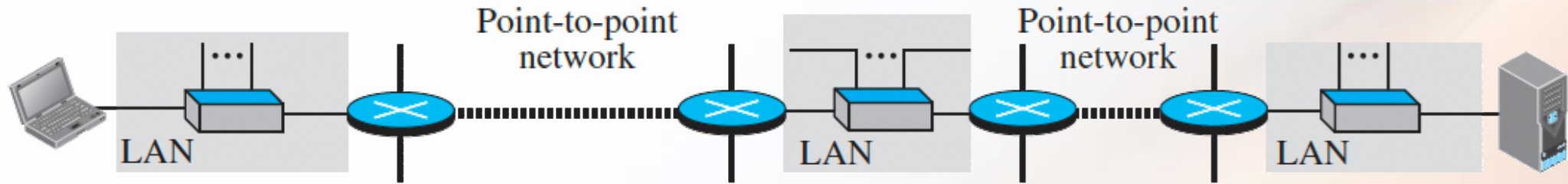
INTRODUCTION

- Communication at the data link layer is node-to-node.



NODE & LINK

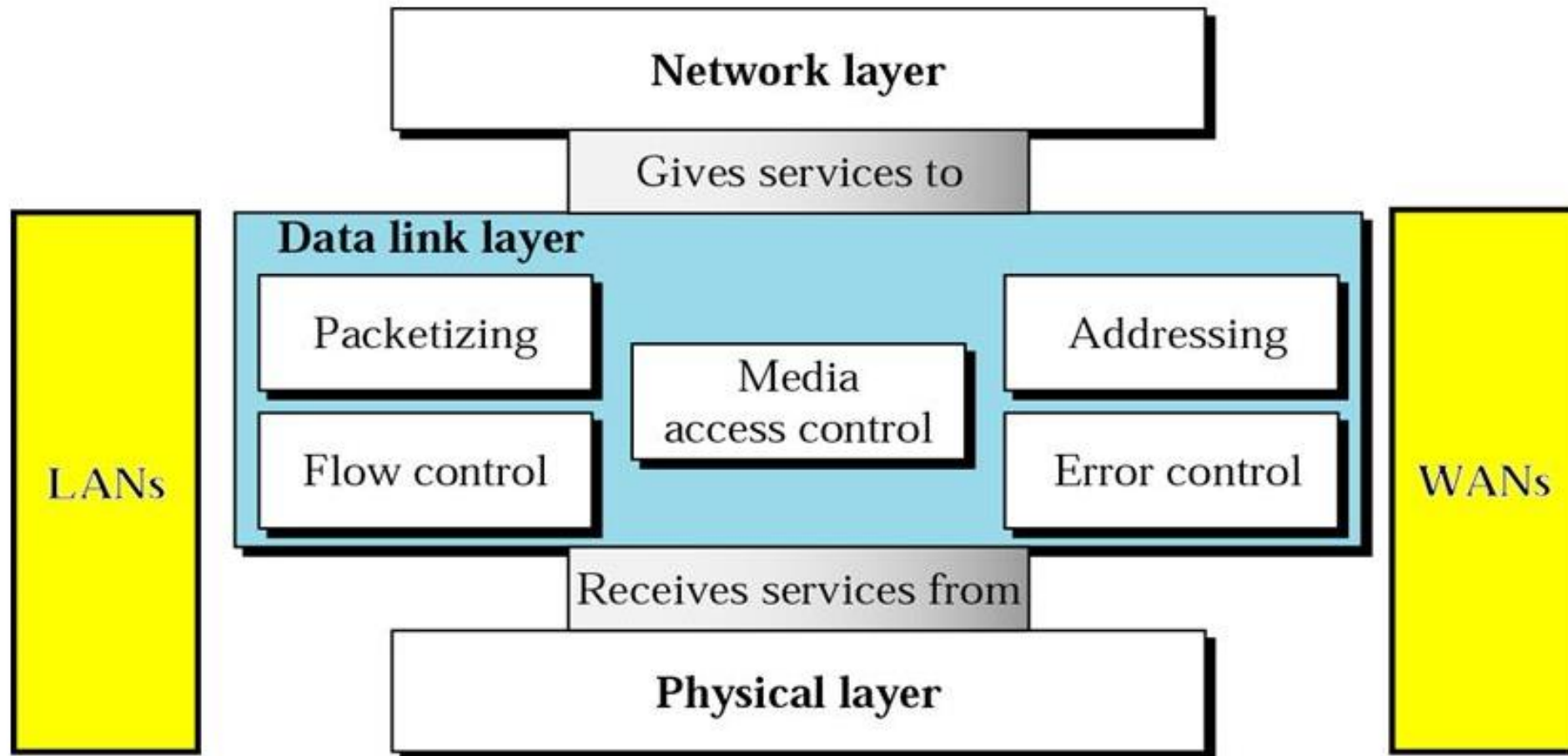
- Two end hosts and the routers as **nodes**
- Networks in between as **links**



a. A small part of the Internet



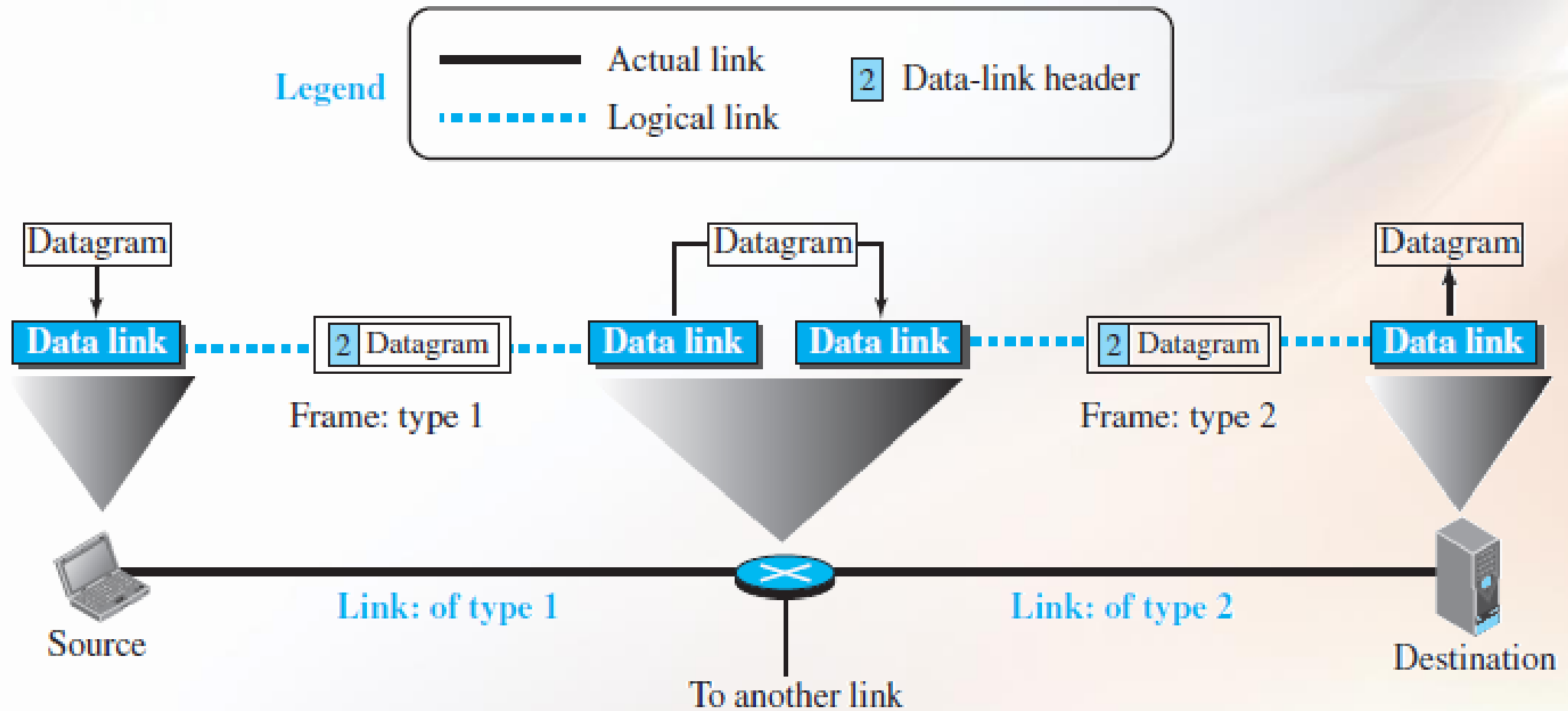
DATA LINK LAYER SERVICES



DATA LINK LAYER SERVICES



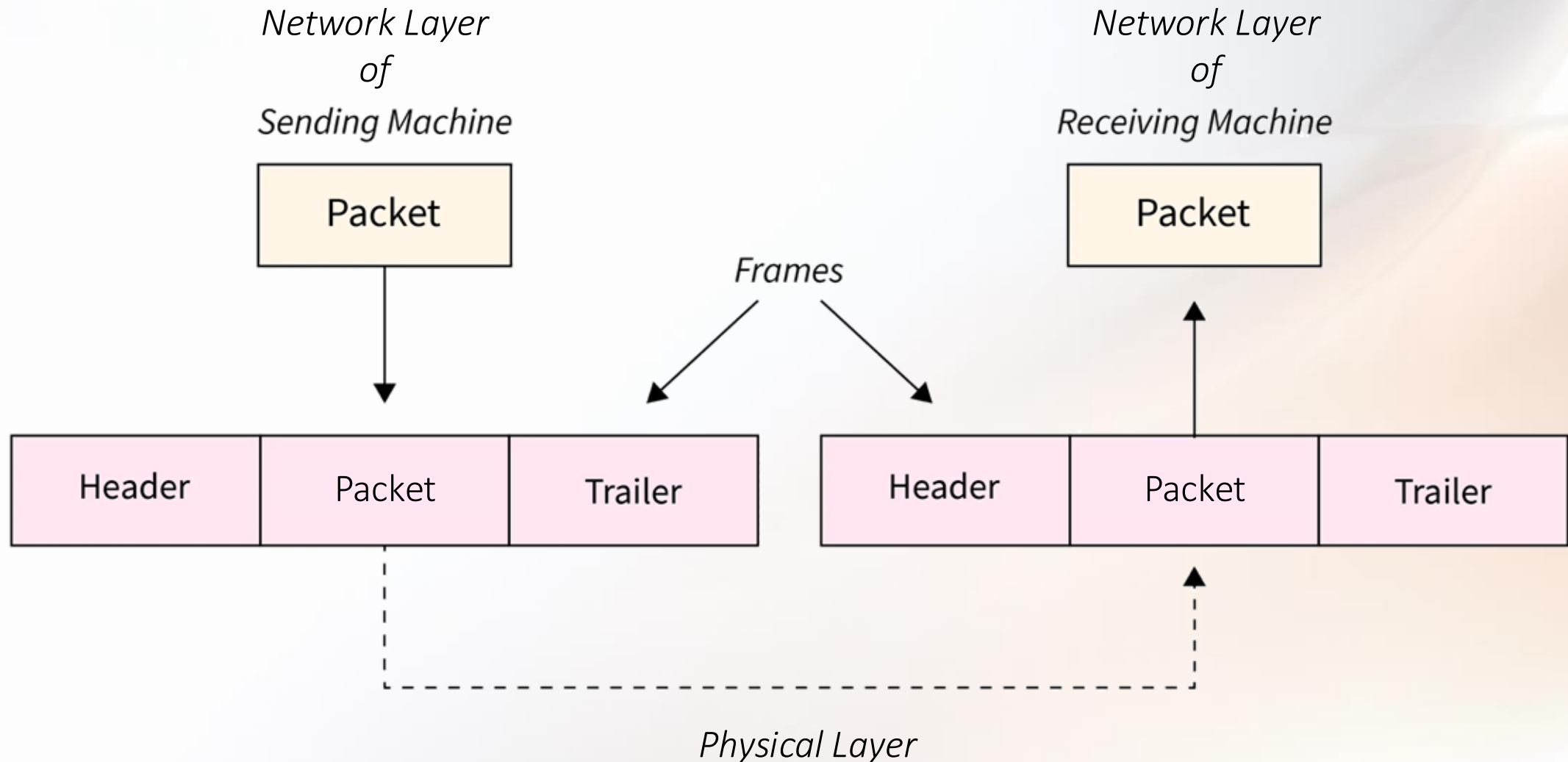
DATA LINK LAYER SERVICES



DATA LINK LAYER SERVICES

- Data link layer of the source host needs only to encapsulate
- The data-link layer of the destination host needs to decapsulate
- But each intermediate node needs to both encapsulate and decapsulate.

FRAMING A packet at the data-link layer is normally called a frame.



FLOW CONTROL

- Producer – Consumer
- Need for buffering
- Drop frames when buffer is full (or)
- Ask sending DLL to stop or slow-down



Image Source: <https://tenor.com/view/who-am-i-jackie-chan-1998-mitsubishi-lancer-power-slide-gif-22658953>

ERROR CONTROL

- At Sender's Side: Frames → Bits → Electromagnetic Signals
- At Receiver's Side: Electromagnetic Signals → Bits → Frames

SIGNAL ERROR

- Either corrected at the receiver node (or)
- Discarded and retransmitted by the sending node.

CONGESTION CONTROL

- No direct congestion control though WAN do.
- Congestion control is of end-to-end nature so considered mainly by network and transport layer.



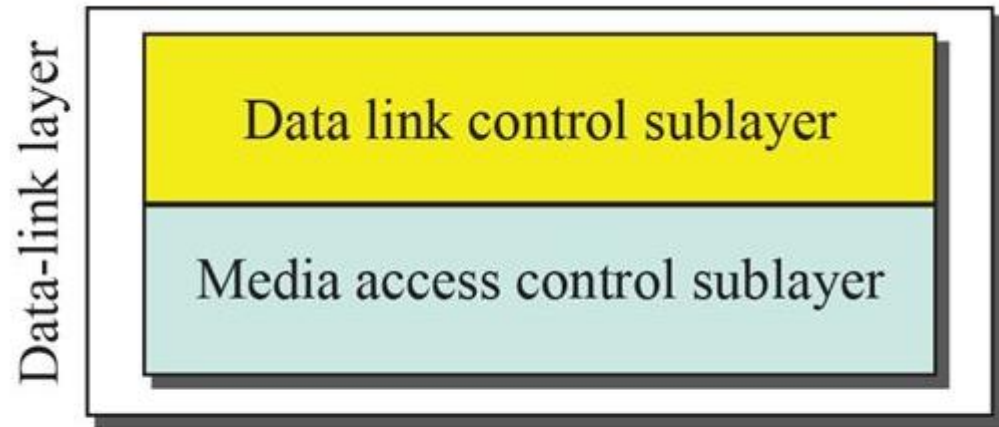
Image Source: <https://giphy.com/gifs/crazy-going-gas-prices-qAhQGJTjYj7x4KvLv6>

TWO CATEGORIES OF LINKS

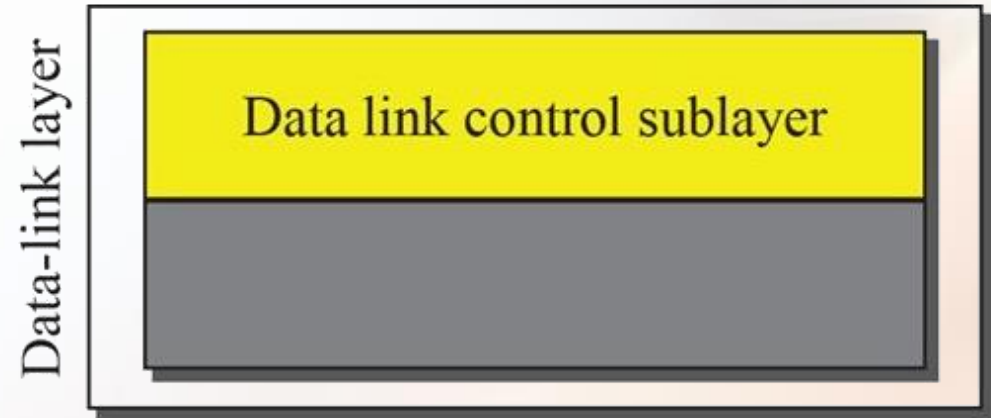
- Broadcast Link
- Point-to-point (Unicast) Link



DATA LINK SUB LAYERS



a. Data-link layer of a broadcast link



b. Data-link layer of a point-to-point link

DATA LINK CONTROL (DLC)

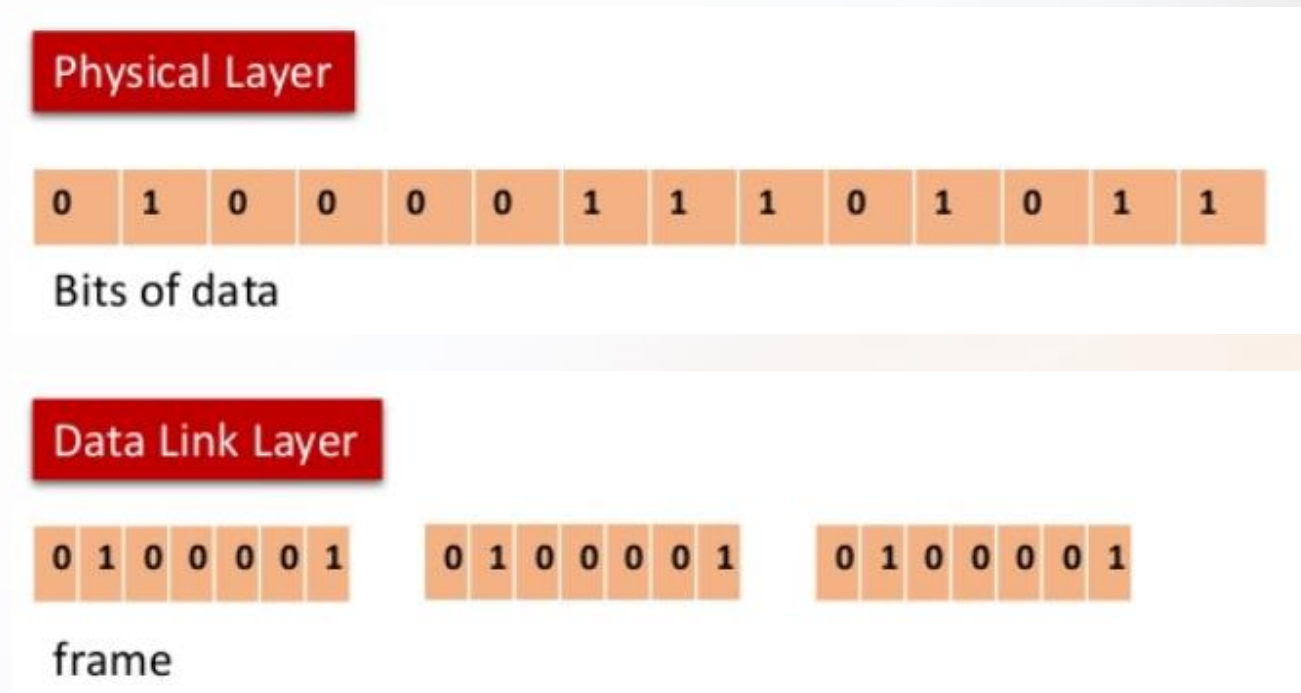
- Deals with procedures for communication between two adjacent nodes

—no matter whether the link is dedicated or broadcast.

- Data link control functions include framing and flow and error control.

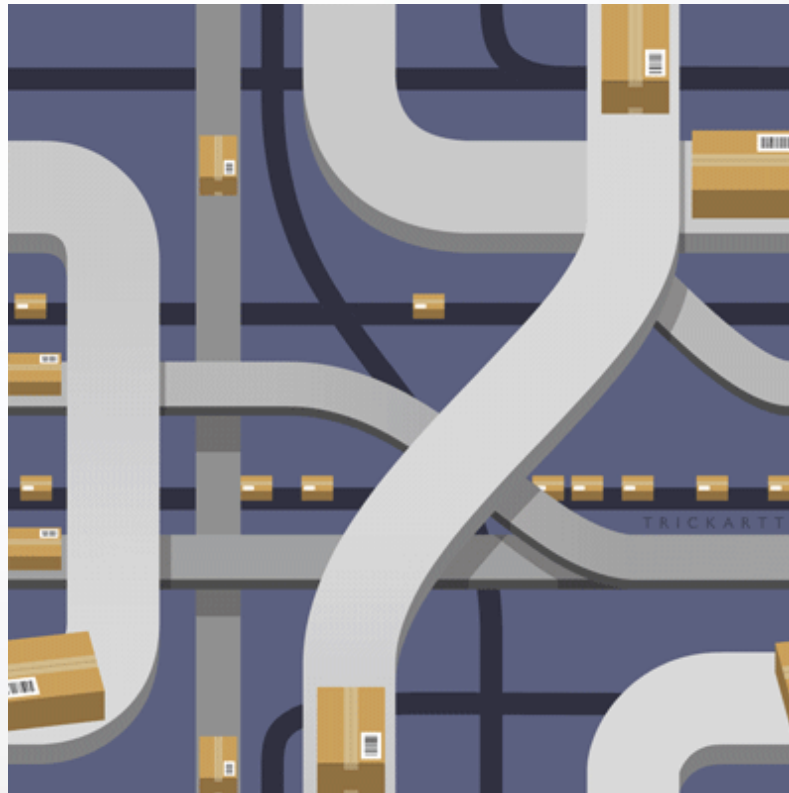
FRAMING

- DLL needs to pack bits into frames, so that each frame is distinguishable from another.



FRAMING

- *Framing* separates a message from one source to a destination by adding a sender address and a destination address.



FRAMING

- Although the whole message could be packed in one frame, that is not normally done, because
 1. A frame is very large, making flow and error control very inefficient.
 2. Even a single-bit error would require the retransmission of the whole frame.

FRAME SIZE

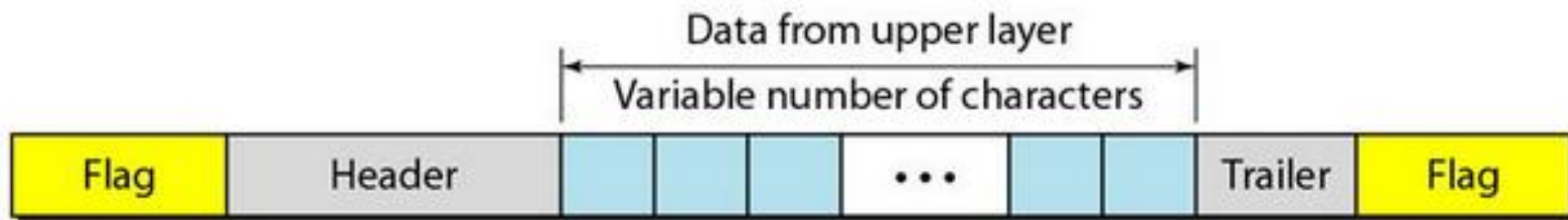
- Frames can be of fixed or variable size.
- In *fixed-size framing*, there is no need for defining the boundaries of the frames.
- Frames of fixed size are called cells.
- In *variable-size framing* (mostly used in LAN), we need a way to define the end of one frame and the beginning of the next.
 - Character-oriented approach
 - Bit-oriented approach

CHARACTER/BYTE ORIENTED FRAMING

- Data to be carried are 8-bit characters from a coding system such as ASCII.
- The header normally carries the source and destination addresses and other control information
- The trailer carries error detection redundant bits

CHARACTER/BYTE ORIENTED FRAMING

- To separate one frame from the next, an 8-bit (1-byte) **flag** is added at the beginning and the end of a frame.
- The flag, composed of protocol-dependent special characters, signals the start or end of a frame.



CHARACTER/BYTE ORIENTED FRAMING

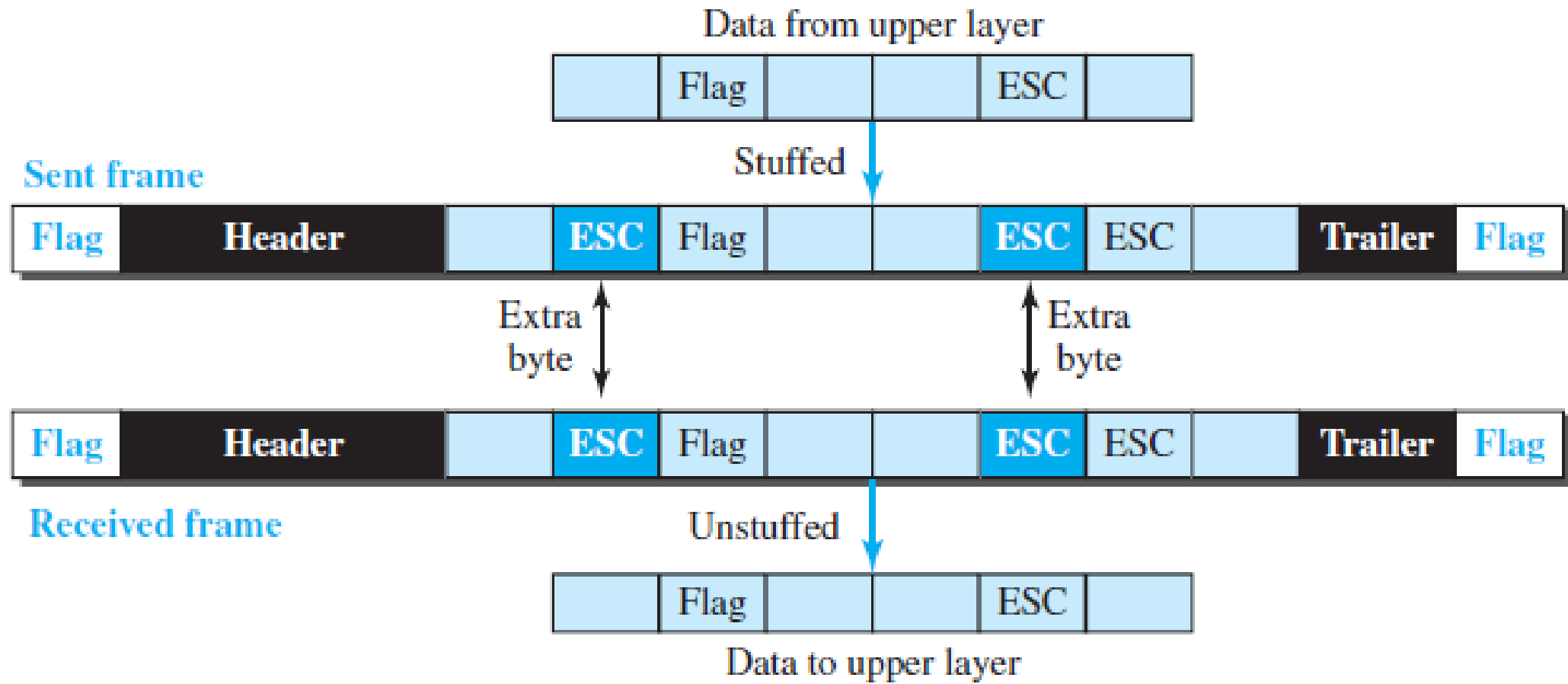


- **Byte stuffing** is the process of adding one extra byte whenever there is a flag or escape character in the text.
- This byte is usually called the *escape character (ESC)* and has a predefined bit pattern.

CHARACTER/BYTE ORIENTED FRAMING

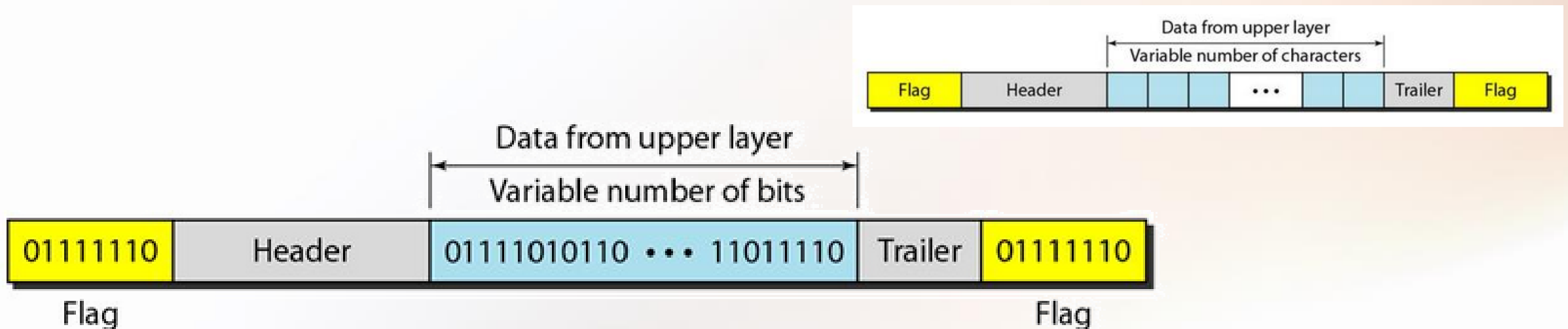


BYTE STUFFING AND UNSTUFFING

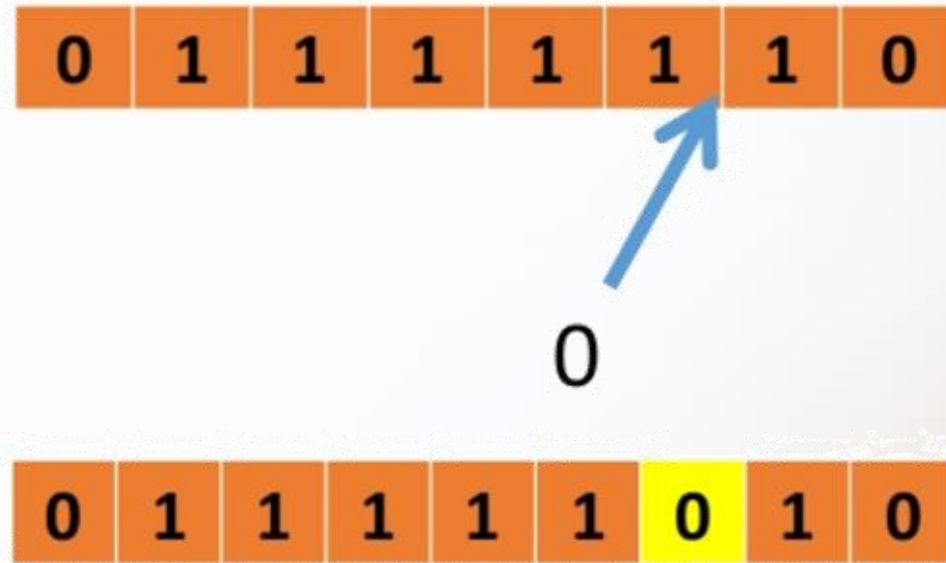


BIT-ORIENTED FRAMING

- The data section, a sequence of bits is to be interpreted by the upper layer as text, graphic, audio, video, and so on.
- Addition to header and trailer, we still need a delimiter to separate one frame from the other.
- Most protocols use a special 8-bit flag, 01111110, as the delimiter

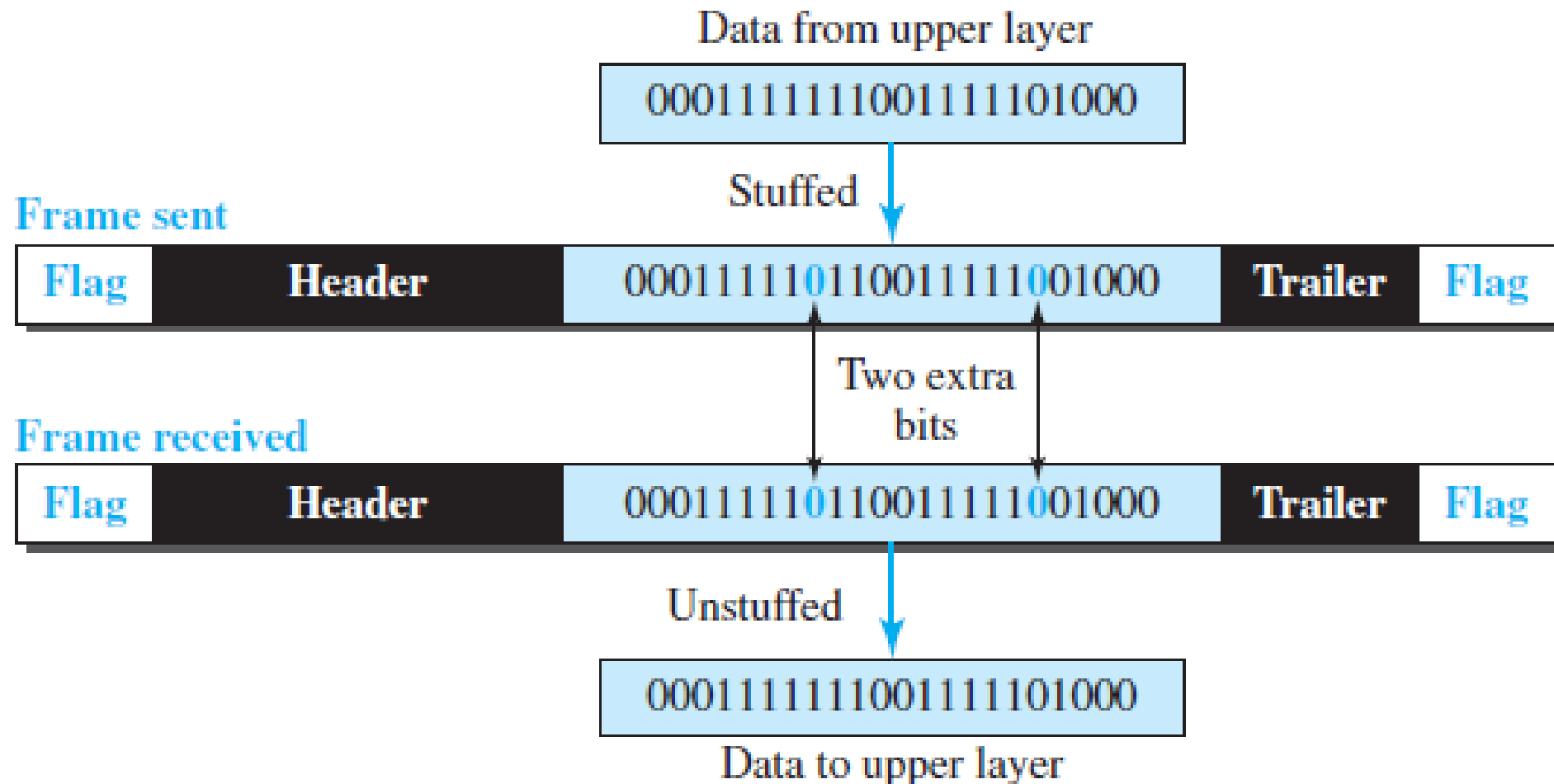


BIT-ORIENTED FRAMING

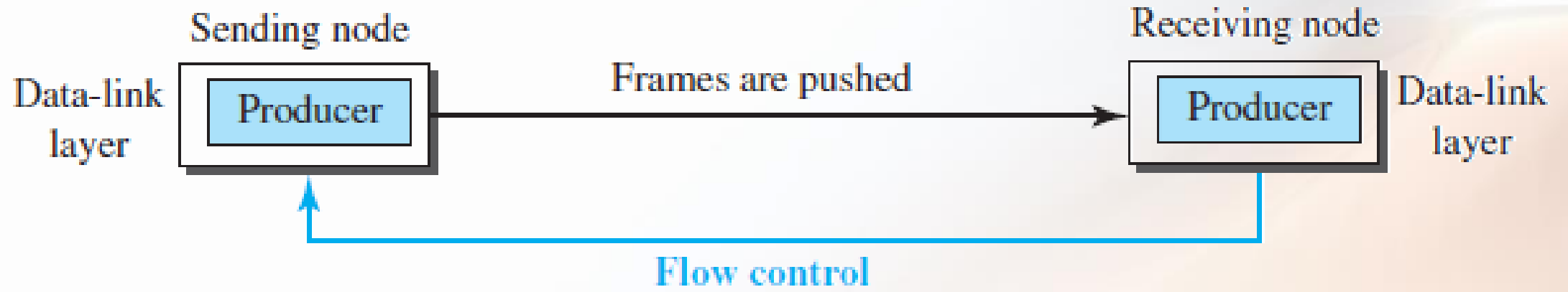


- Bit stuffing is the process of adding one extra 0 whenever five consecutive 1s follow a 0 in the data, so that the receiver does not mistake the pattern 0111110 for a flag.

BIT STUFFING AND UNSTUFFING



FLOW CONTROL



FLOW CONTROL

- One of the solutions is normally to use two *buffers*;
 - one at the sending data-link layer and the other at the receiving data-link layer.
- A buffer is a set of memory locations that can hold packets at the sender and receiver.
- When the buffer of the receiving data-link layer is full, it informs the sending data-link layer to stop pushing frames.

ERROR CONTROL



Image Source: https://resize.indiatvnews.com/en/resize/newbucket/1200_-/2025/06/amazon-introduces-packaging-that-signals-tampering-1748938032.webp

ERROR CONTROL

- CRC (Cyclic Redundancy Check) is added to the frame header by the sender and checked by the receiver.
- If the frame is corrupted, it is silently discarded.
- Error control - 2 methods
 1. Error Detection Only (No Acknowledgment)
 2. Error Detection + Acknowledgment (Reliable Delivery)

COMBINATION OF FLOW & ERROR CONTROL

- The acknowledgment serves two purposes – flow and error control
 - Flow Control: "I have received the frame, and I am ready to receive the next one."
 - Error Control: "The frame was received correctly (without errors)."
- A frame that carries an acknowledgment is normally called an **ACK** to distinguish it from the data frame.

TYPES OF PROTOCOLS

- Connectionless Protocols
- Connection-Oriented Protocols

CONNECTIONLESS PROTOCOLS

- Frames are independent, no connection between them.
- They are not numbered and not ordered.
- DLL LAN protocols are connectionless.

CONNECTION-ORIENTED PROTOCOLS

- Has setup, transfer and teardown phases
- They are numbered and ordered.
- Receiver to wait until all frames in the same set are received.
- Used in some point-to-point protocols, some wireless LANs & WANs

PROTOCOLS

- Traditionally four protocols defined for flow and error control:
 1. Simple
 2. Stop-and-Wait
 3. Go-Back-N
 4. Selective-Repeat

PROTOCOLS

- Traditionally four protocols defined for flow and error control:

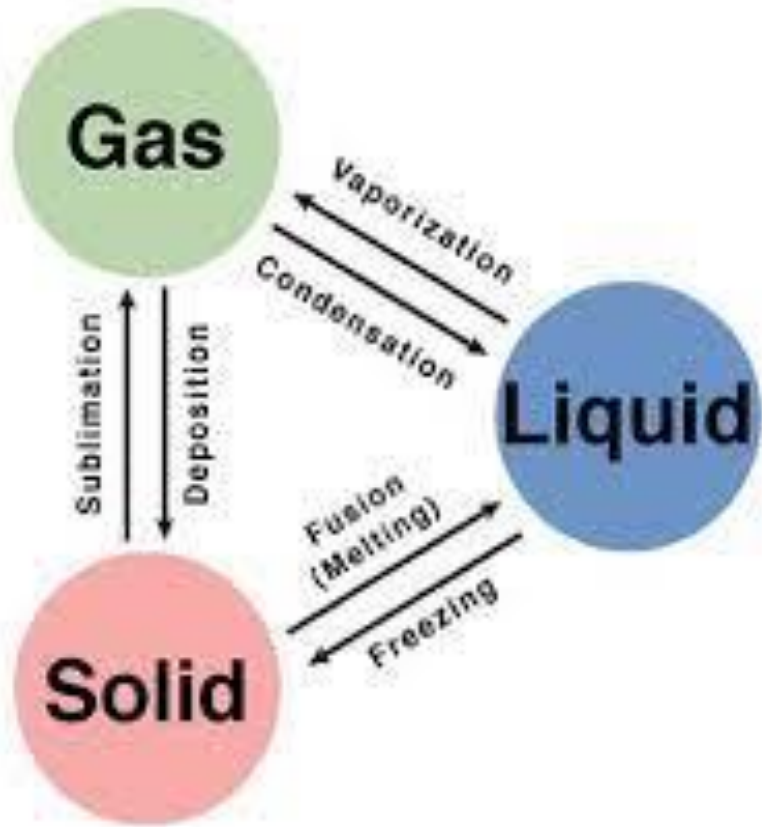
1. Simple

2. Stop-and-Wait

- ~~3. Go-Back-N~~

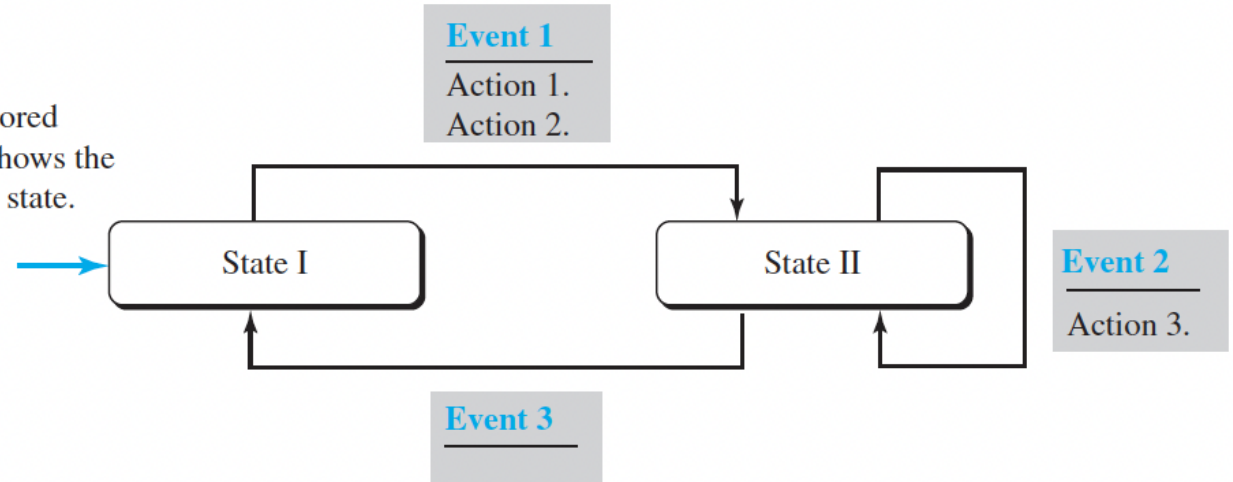
- ~~4. Selective Repeat~~

AS FINITE STATE MACHINE



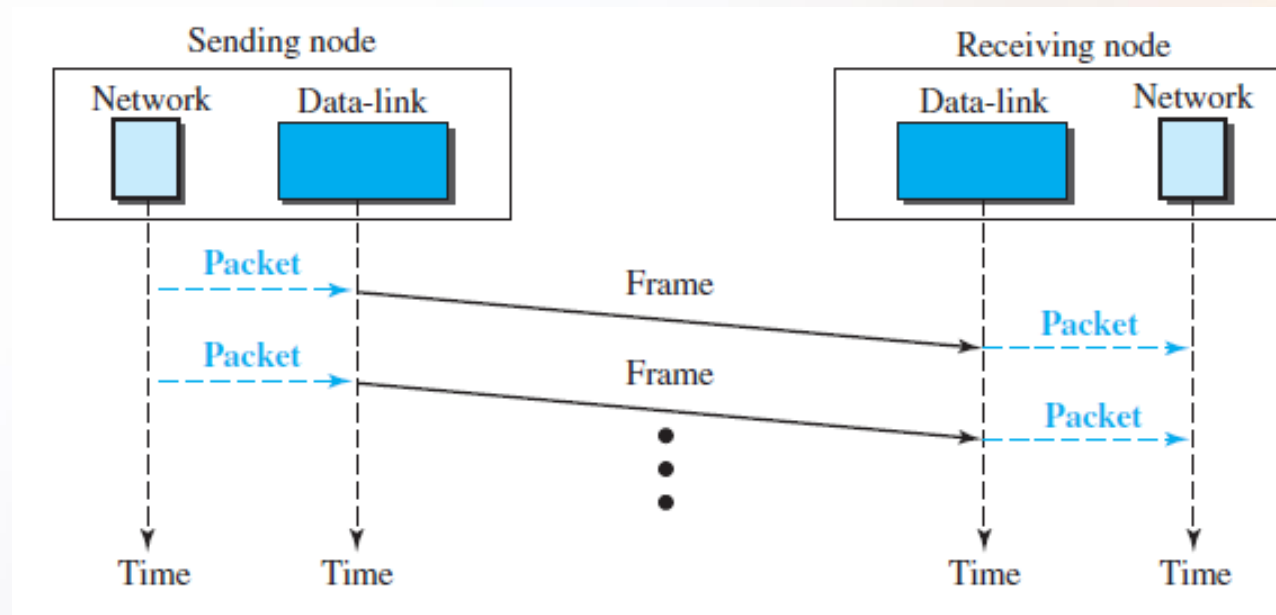
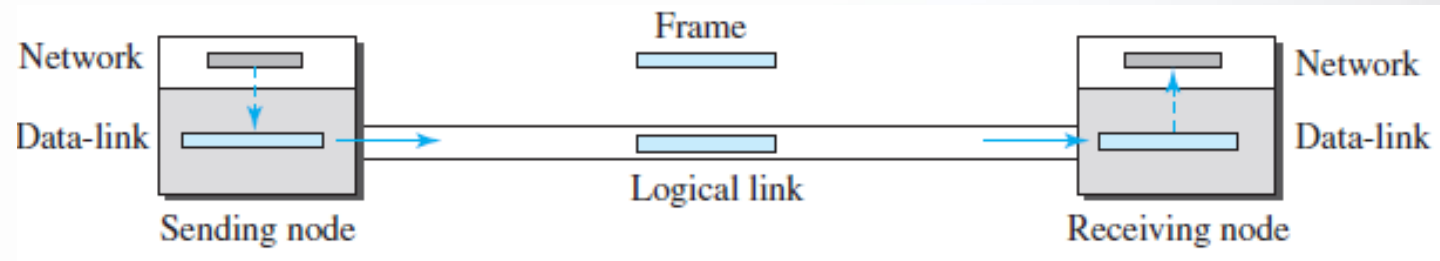
Note:

The colored arrow shows the starting state.

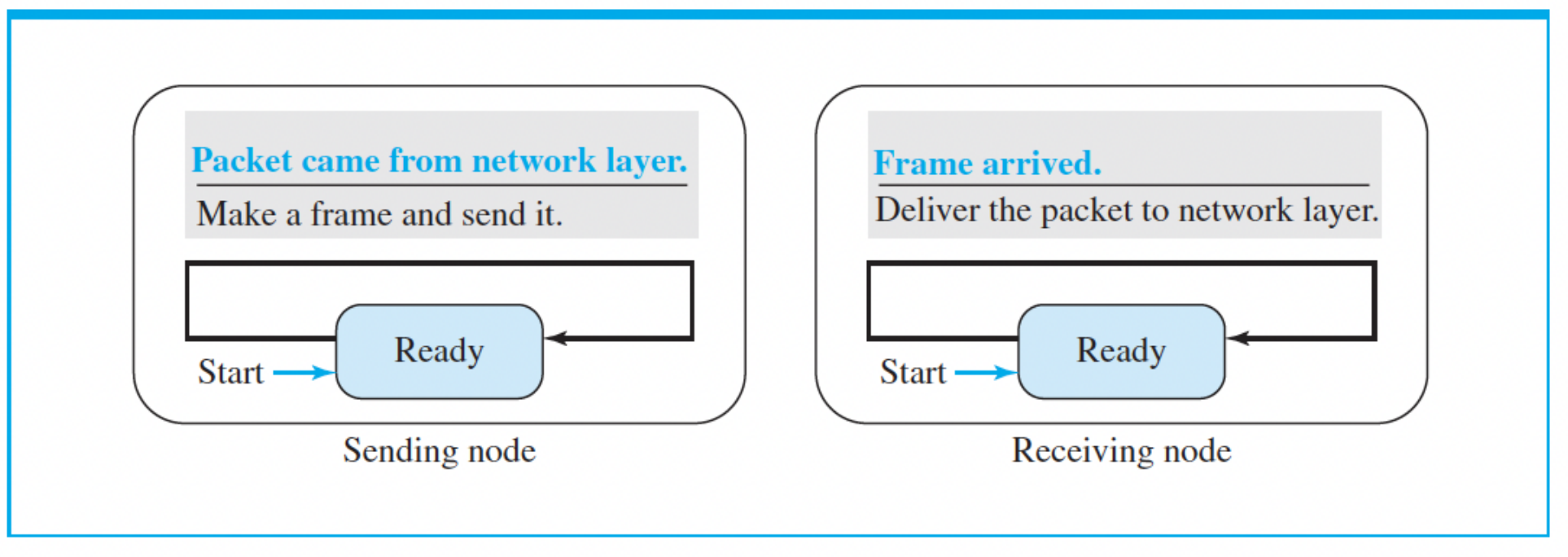


SIMPLE PROTOCOL

- Simple protocol has neither flow nor error control



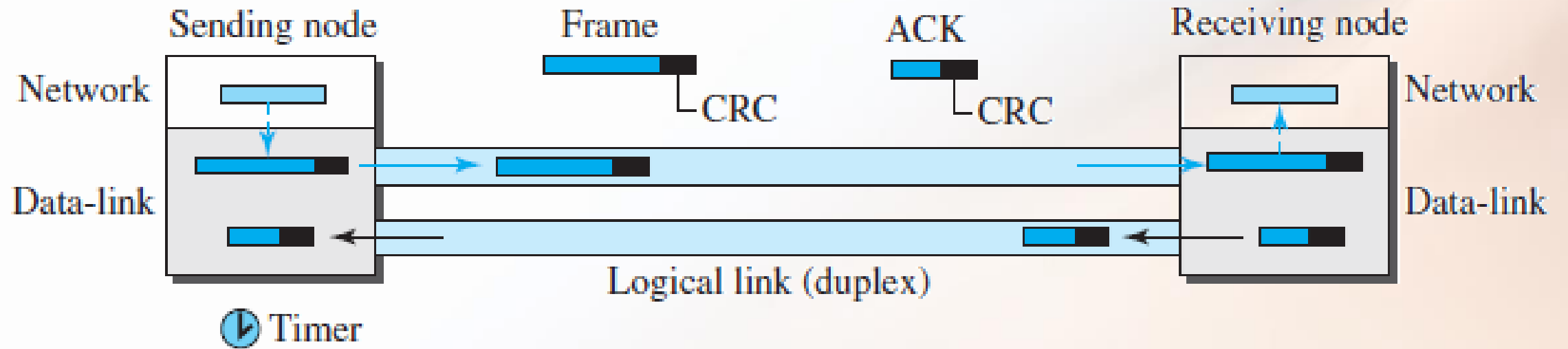
FSM FOR SIMPLE PROTOCOL



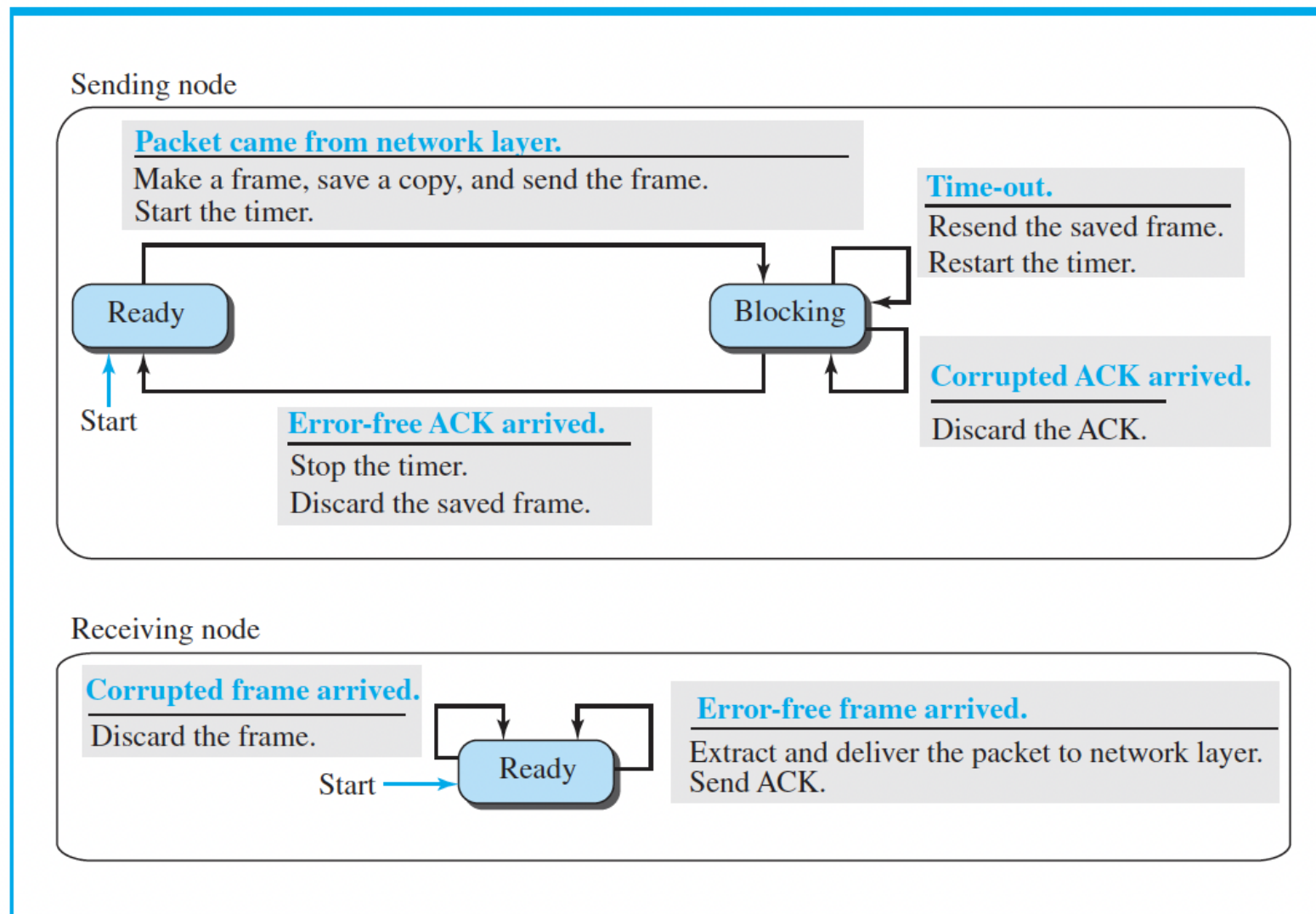
STOP-AND-WAIT PROTOCOL

- Uses both flow and error control.
- The sender sends one frame at a time and waits for an acknowledgment before sending the next one.
- To detect corrupted frames, we need to add a CRC to each data frame.
- Note that only one frame and one acknowledgment can be in the channels at any time.

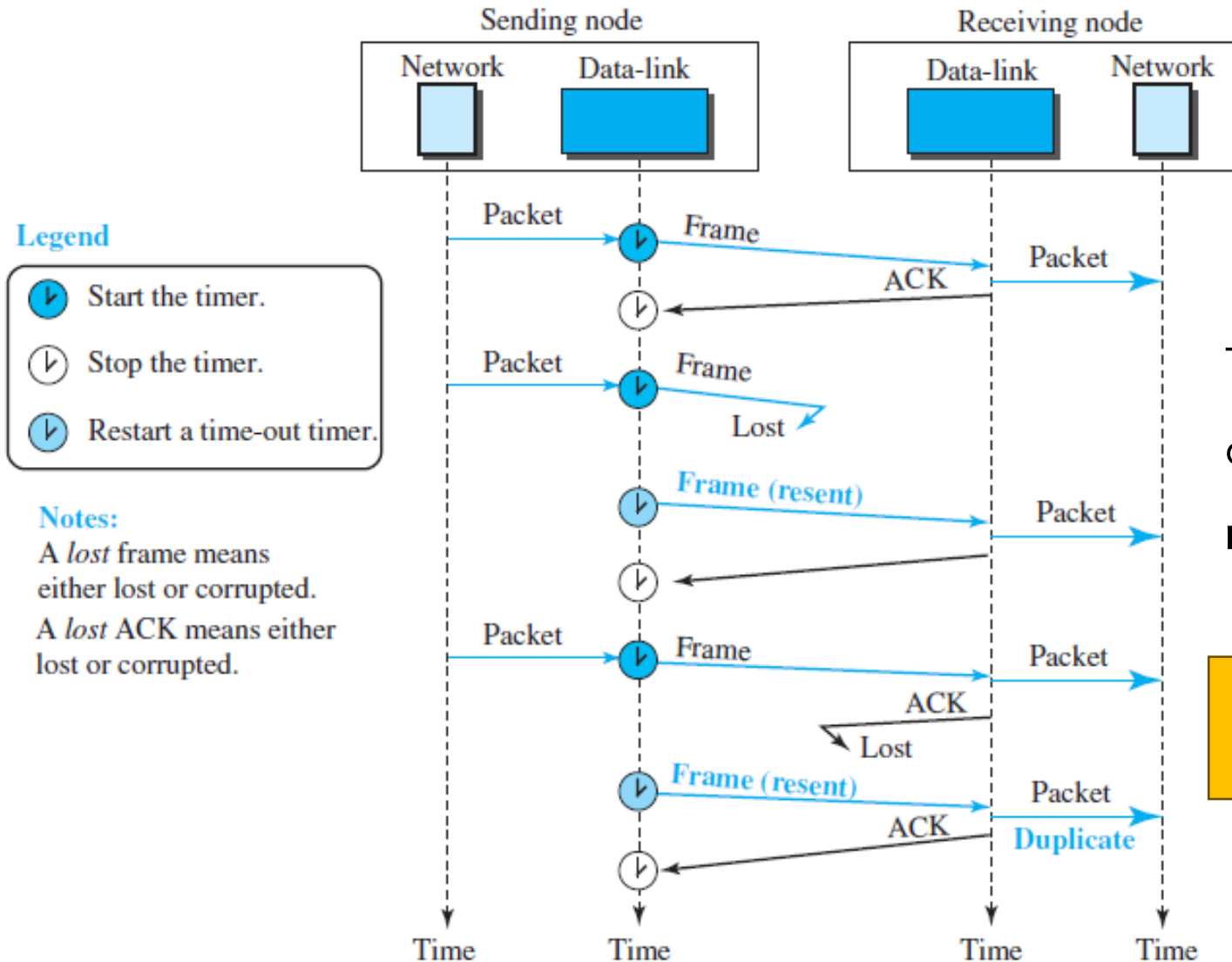
STOP-AND-WAIT PROTOCOL



FSM FOR STOP-AND-WAIT PROTOCOL



STOP-AND-WAIT PROTOCOL



To add **sequence numbers** to the data frames and **acknowledgment numbers** to the ACK frames

Any Issues?

STOP-AND-WAIT PROTOCOL

AUTOMATIC REPEAT REQUEST (ARQ)

Legend

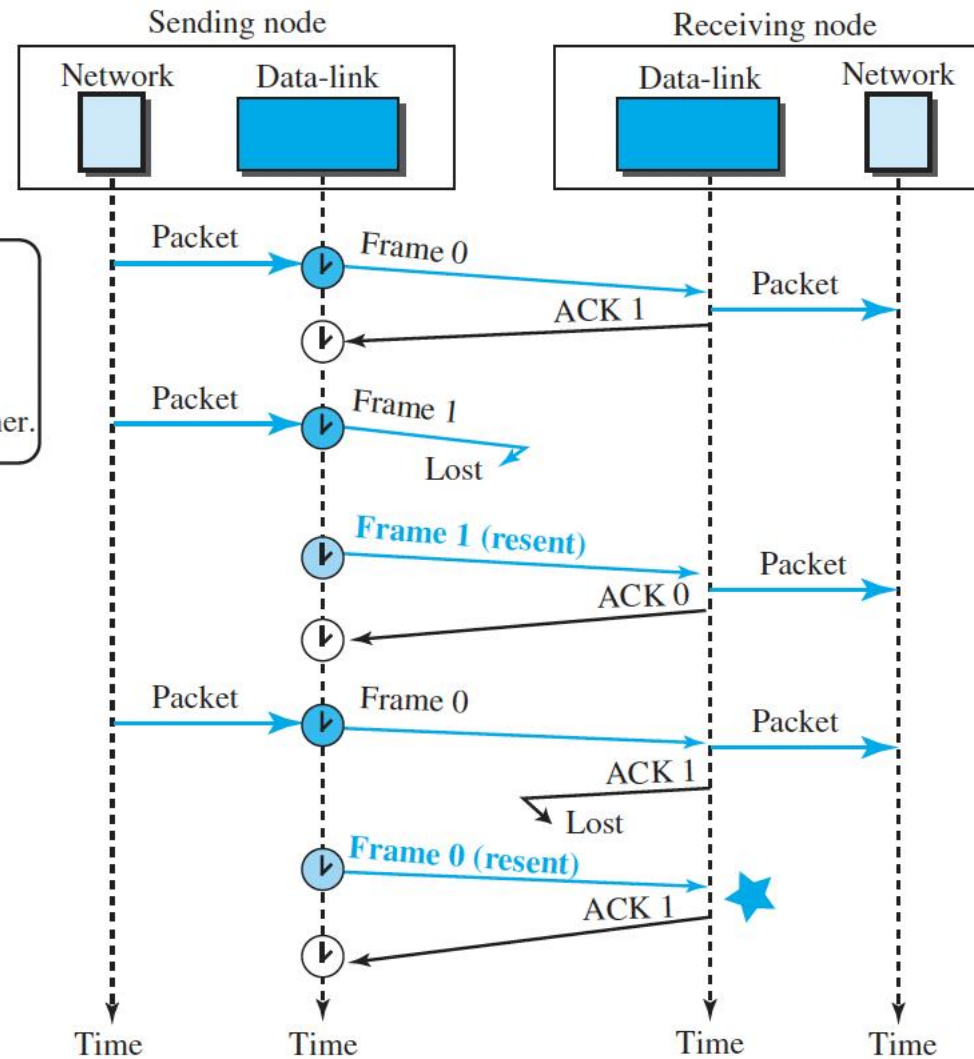
- Start the timer.
- Stop the timer.
- Restart a time-out timer.

Notes:

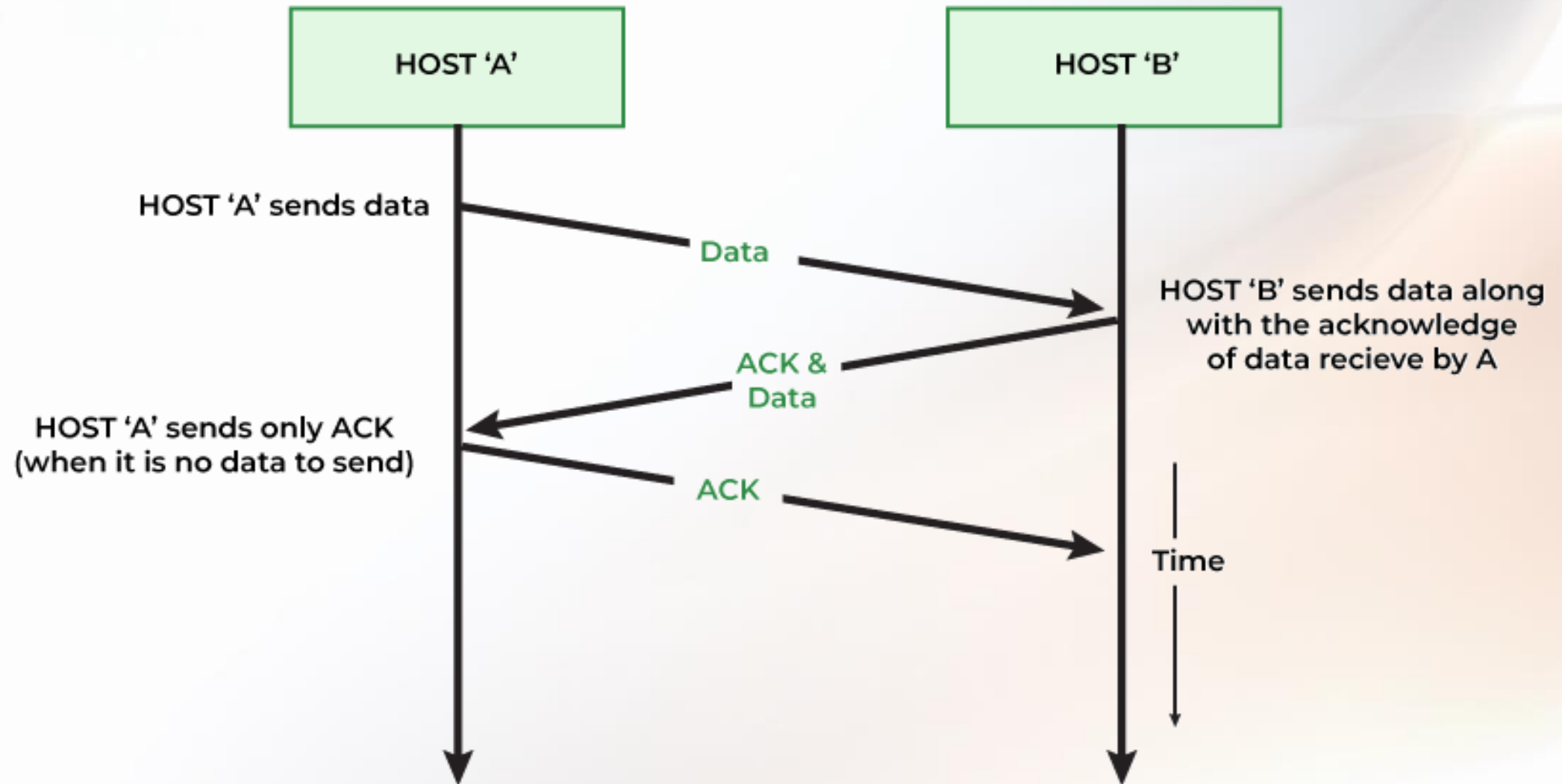
A *lost* frame means either lost or corrupted.
A *lost* ACK means either lost or corrupted.



Frame 0 is discarded because the receiver expects frame 1.



PIGGYBACKING



PIGGYBACKING

- The data in one direction is piggybacked with the acknowledgment in the other direction.
- Node A is sending data to node B, Node A also acknowledges the data received from node B.

A hand holding a white cup, with a blurred background.

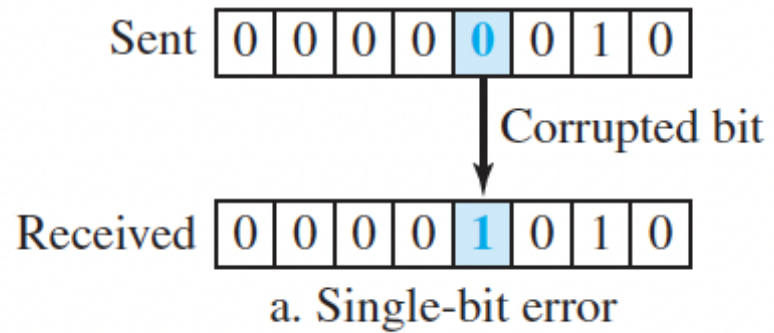
ERROR DETECTION & CORRECTION

TYPES OF ERRORS

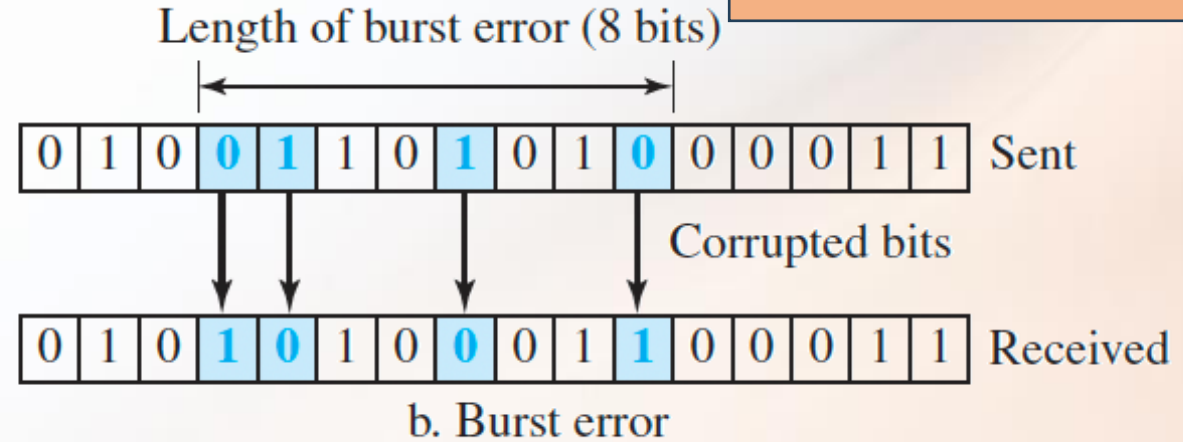
- Whenever bits flow from one point to another, they are subject to unpredictable changes because of interference.
- Two types:
- Single-bit error
- Burst error

TYPES OF ERRORS

Change in 1 bit



Change in 2 or more bits



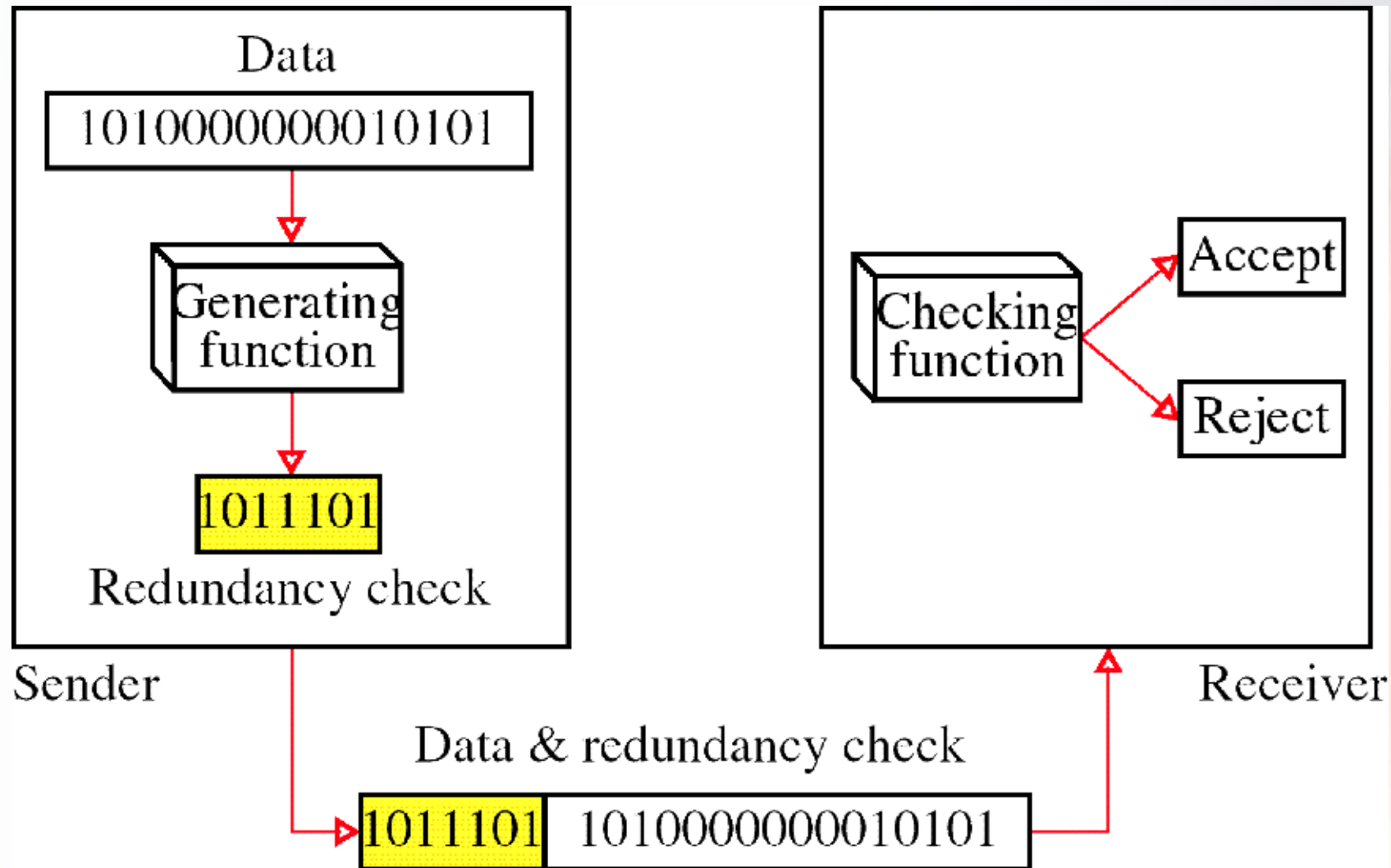
BURST ERRORS – THE MOST LIKELY ONES TO HAPPEN

- The number of bits affected depends on the data rate and duration of noise.
- For example, if we are sending data at 1 kbps, a noise of 1/100 second can affect 10 bits

REDUNDANCY

- The central concept in detecting or correcting errors is redundancy.
- To be able to detect or correct errors, we need to send some extra bits with our data.
- These redundant bits are added by the sender and removed by the receiver.

REDUNDANCY



DETECTION VS. CORRECTION

- Correction of Errors **more difficult** than Detection of Errors

ERROR DETECTION

- Only answers YES or NO to the question – Whether any error has occurred?
- Does not even bother about the number of corrupted bits.
- A single-bit error is the same as a burst error.

ERROR CORRECTION

- Need to know the exact number of bits that are corrupted and, more importantly, their location in the message.
- The number of errors and the size of the message are important factors.
 - If we need to correct a single error in an 8-bit data unit, we need to consider eight possible error locations
 - If we need to correct two in a data unit of the same size, we need to consider 28 (permutation of 8 by 2) possibilities.

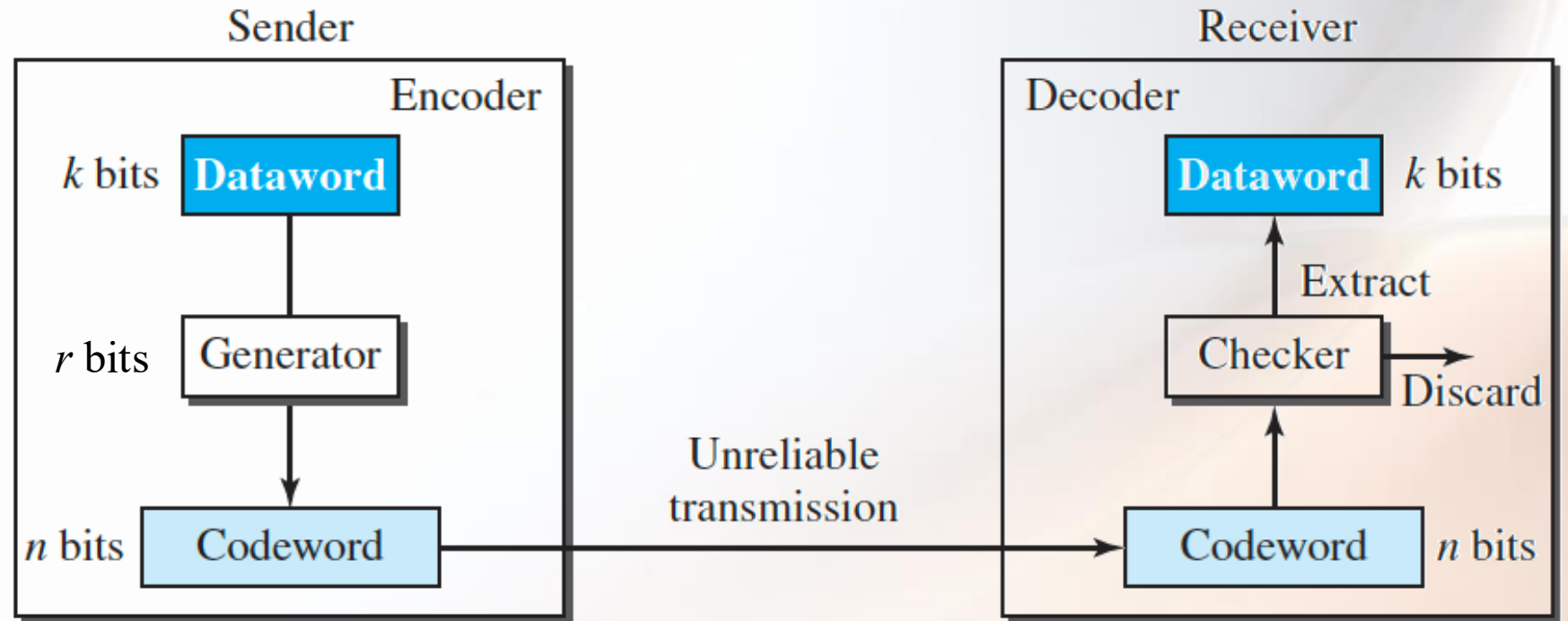
CODING

- Redundancy is achieved through various coding schemes.
 1. The sender adds redundant bits through a process that creates a relationship between the redundant bits and the actual data bits.
 2. The receiver checks the relationships between the two sets of bits to detect errors.

CODING

- The ratio of redundant bits to data bits and the robustness of the process are important factors in any coding scheme.
- We can divide coding schemes into two broad categories: **block coding and convolution coding.**

BLOCK CODING



- 2^k datawords – 2^n codewords
- The block coding process is one-to-one; the same dataword is always encoded as the same codeword.

BLOCK CODING

- If the following two conditions are met, the receiver can detect a change in the original codeword.
 1. The receiver has (or can find) a list of valid codewords.
 2. The original codeword has changed to an invalid one.

BLOCK CODING

<i>Dataword</i>	<i>Codeword</i>	<i>Dataword</i>	<i>Codeword</i>
00	000	10	101
01	011	11	110

- Scenarios (Eg: 011)
 - The receiver receives 011.
 - The codeword is corrupted during transmission, and 111 is received (the leftmost bit is corrupted).
 - The codeword is corrupted during transmission, and 000 is received (the right two bits are corrupted).

CYCLIC CODES

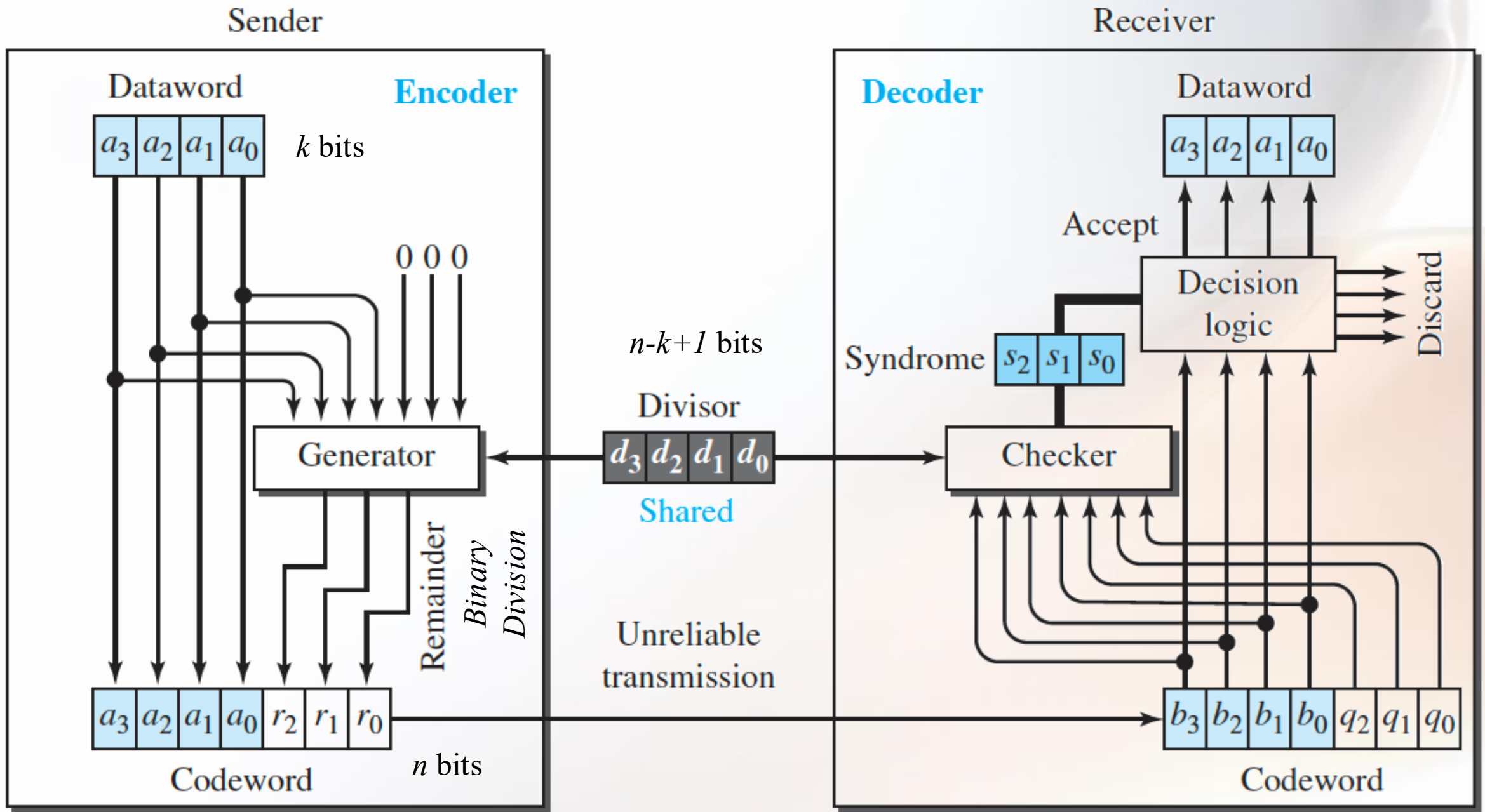
- Cyclic codes are special linear block codes with one extra property.
- In a cyclic code, if a codeword is cyclically shifted (rotated), the result is another codeword.
- For example, if 1011000 is a codeword and we cyclically left-shift, then 0110001 is also a codeword.

$$b_1 = a_0 \quad b_2 = a_1 \quad b_3 = a_2 \quad b_4 = a_3 \quad b_5 = a_4 \quad b_6 = a_5 \quad b_0 = a_6$$

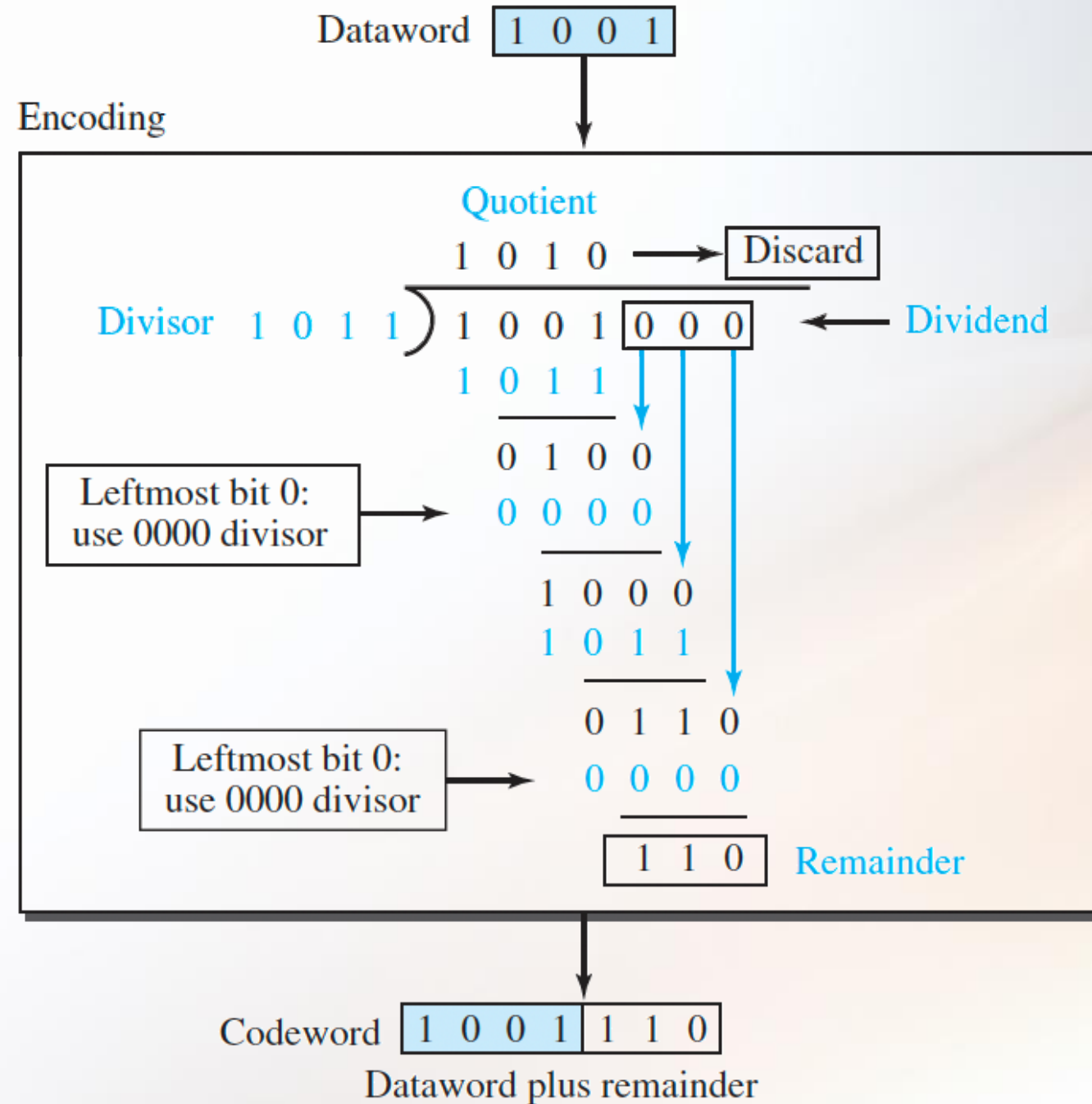
CYCLIC REDUNDANCY CHECK (CRC)

- A subset of cyclic codes which is used in networks such as LANs and WANs.

<i>Dataword</i>	<i>Codeword</i>	<i>Dataword</i>	<i>Codeword</i>
0000	0000 000	1000	1000 101
0001	0001 011	1001	1001 110
0010	0010 110	1010	1010 011
0011	0011 101	1011	1011 000
0100	0100 111	1100	1100 010
0101	0101 100	1101	1101 001
0110	0110 001	1110	1110 100
0111	0111 010	1111	1111 111

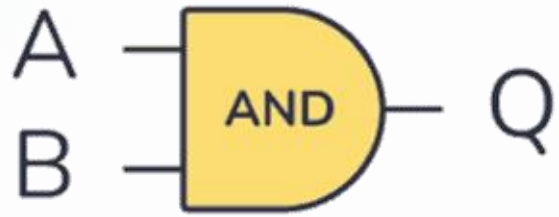


ENCODER



Note:

Multiply: AND
Subtract: XOR

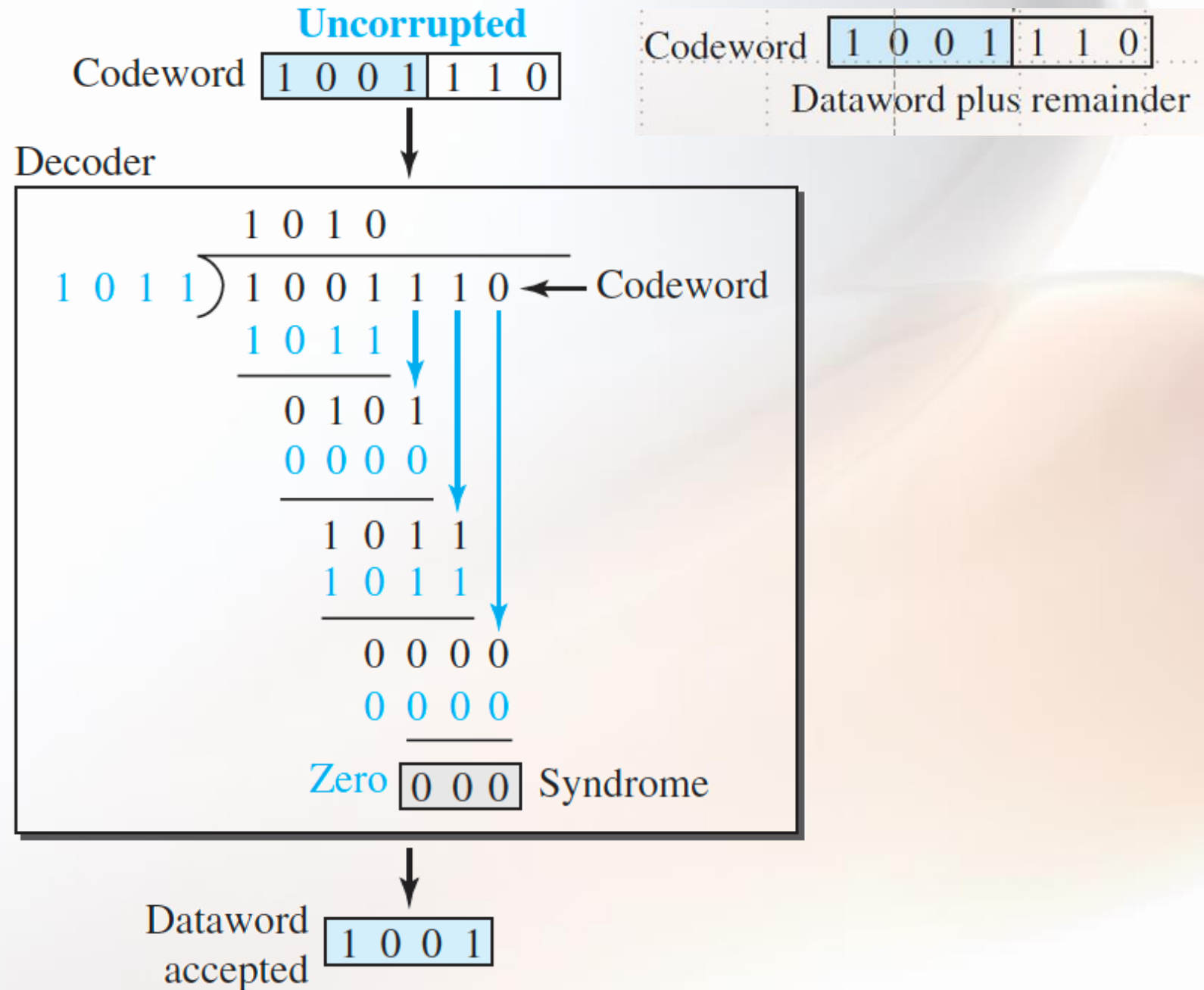


A	B	Q
0	0	0
0	1	0
1	0	0
1	1	1



A	B	Q
0	0	0
0	1	1
1	0	1
1	1	0

DECODER



DECODER

Corrupted
Codeword

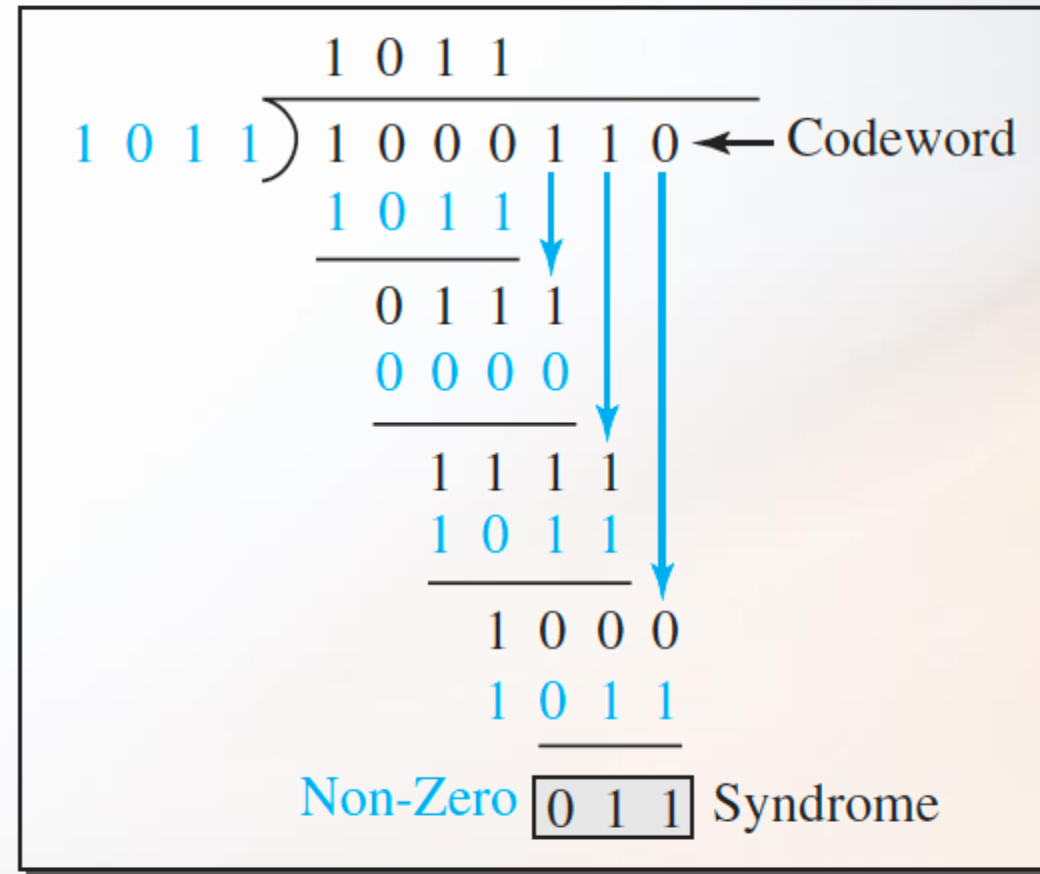
1	0	0	0	1	1	0
---	---	---	---	---	---	---

Codeword

1	0	0	1	1	1	0
---	---	---	---	---	---	---

Dataword plus remainder

Decoder



Dataword discarded

--

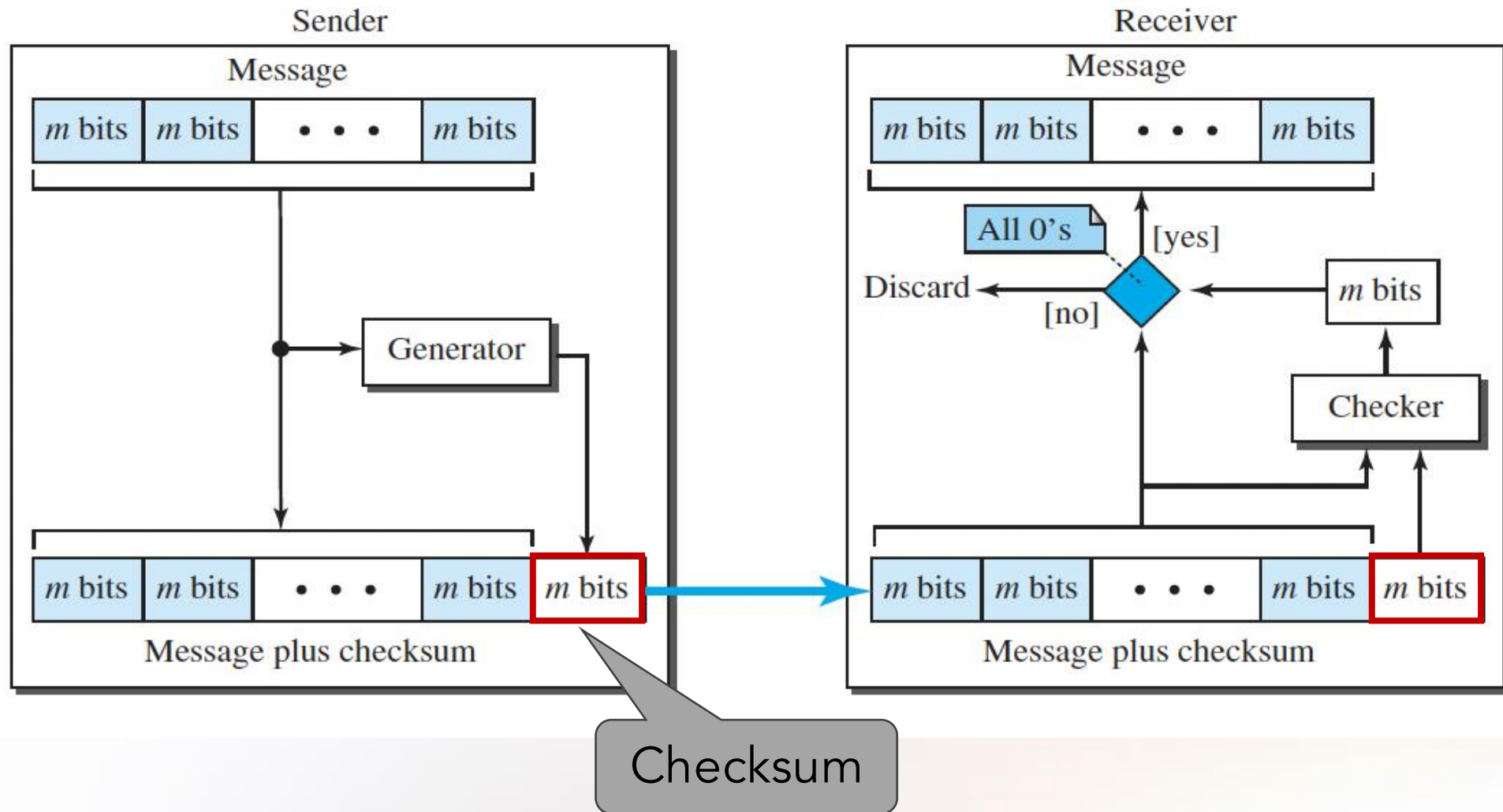
How was 1011 chosen as the divisor?

A generator polynomial that has desirable mathematical properties for detecting errors.

CHECKSUM

- An error-detecting technique that can be applied to a message of any length.
- In the Internet, the checksum technique is mostly used at the network and transport layer rather than the data-link layer.

CHECKSUM



EXAMPLE

- Set of numbers to be sent - $(7, 11, 12, 0, 6)$
- What is actually sent - $(7, 11, 12, 0, 6, 36)$

EXAMPLE

- The example has one major drawback.
- $(7, 11, 12, 0, 6)$, we send $(7, 11, 12, 0, 6, 36)$, where 36 is the sum of the original numbers.
- Each number can be written as a 4-bit word (each is less than 15) except for the sum.

ONE SOLUTION - TO USE ONE'S COMPLEMENT ARITHMETIC

- In this arithmetic, we can represent unsigned numbers between 0 and $2^m - 1$ using only m bits.
- If the number has more than m bits, the extra leftmost bits need to be added to the m rightmost bits (wrapping).

ONE'S COMPLEMENT ARITHMETIC

- In the previous example, the decimal number 36 in binary is **$(100100)_2$** .
- To change it to a 4-bit number we add the extra leftmost bit to the right four bits as shown below.

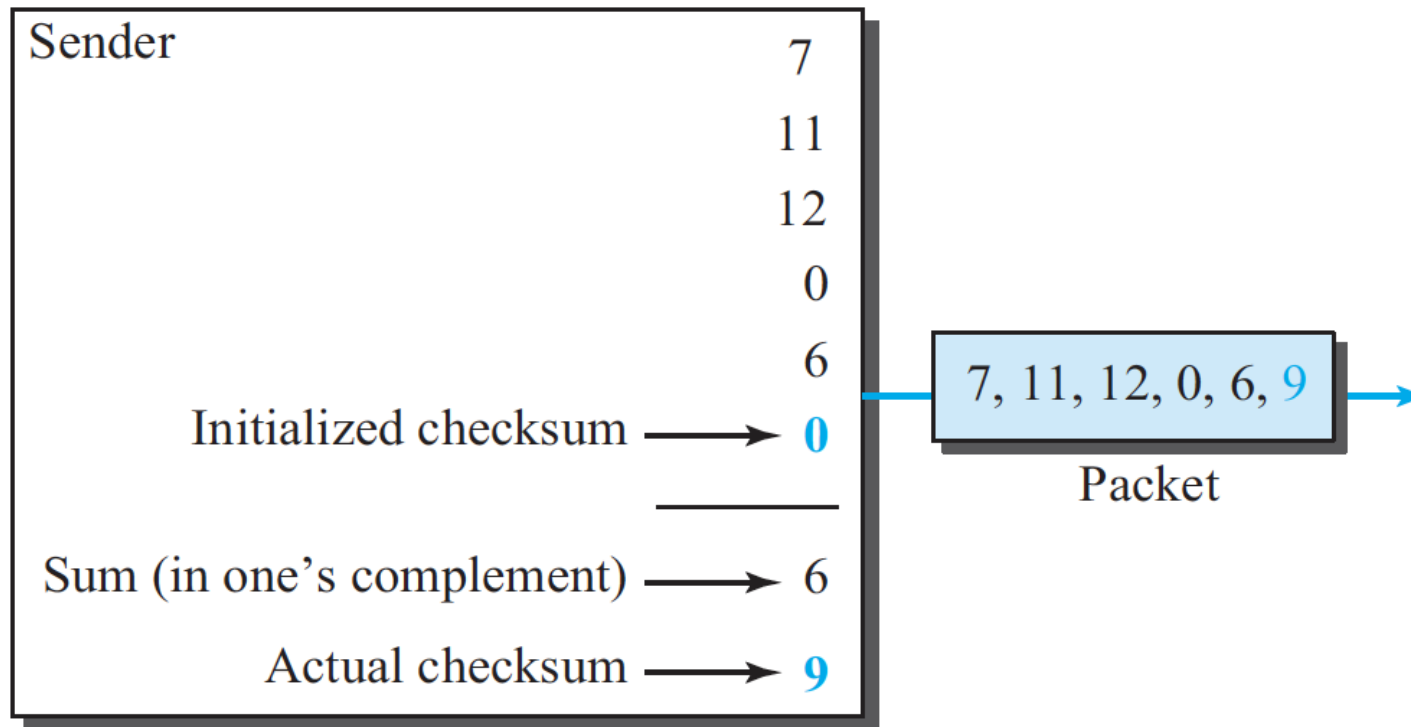
$$(10)_2 + (0100)_2 = (0110)_2 \rightarrow (6)_{10}$$

- Sum = 6
- Checksum = 1's complement of 6 which is $0110 \rightarrow 1001$
- So, checksum = $1001 = 9$

SENDER'S SIDE

Decimal	In Binary	1's Complement Addition
7	0111	0111 + 1011 = 10010 (Decimal: 18). To change to 4 bits: 0010 + 1 = 0011 (Decimal: 3)
11	1011	
12	1100	1100 + 0011 = 1111 (15)
0	0000	0000 + 1111 = 1111 (15)
6	0110	0110 + 1111 = 10101 (Decimal: 21). To change to 4 bits: 0101 + 1 = 0110 (Decimal: 6)
		Checksum: One's complement of 0110: 1001 (Decimal: 9)

SENDER'S SIDE



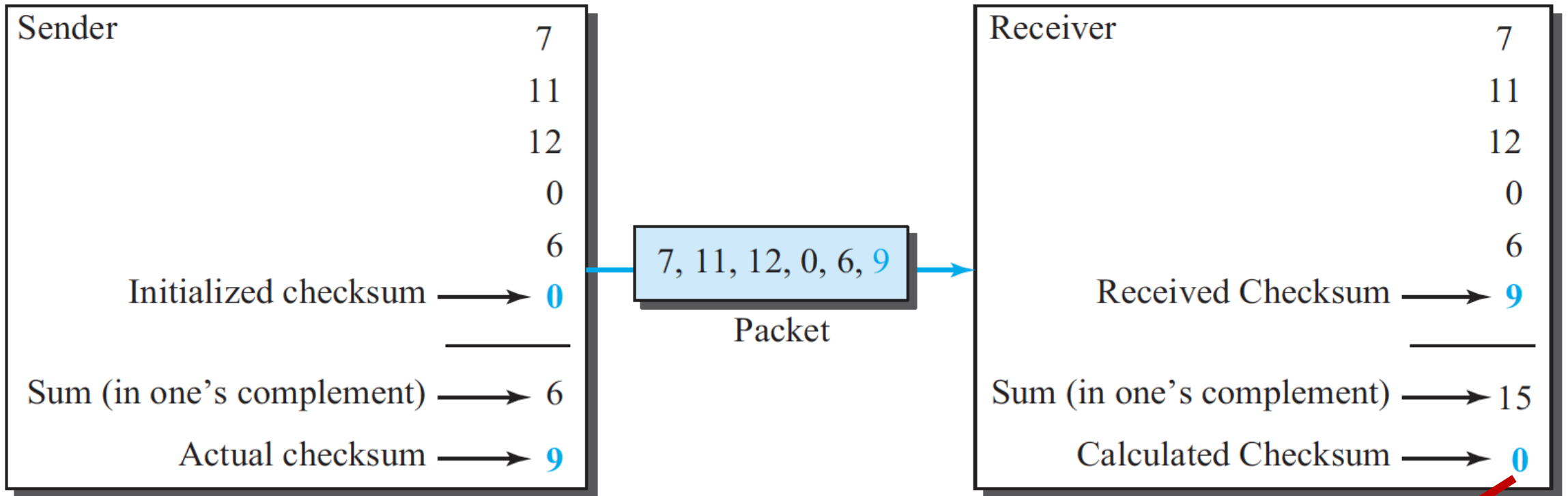
IN ONE'S COMPLEMENT ARITHMETIC

- We have two 0s : positive zero has all m bits set to 0; negative zero has all bits set to 1; they are complement to each other.
- If we add a number with its complement, we get a negative zero (a number with all bits set to 1). (Example: $0110 (6) + 1001 (9) = 1111$)
- When the receiver adds all five numbers (including the checksum), it gets a negative zero.
- The receiver can complement the result again to get a positive zero.

RECEIVER'S SIDE

Decimal	In Binary	1's Complement Addition
7	0111	0111 + 1011 = 10010 (Decimal: 18). To change to 4 bits: 0010+1 = 0011 (Decimal: 3)
11	1011	
12	1100	1100 + 0011 = 1111 (15)
0	0000	0000 + 1111 = 1111 (15)
6	0110	0110 + 1111 = 10101 (Decimal: 21). To change to 4 bits: 0101 + 1 = 0110 (Decimal: 6)
		Checksum: One's complement of 0110: 1001 (Decimal: 9)
		Add Data + Checksum 0110 + 1001 = 1111 (Decimal: 15)
		Complement of 1111 = 0000 (Decimal: 0)

RECEIVER'S SIDE



This shows that data have not been corrupted

INTERNET CHECKSUM

<i>Sender</i>	<i>Receiver</i>
1. The message is divided into 16-bit words. 2. The value of the checksum word is initially set to zero. 3. All words including the checksum are added using one's complement addition. 4. The sum is complemented and becomes the checksum. 5. The checksum is sent with the data.	1. The message and the checksum are received. 2. The message is divided into 16-bit words. 3. All words are added using one's complement addition. 4. The sum is complemented and becomes the new checksum. 5. If the value of the checksum is 0, the message is accepted; otherwise, it is rejected.

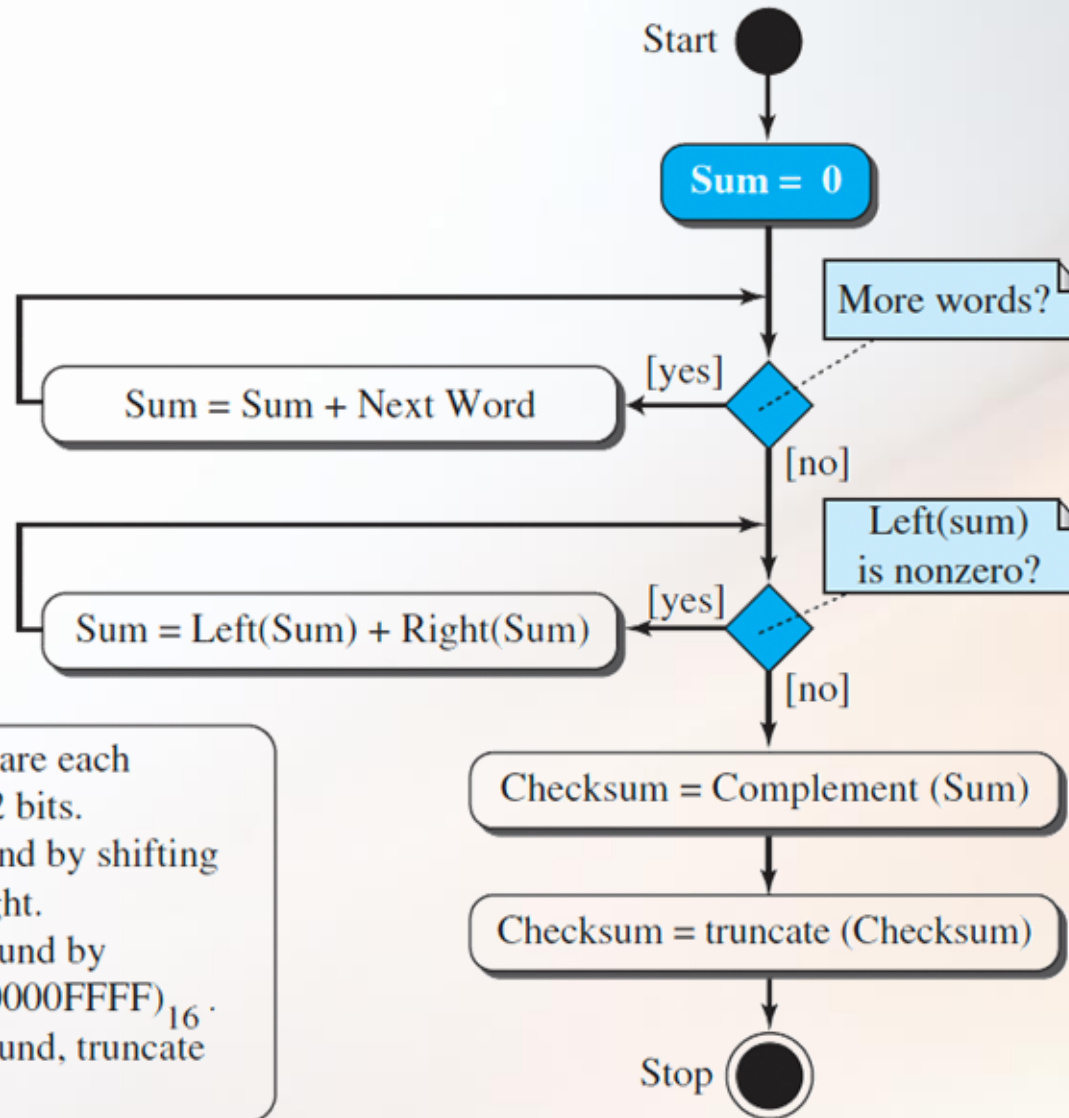
ALGORITHM

Two's complement addition

One's complement conversion

Notes:

- Word and Checksum are each 16 bits, but Sum is 32 bits.
- Left(Sum) can be found by shifting Sum 16 bits to the right.
- Right(Sum) can be found by ANDing Sum with $(0000FFFF)_{16}$.
- After Checksum is found, truncate it to 16 bits.



PERFORMANCE

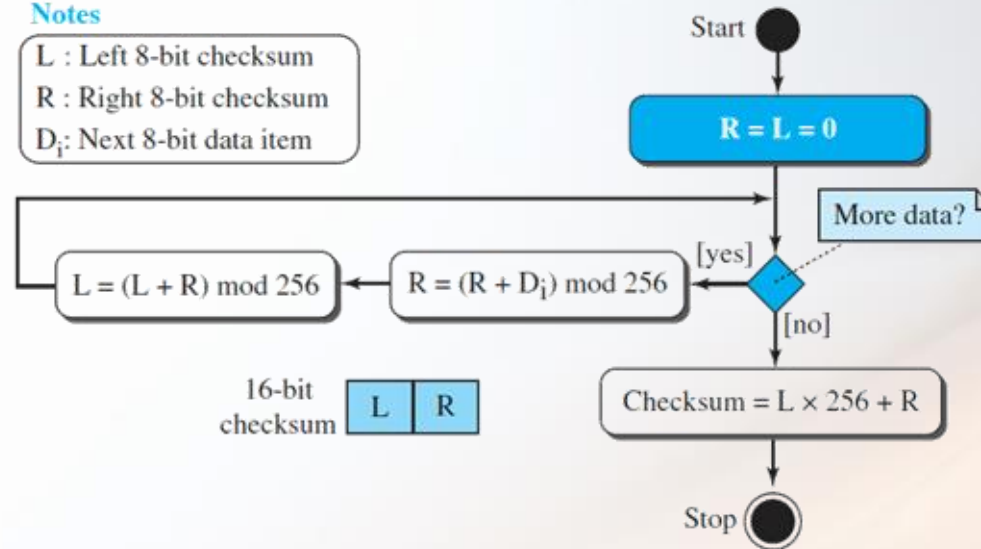
- The traditional checksum uses a small number of bits (16) to detect errors in a message of any size (sometimes thousands of bits).
- However, it is not as strong as the CRC in error-checking capability.
- When sum and checksum values are same, errors are not detected.

OTHER APPROACHES TO CHECKSUM

- Fletcher Checksum

Notes

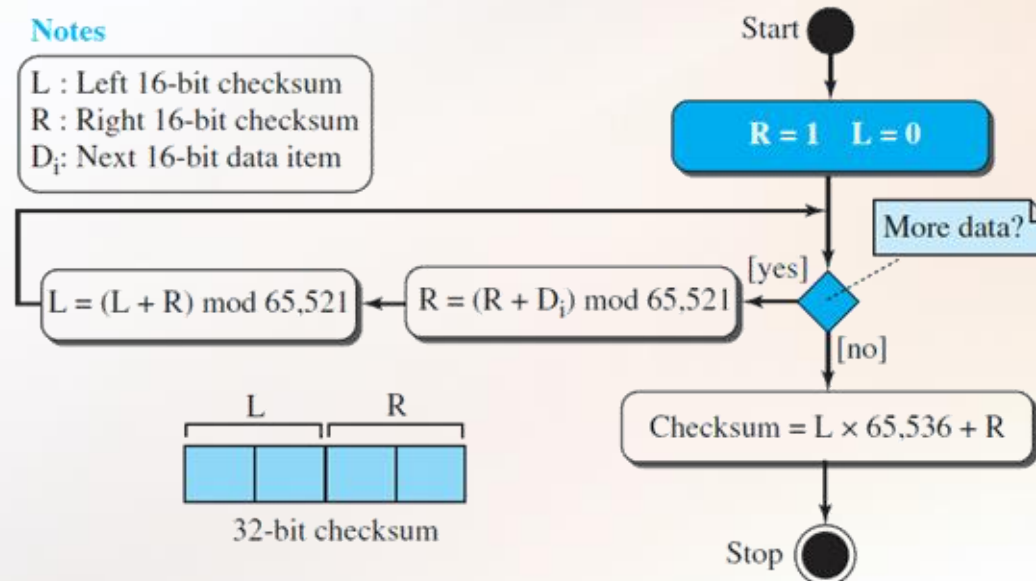
L : Left 8-bit checksum
R : Right 8-bit checksum
 D_i : Next 8-bit data item



- Adler Checksum

Notes

L : Left 16-bit checksum
R : Right 16-bit checksum
 D_i : Next 16-bit data item

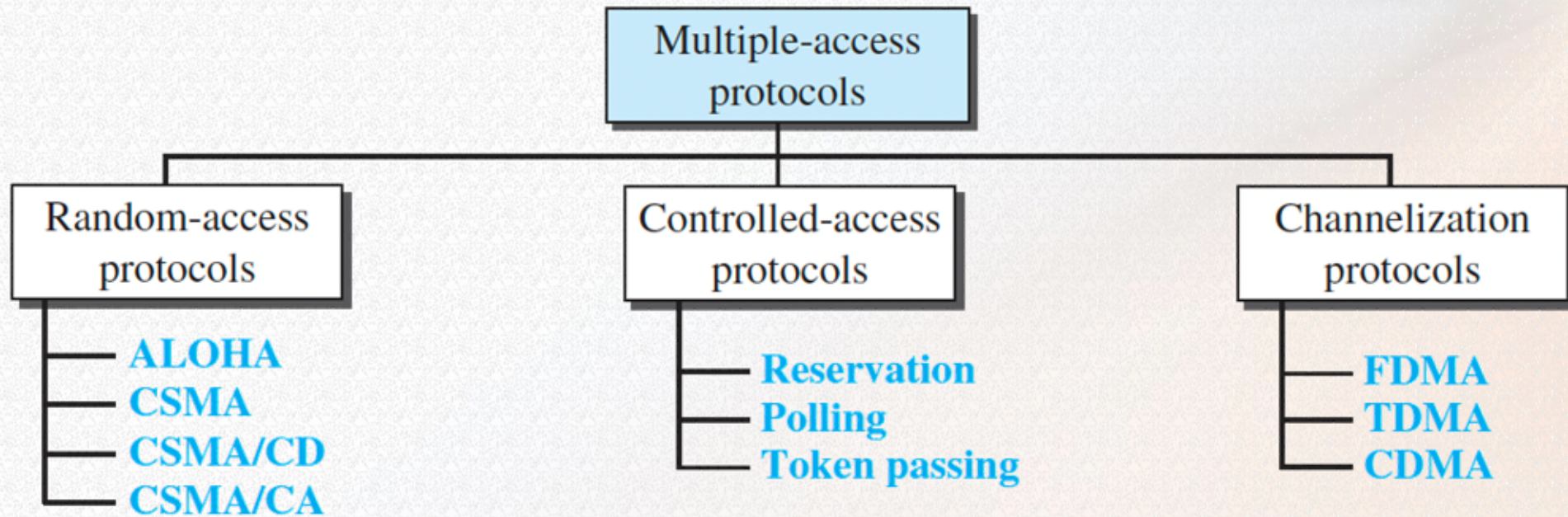


3.2 MAC AND ETHERNET

MEDIA ACCESS CONTROL

- Nodes or stations connected can use a common link, called a *multipoint* or *broadcast link*
- We need a **multiple-access protocol** to coordinate access to the link.
- All of these protocols belong to a sublayer in the data-link layer called ***Media Access Control (MAC)***.

MEDIA ACCESS CONTROL



RANDOM ACCESS

- In **random-access** or **contention** methods, no station is superior to another station and none is assigned control over another.
- At each instance, a station that has data to send uses a procedure (defined by the protocol) to make a decision on whether or not to send.
- This decision depends on the state of the medium (idle or busy).

WHY THE NAME?

1. There is no scheduled time for a station to transmit.
 - Transmission is random among the stations.
2. No rules specify which station should send next.
 - Stations compete with one another to access the medium.





RANDOM ACCESS

If more than one station tries to send - COLLISION



RANDOM ACCESS

- To avoid collision, each station follows a procedure that answers the following questions:
 - When can the station access the medium?
 - What can the station do if the medium is busy?
 - How can the station determine the success or failure of the transmission?
 - What can the station do if there is an access conflict?

RANDOM ACCESS

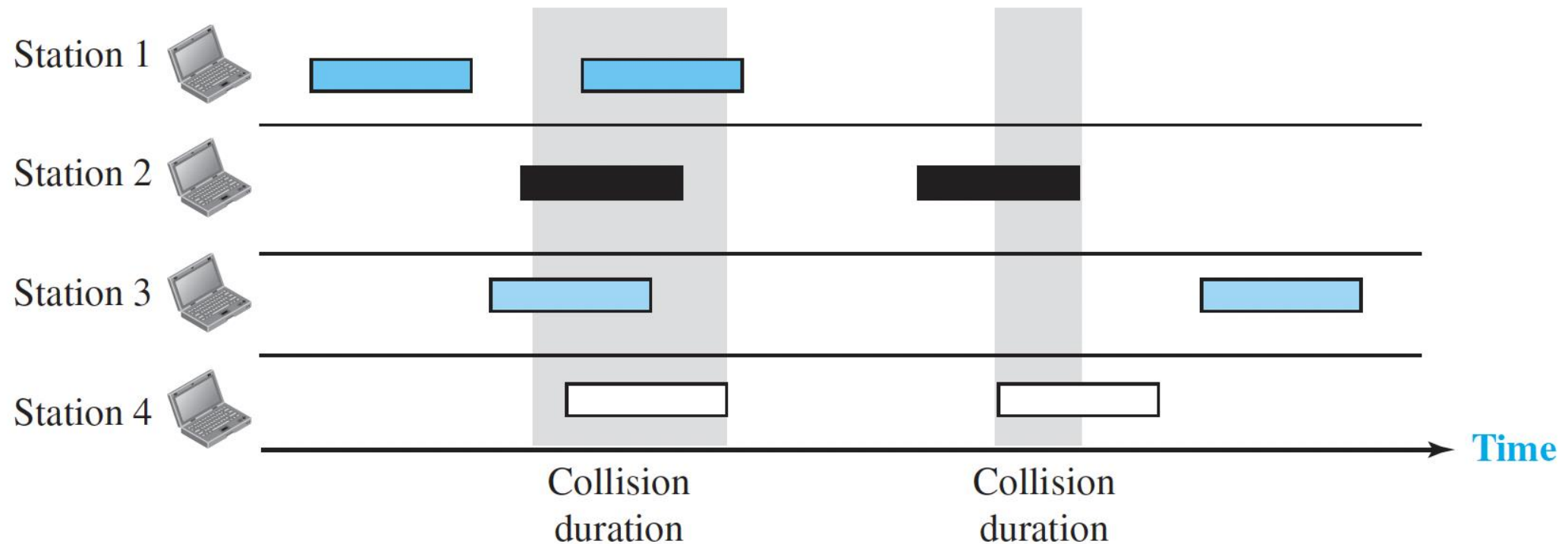
- The random-access methods evolved from *ALOHA*, which used a very simple procedure called *multiple access (MA)*.
- The method was improved and called *carrier sense multiple access (CSMA)*.
- This method later evolved into two parallel methods:
 - *Carrier Sense Multiple Access with Collision Detection (CSMA/CD)*
 - *Carrier Sense Multiple Access with Collision Avoidance (CSMA/CA)*

ALOHA

- Developed at the University of Hawaii in early 1970.
- It was designed for a radio (wireless) LAN, but it can be used on any shared medium.
- It is obvious that there are potential collisions when stations attempt to send data at the same time.

PURE ALOHA – THE ORIGINAL ALOHA PROTOCOL

- Each station sends a frame whenever it has a frames to send.
- But there is only one channel and there is the possibility of collision between frames from different stations.



PURE ALOHA

- This protocol relies on acknowledgments from the receiver.
- When a station sends a frame, it expects the receiver to send an acknowledgment.
 - If the acknowledgment does not arrive **after a TIME-OUT PERIOD**, the station assumes that the frame (or the acknowledgment) has been destroyed and resends the frame.
- Now what happens?
- If all these stations try to resend their frames after the time-out, the frames will collide again.

BACKOFF TIME T_B

- Pure ALOHA dictates that when the time-out period passes, each station **waits a random amount of time** before resending its frame.
- The randomness will help avoid more collisions.
- And this time the **BACKOFF TIME T_B** .

PREVENTING CONGESTION WITH RETRANSMITTED FRAMES

- After a maximum number of retransmission attempts $K_{\max}$, a station must give up and try later.
- K = number of unsuccessful transmissions
- The value of $K_{\max}$ is usually chosen as 15.

TIMEOUT PERIOD

- It is equal to the maximum possible round-trip propagation delay
- Which is twice the amount of time required to send a frame between the two most widely separated stations
- **TIMEOUT PERIOD = $(2 \times T_p)$**
- where T_p = Maximum Propagation Time

BACKOFF TIME T_B

- A random value that normally depends on K
- **K = the number of attempted unsuccessful transmissions.**
- The formula for T_B depends on the implementation.
- One common formula is the **binary exponential backoff**.

BINARY EXPONENTIAL BACKOFF

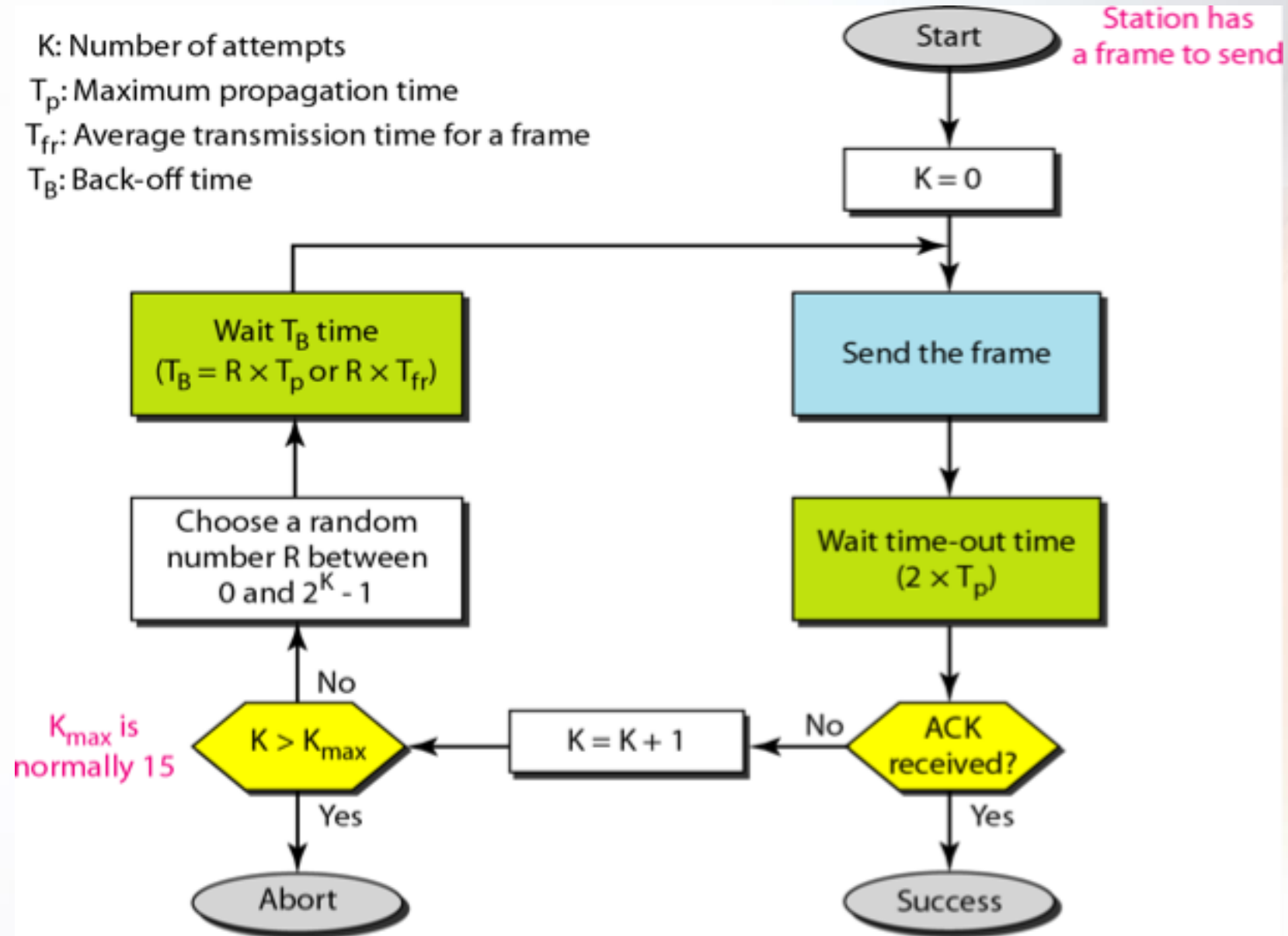
- After **1st collision** ($K = 1$) → device waits randomly between 0 and 1 time slots.
- After **2nd collision** ($K = 2$) → device waits randomly between 0 and 3 time slots.
- After **3rd collision** ($K = 3$) → device waits randomly between 0 and 7 time slots.
- ... and so on.
- Each time, the maximum wait time **doubles (exponential)**.

BACKOFF TIME T_B – BINARY EXPONENTIAL BACKOFF

- $T_B = R \times T_p$ (or) $R \times T_{fr}$
- where $R = 0$ to $2^K - 1$ which is a multiplier that is randomly chosen
- T_p = maximum propagation time
 - $T_p = \text{Distance} / \text{Propagation speed}$
- T_{fr} = the average time required to send out a frame
 - $T_{fr} = \text{Transmission rate} / \text{Frame size}$

PURE ALOHA PROTOCOL

K: Number of attempts
 T_p : Maximum propagation time
 T_{fr} : Average transmission time for a frame
 T_B : Back-off time



PURE ALOHA - VULNERABLE TIME

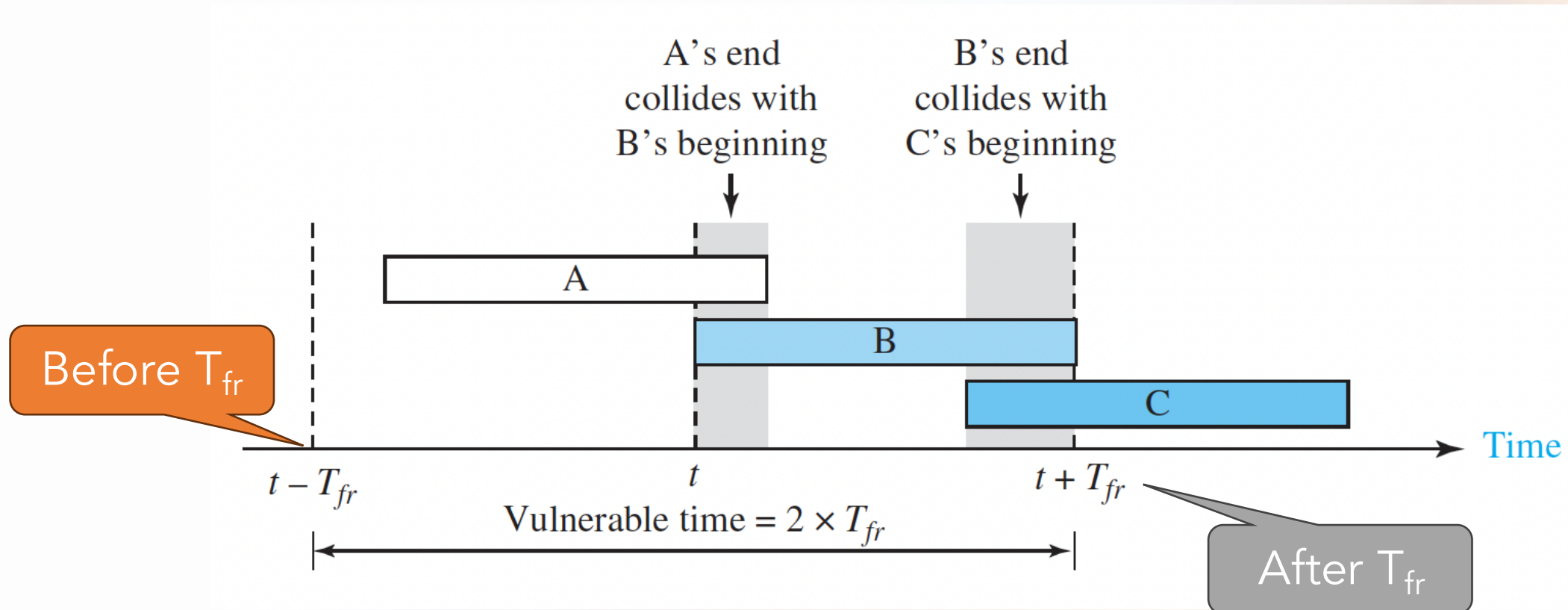


- The length of time in which there is a possibility of collision
- We assume that the stations send fixed-length frames with each frame taking T_{fr} seconds to send

PURE ALOHA - VULNERABLE TIME

T_{fr} = the average time required to send out a frame

- Vulnerable time during which a collision may occur in pure ALOHA is 2 times the frame transmission time T_{fr}



PURE ALOHA – THROUGHPUT

- Average number of successfully transmitted frames, throughput

$$S = G \times e^{-2G}$$

- where G the average number of frames generated by the system during one frame transmission time.

PURE ALOHA – THROUGHPUT

Think of G as “how busy the channel is”

G is very small $\rightarrow$ few frames

G is very large $\rightarrow$ too many frames

Need to strike a balance

- Throughput $S = G \times e^{-2G}$
- Maximum throughput $S_{\max}$ is achieved when $G = \frac{1}{2}$, i.e., $S = \frac{1}{2}e = 0.184$
 - One frame is generated during two frame transmission times
 - 18.4 percent of these frames reach their destination successfully.
- Since the vulnerable time is $2 \times T_{fr}$, that's just enough spacing so that collisions are minimised.
- If a station generates only one frame in this vulnerable time (and no other stations generate a frame during this time), the frame will reach its destination successfully.

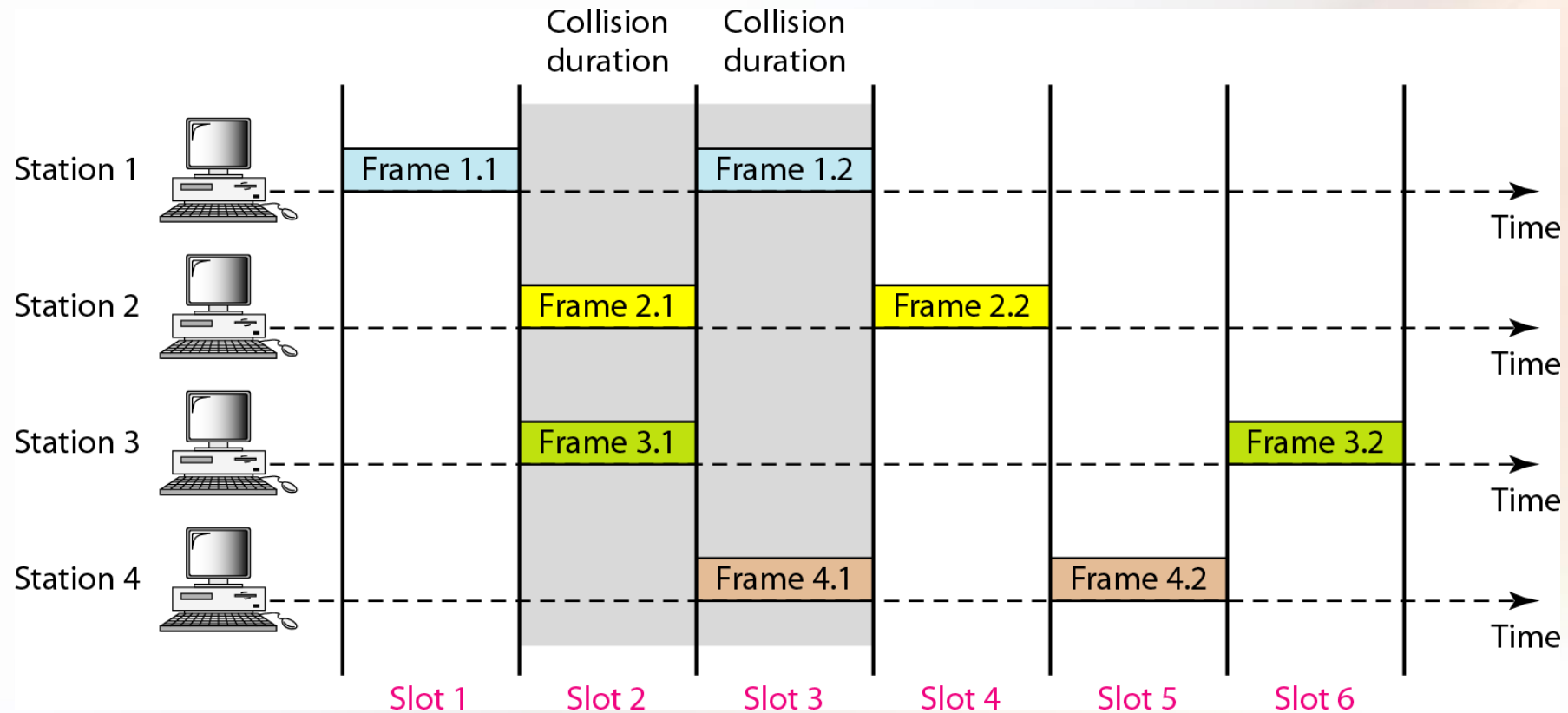
ISSUES WITH PURE ALOHA

- Pure ALOHA has a vulnerable time of $2 \times T_{fr}$.
- Because there is no rule that defines when the station can send.
- A station may send soon after another station has started or just before another station has finished.
- Even at best, **only 18.4% of the channel capacity is used successfully.**
- So, there was a need to improve the efficiency

Solution: Slotted ALOHA

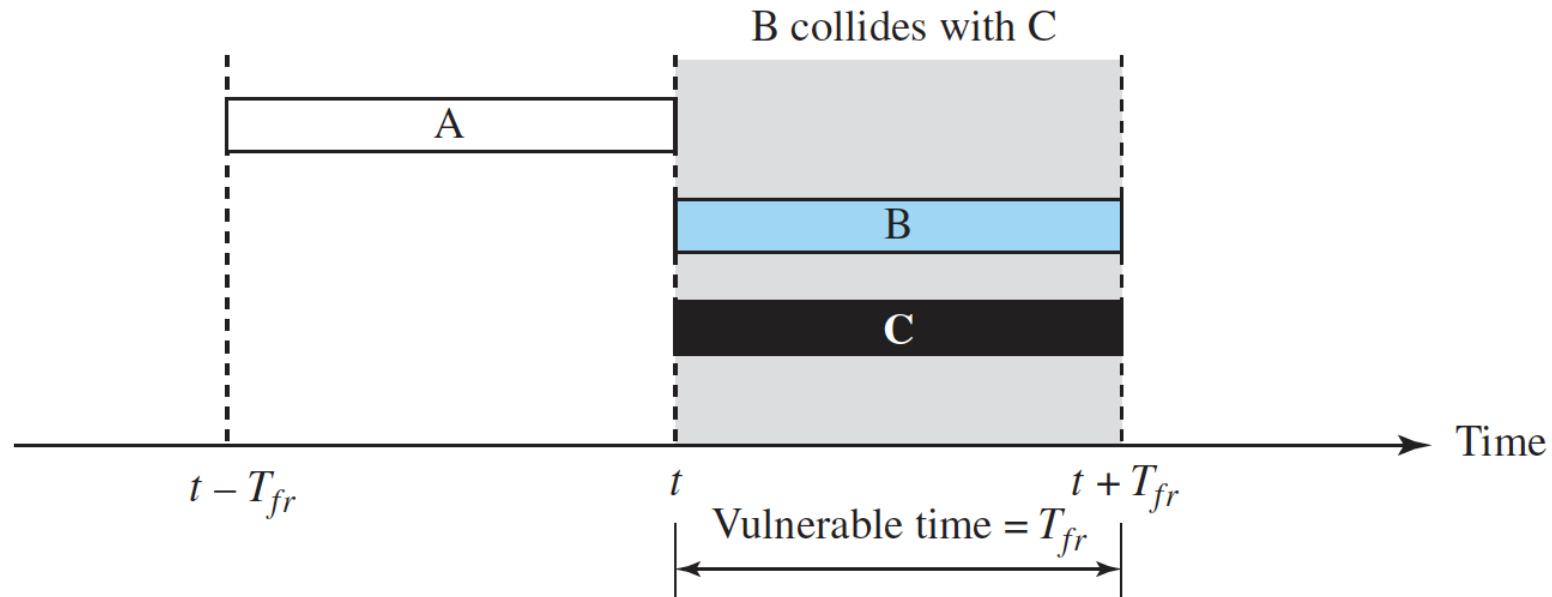
SLOTTED ALOHA

- Divide the time into slots of T_{fr} **seconds** and force the station to send only at the beginning of the time slot.



VULNERABLE TIME FOR SLOTTED ALOHA

- Reduced to half than Pure ALOHA



SLOTTED ALOHA – THROUGHPUT

- Throughput $S = G \times e^{-G}$
- Maximum throughput $S_{\max} = 0.368$ is achieved when $G = 1$
 - One frame is generated during one frame transmission time, which is equal to vulnerable time
 - 36.8 percent of these frames reach their destination successfully.
- If a station generates only one frame in this vulnerable time (and no other stations generate a frame during this time), the frame will reach its destination successfully.

CSMA

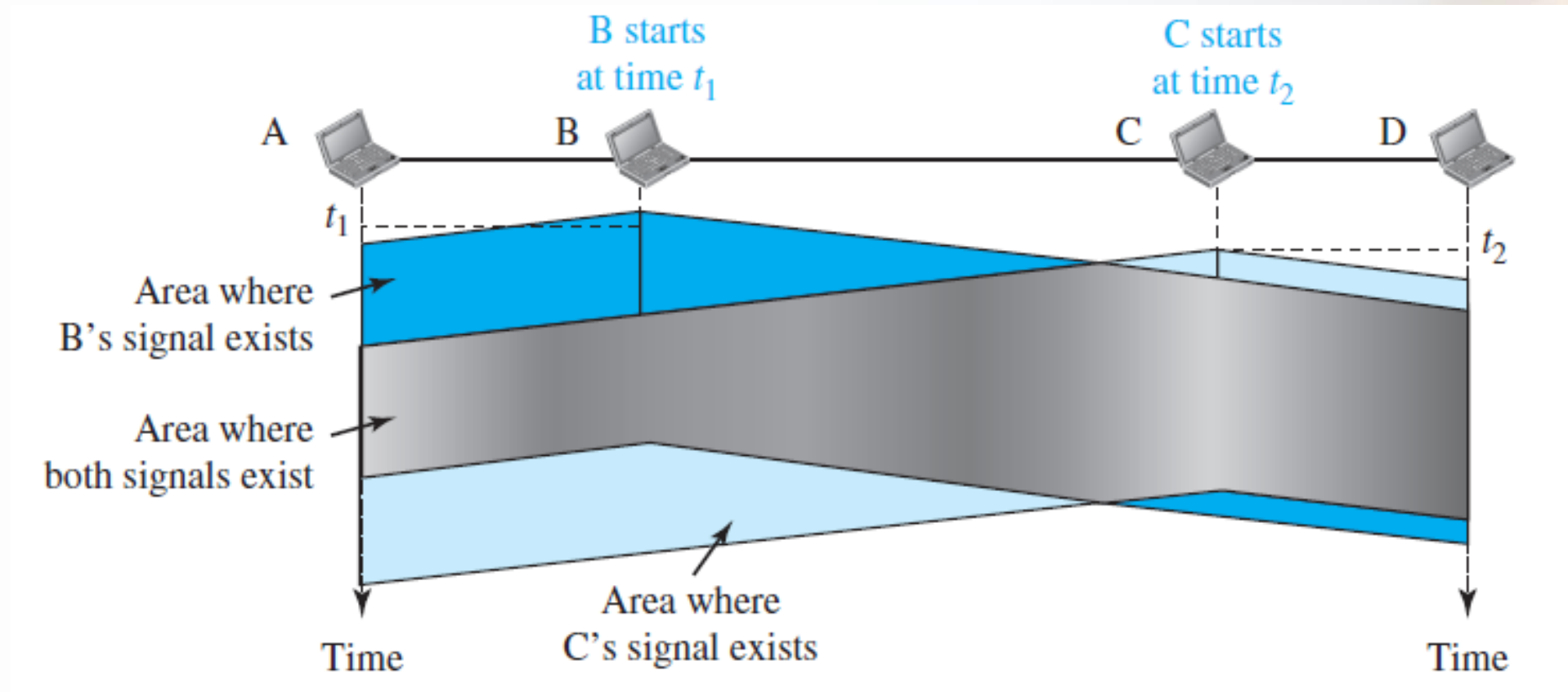
- To minimize the chance of collision and, therefore, increase the performance, the CSMA method was developed.
- **Carrier Sense Multiple Access (CSMA)** is based on the principle "sense before transmit" or "listen before talk."

Wireless
Access Point



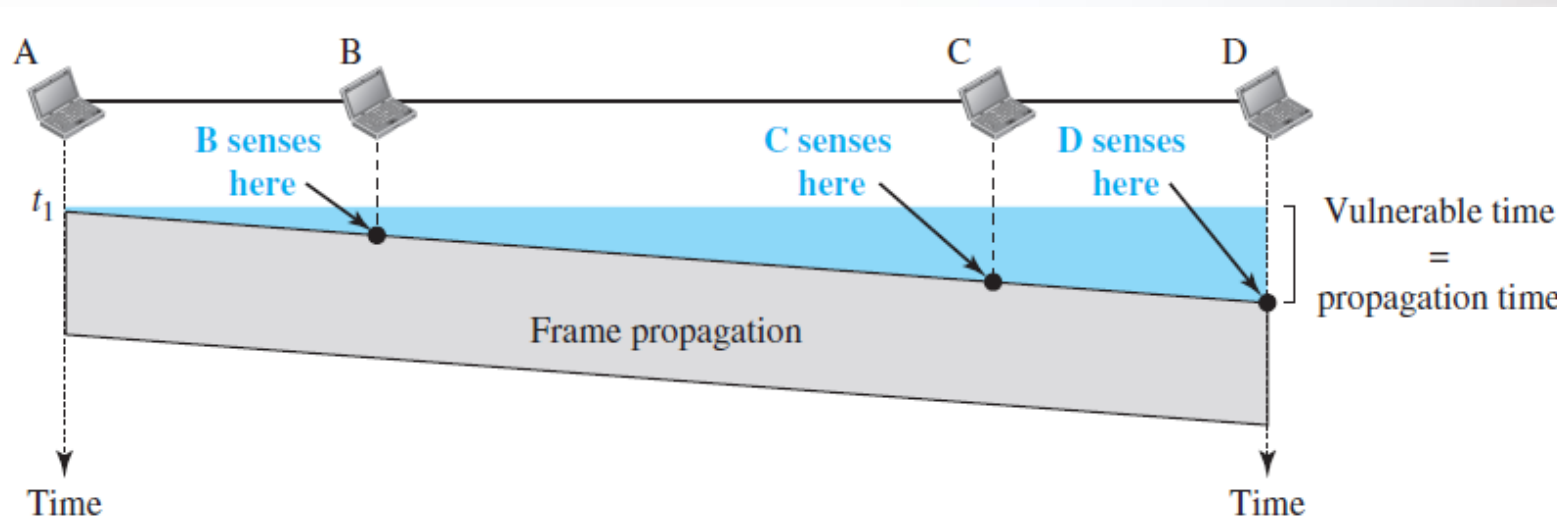
CSMA

- CSMA can reduce the possibility of collision, but it cannot eliminate it.



CSMA - VULNERABLE TIME

- The vulnerable time is the propagation time T_p

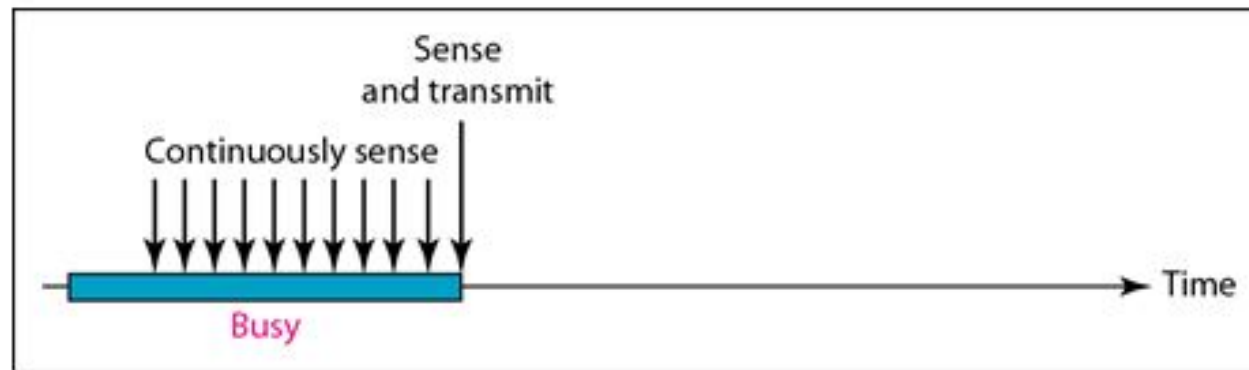
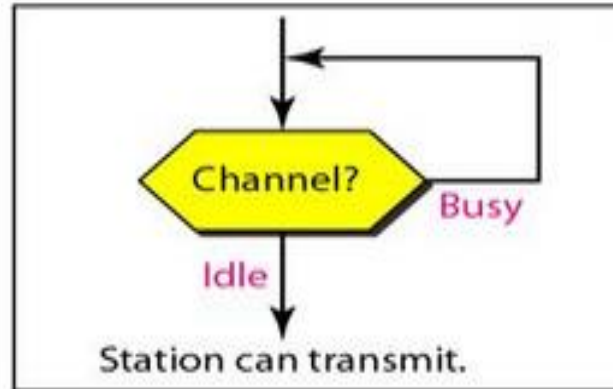


- The leftmost station, A, sends a frame at time t_1 , which reaches the rightmost station, D, at time $t_1 + T_p$.
- The grey area shows the vulnerable area in time and space.

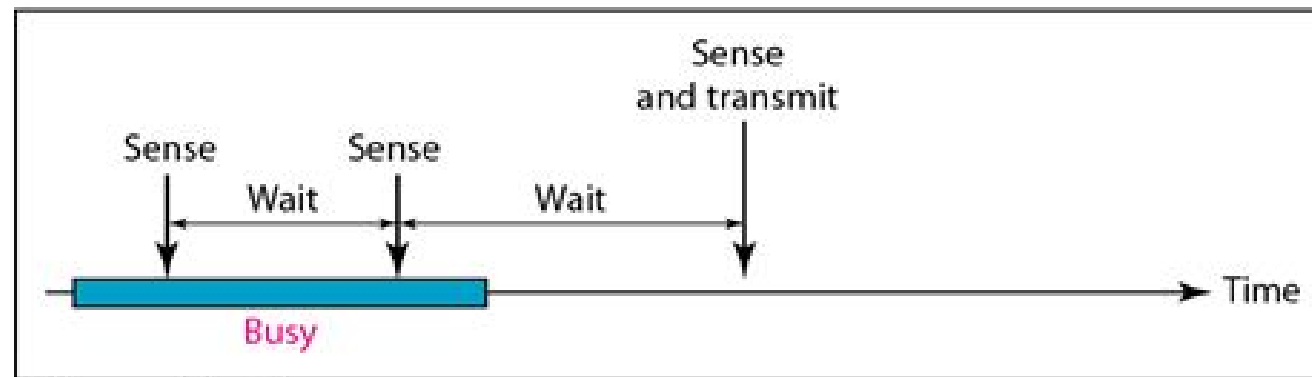
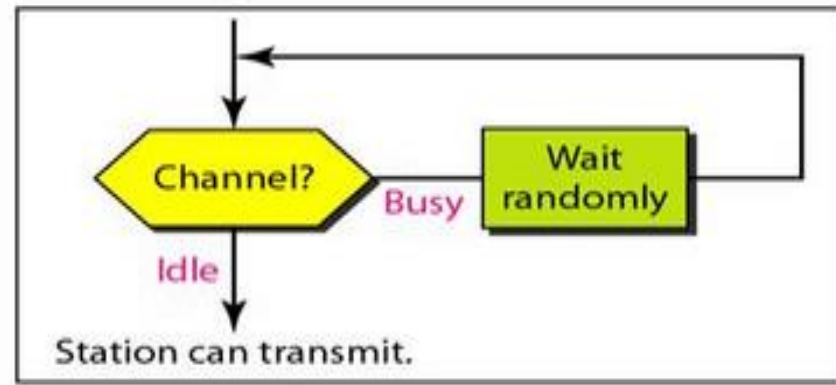
PERSISTENCE METHODS

- What should a station do if the channel is busy/idle?
- Three methods have been devised to answer:
 - 1-persistent method
 - nonpersistent method
 - p -persistent method

1-PERSISTENT METHOD

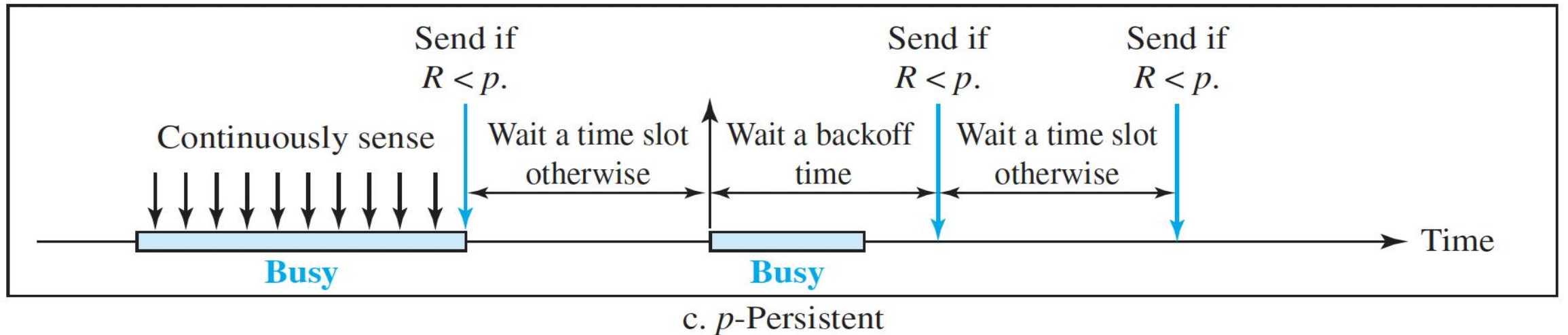


NONPERSISTENT METHOD

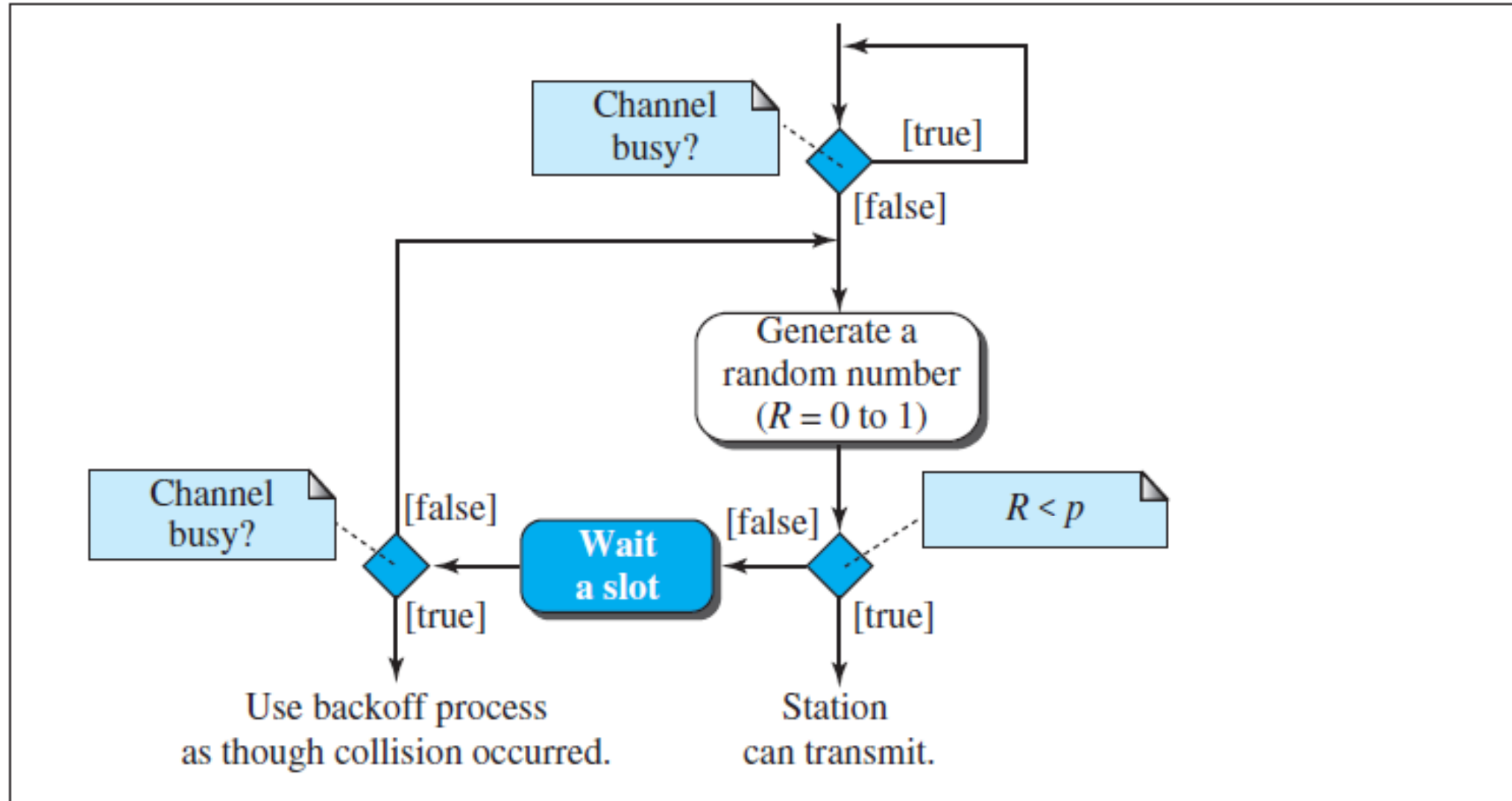


p -PERSISTENT METHOD

p – probability of transmitting; fixed by the protocol

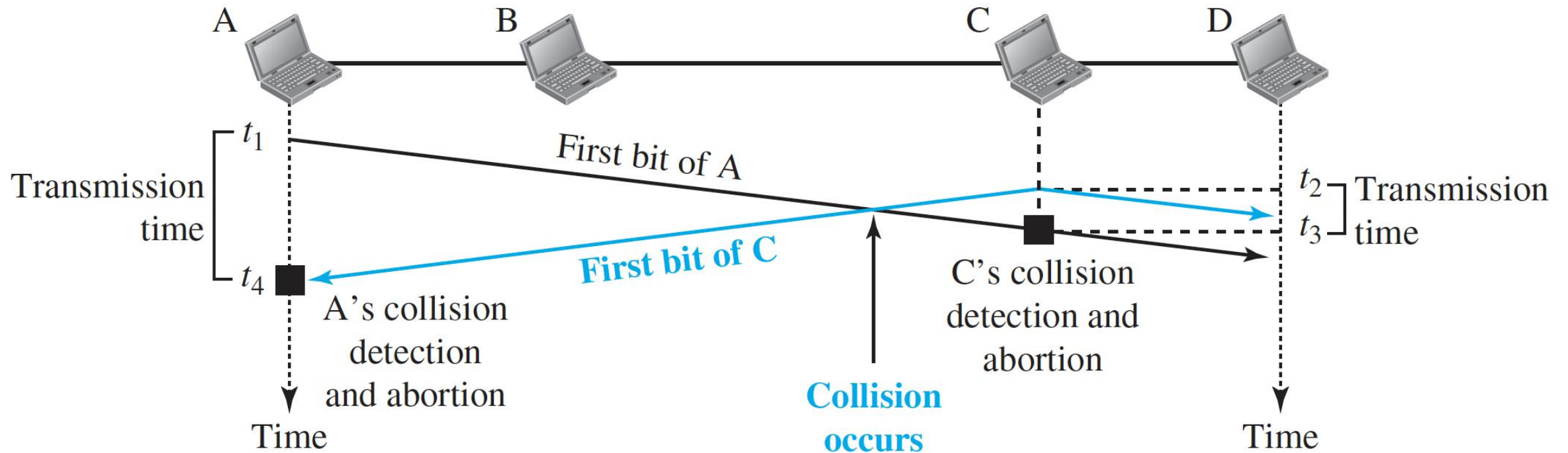


p -PERSISTENT METHOD



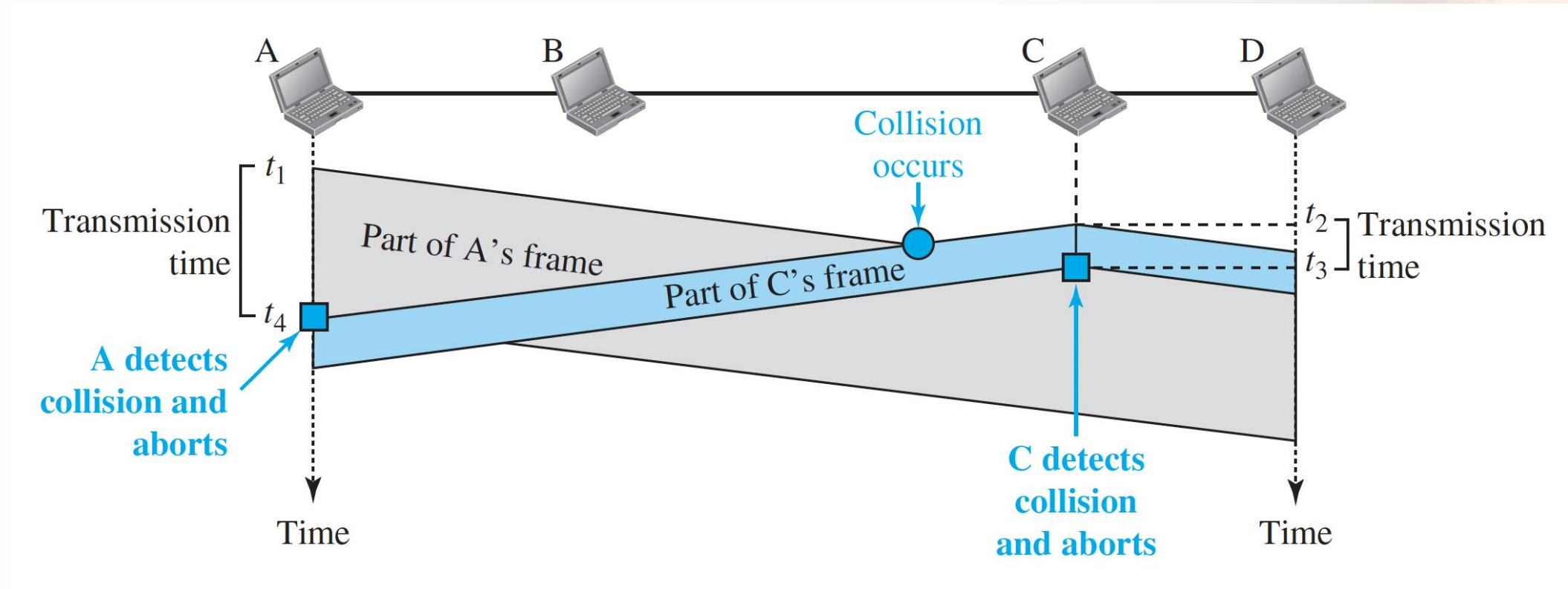
CSMA/CD

- Collision of the first bits in CSMA/CD



CSMA/CD

- Collision and abortion in CSMA/CD



COLLISION DETECTION MUST HAPPEN BEFORE THE FRAME ENDS

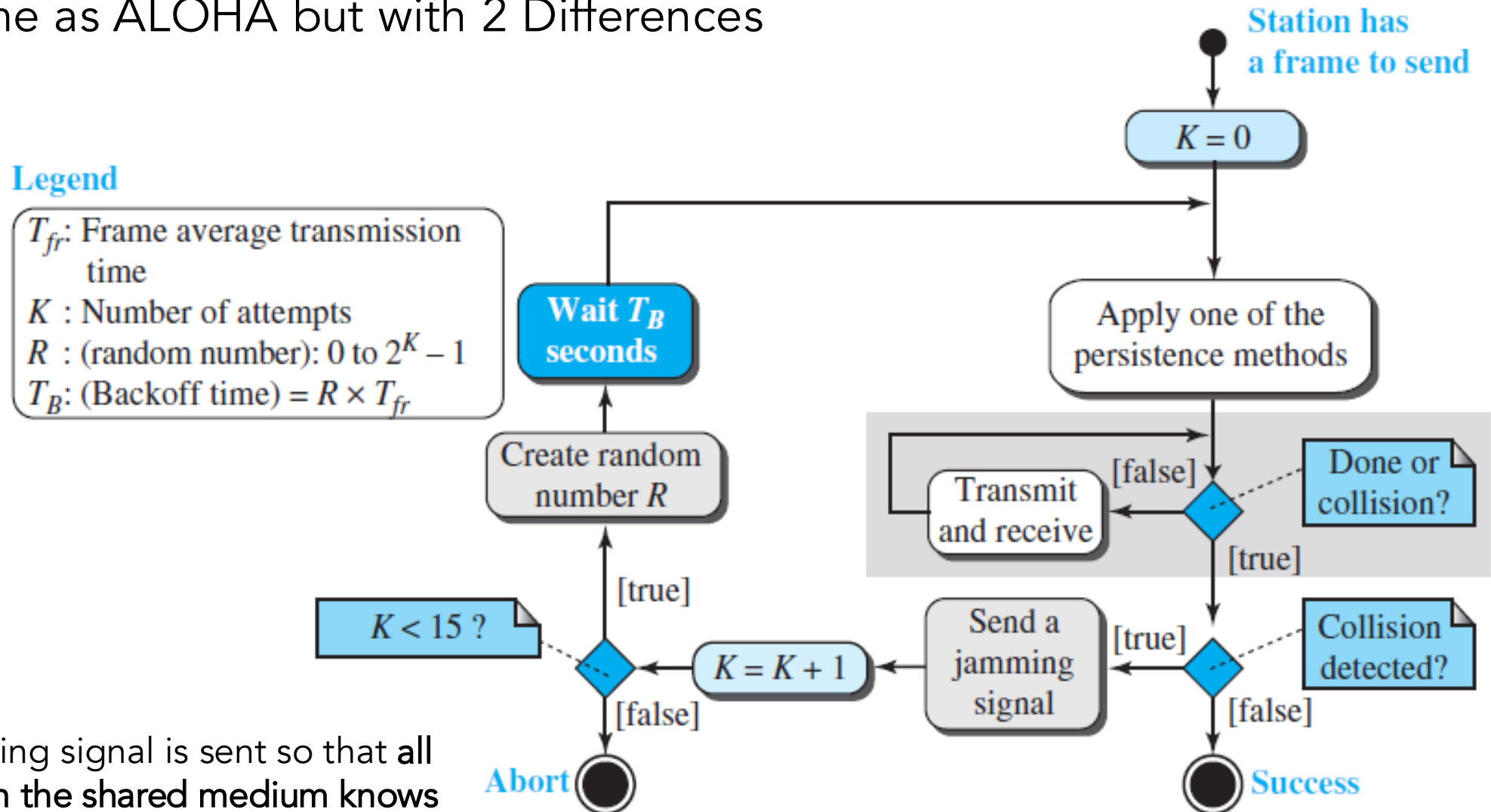
- When a station transmits, its bits take some **time to propagate** across the cable (propagation delay T_p)
- A collision may happen **far away** (for example, station A and station D transmitting almost simultaneously).
- If the sender finishes sending **all bits** before the collision has time to propagate back, the sender won't notice the collision.
- That would be disastrous — the sender would think the frame was delivered, but it was actually destroyed.

MINIMUM FRAME SIZE

- Before sending the last bit of the frame, the sending station must detect a collision, if any, and abort transmission.
 - Once the entire frame is sent, station does not keep a copy and does not monitor for collision.
- The frame transmission time T_{fr} must be at least **two times** the maximum propagation time T_p .
- So, the requirement is that the first station must still be transmitting after $2T_p$.

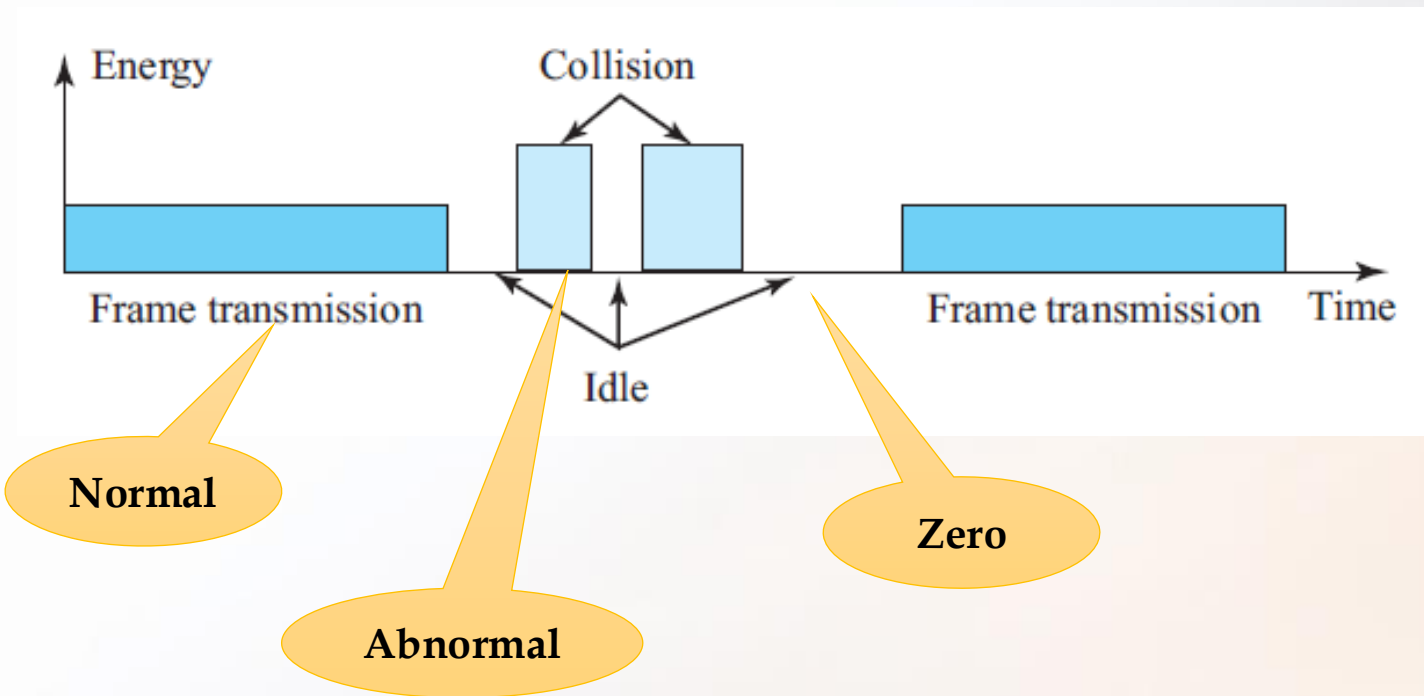
CSMA/CD ALGORITHM WITH EXPONENTIAL BACKOFF

Same as ALOHA but with 2 Differences



The jamming signal is sent so that all stations on the shared medium know that a collision has occurred.

ENERGY LEVEL



THROUGHPUT

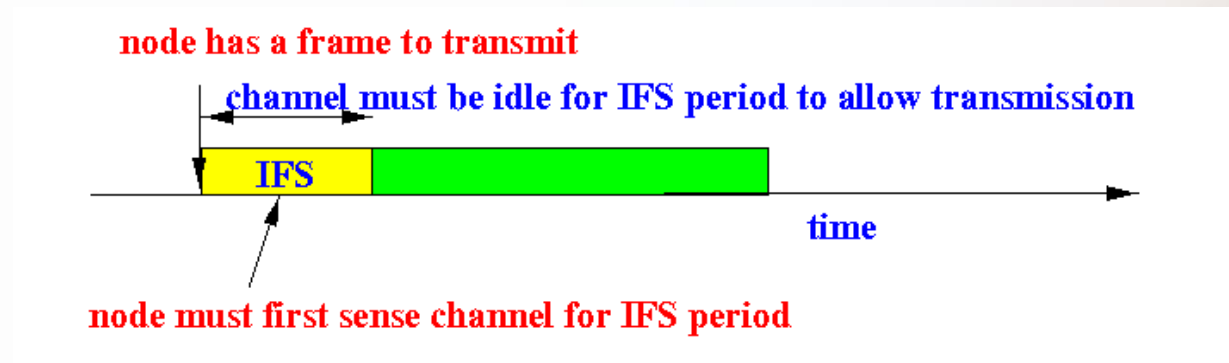
- The maximum throughput occurs at a different value of G and is based on the persistence method and the value of p in the p -persistent approach.
 - G is the average number of frames generated by the system during one frame transmission time
- For the 1-persistent method, the maximum throughput is around 50 percent when $G = 1$.
- For the nonpersistent method, the maximum throughput can go up to 90 percent when G is between 3 and 8.

CSMA/CA

- **Carrier Sense Multiple Access with Collision Avoidance (CSMA/CA)** was invented for wireless networks.
- Collisions are avoided through the use of CSMA/CA's three strategies:
 - the interframe space
 - the contention window
 - acknowledgments

INTERFRAME SPACE (IFS)

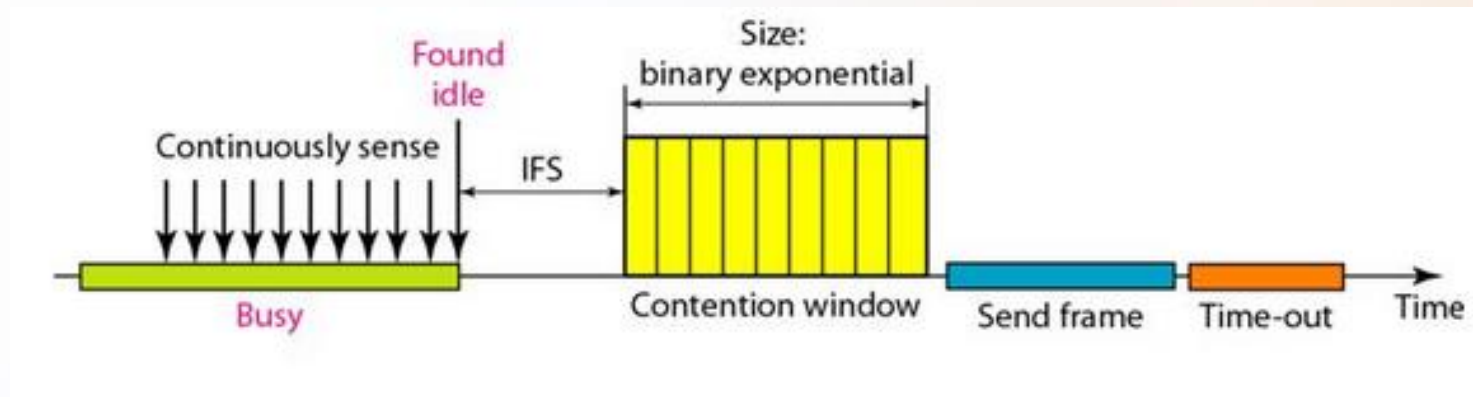
- When an idle channel is found, the station does not send immediately but waits for a period of time called the *interframe space*



- After waiting an IFS time, if the channel is still idle, the station can send, but it still needs to wait a time equal to the contention window

CONTENTION WINDOW

- A station that is ready to send chooses a random number of slots as its wait time.
- The number of slots in the window changes according to the binary exponential backoff strategy.



- The station needs to sense the channel after each time slot.

ACKNOWLEDGEMENT

- There still may be a collision resulting in destroyed data.
- Also, the data may be corrupted during the transmission.
- The positive acknowledgment and the time-out timer can help guarantee that the receiver has received the frame.

Legend

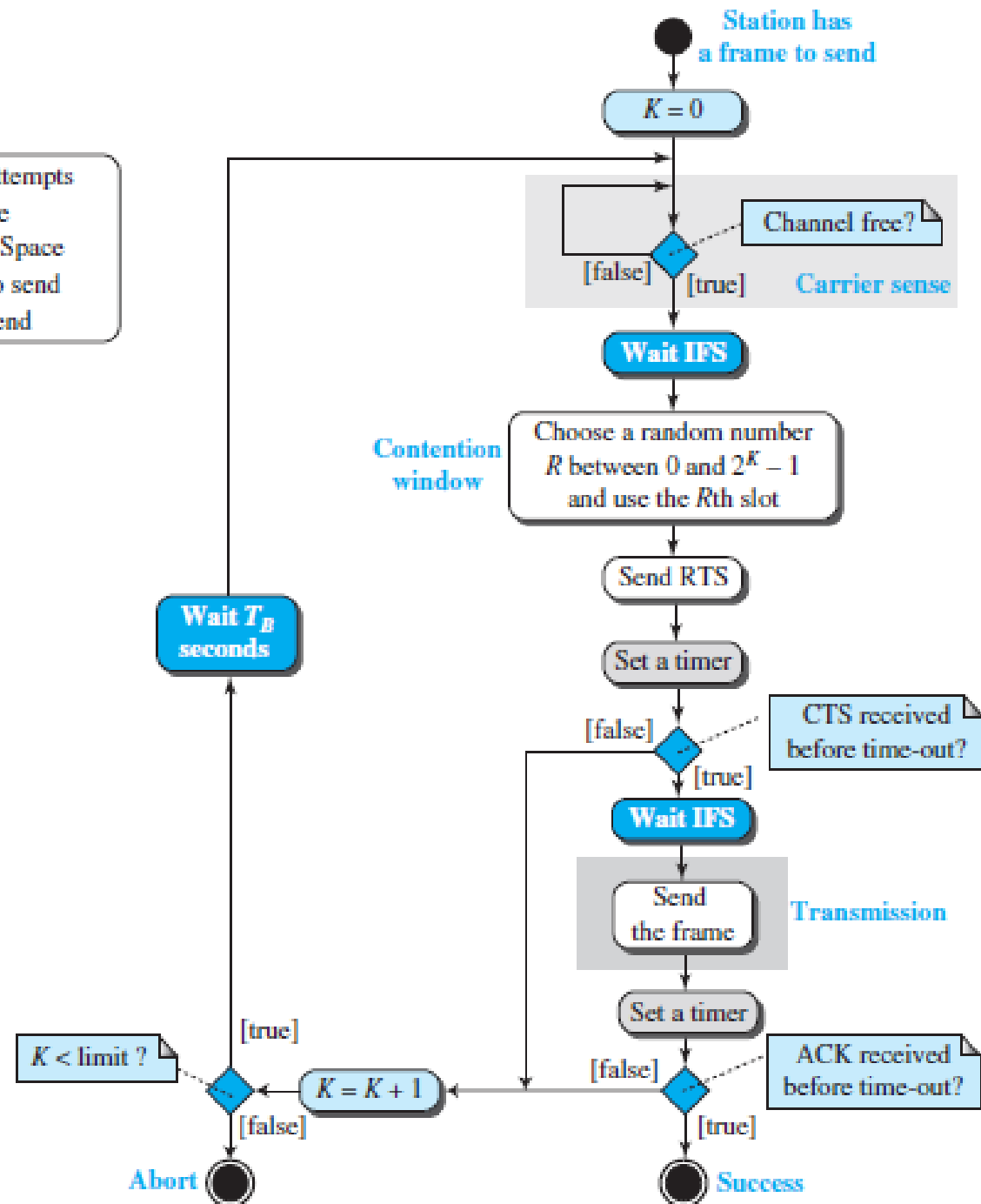
K : Number of attempts

T_B : Backoff time

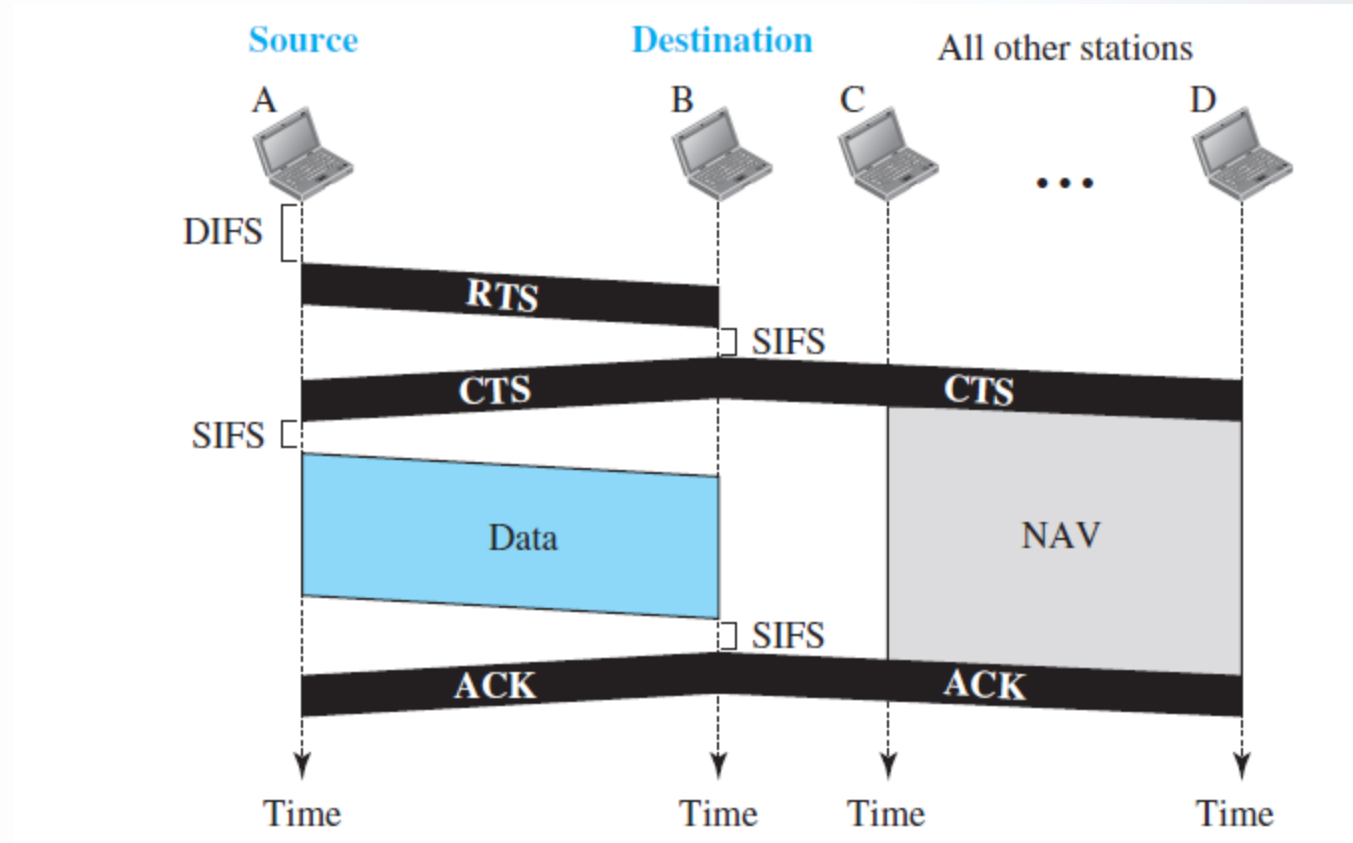
IFS: Interframe Space

RTS: Request to send

CTS: Clear to send



FRAME EXCHANGE TIMELINE



- DIFS – Distributed Coordination Function IFS, RTS - Request To Send, SIFS - Short IFS, CTS - Clear to Send, NAV - Network Allocation Vector, ACK - Acknowledgement

CSMA/CA

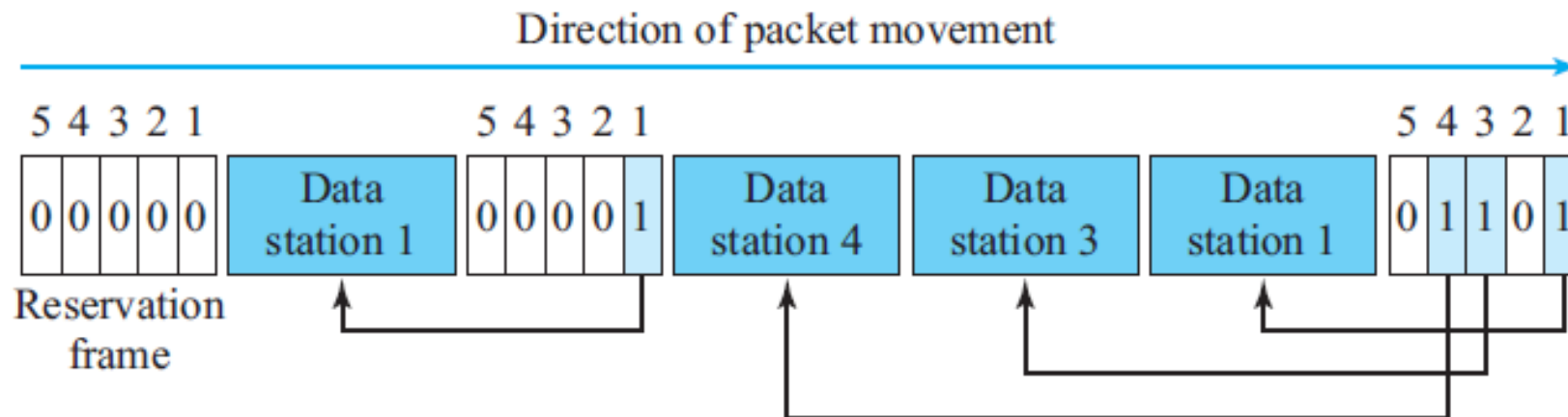
- Collision During Handshaking?
- Hidden-Station Problem?

CONTROLLED ACCESS

- In **controlled access**, the stations consult one another to find which station has the right to send.
- A station cannot send unless it has been authorized by other stations.

RESERVATION

- In the **reservation** method, a station needs to make a reservation before sending data.
- Time is divided into intervals.
 - In each interval, a reservation frame precedes the data frames sent in that interval.

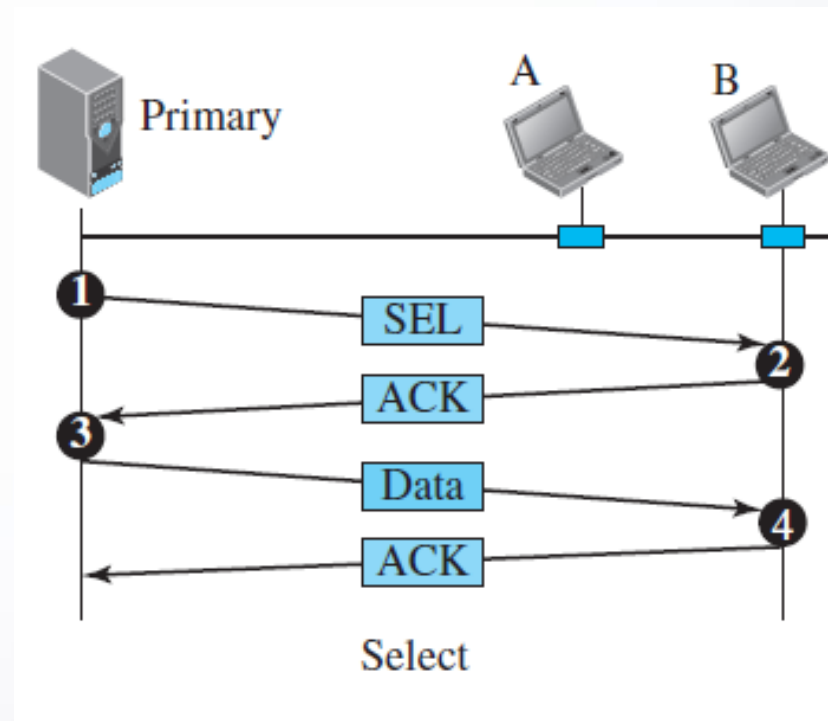


POLLING

- **Polling** works with topologies in which one device is designated as a *primary station* and the other devices are *secondary stations*.
- All data exchanges must be made through the primary device even when the ultimate destination is a secondary device.
- The primary device controls the link; the secondary devices follow its instructions.
 - Primary device is always the initiator of a session

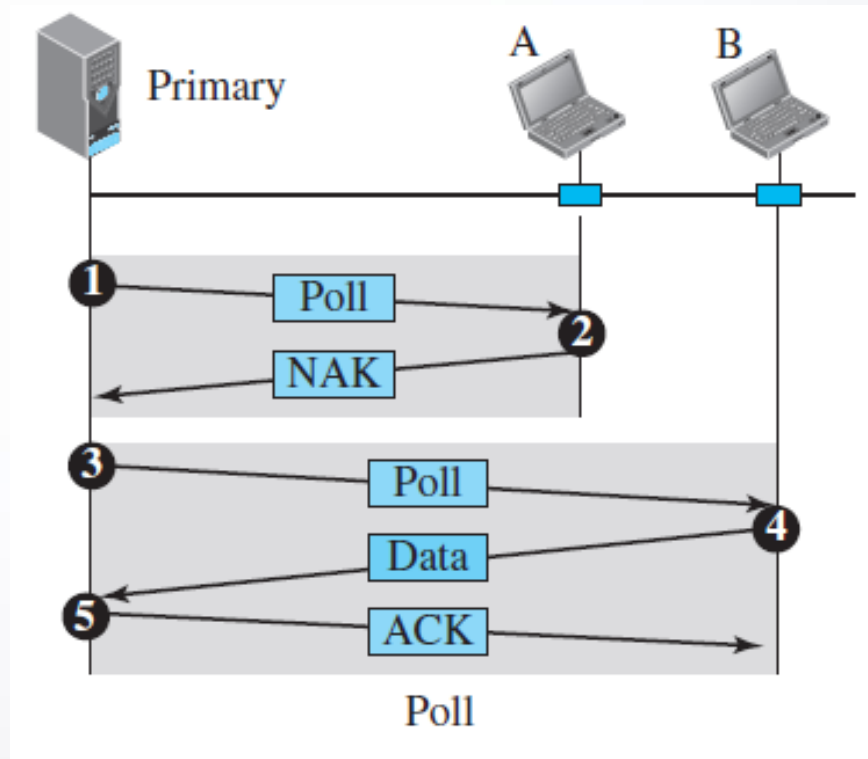
SELECT FUNCTION

- Used whenever the primary device has something to send.



POLL FUNCTION

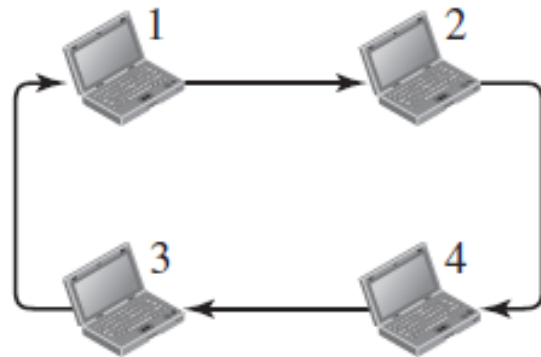
- Used by the primary device to solicit transmissions from the secondary devices.



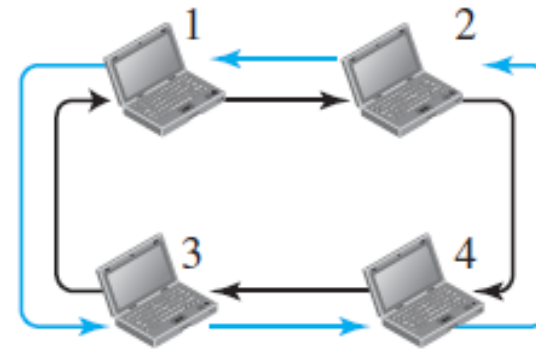
TOKEN PASSING

- The stations in a network are organized in a logical ring; for each station, there is a *predecessor* and a *successor*.
- The current station is the one that is accessing the channel now.
 - The right to access has been passed from the predecessor and it will be passed to the successor from here.
- A special packet called a **token** circulates through the ring.
- The possession of the token gives the station the right to access the channel and send its data.
- Token management is important - how long the token can be held, lost tokens, station priorities based on data.

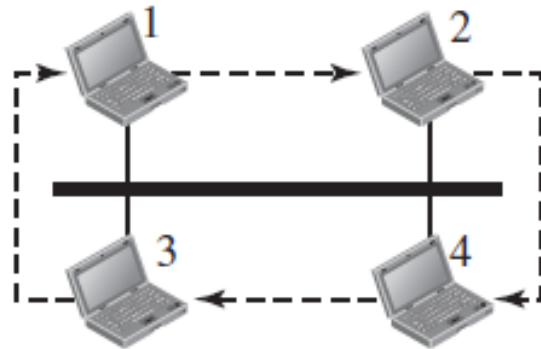
LOGICAL RING



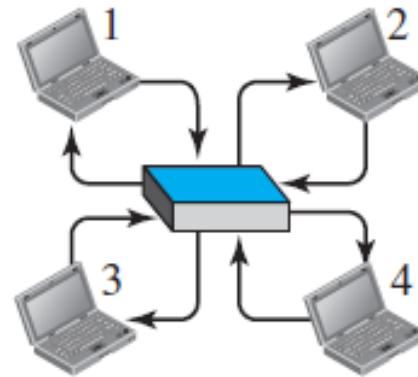
a. Physical ring



b. Dual ring



c. Bus ring



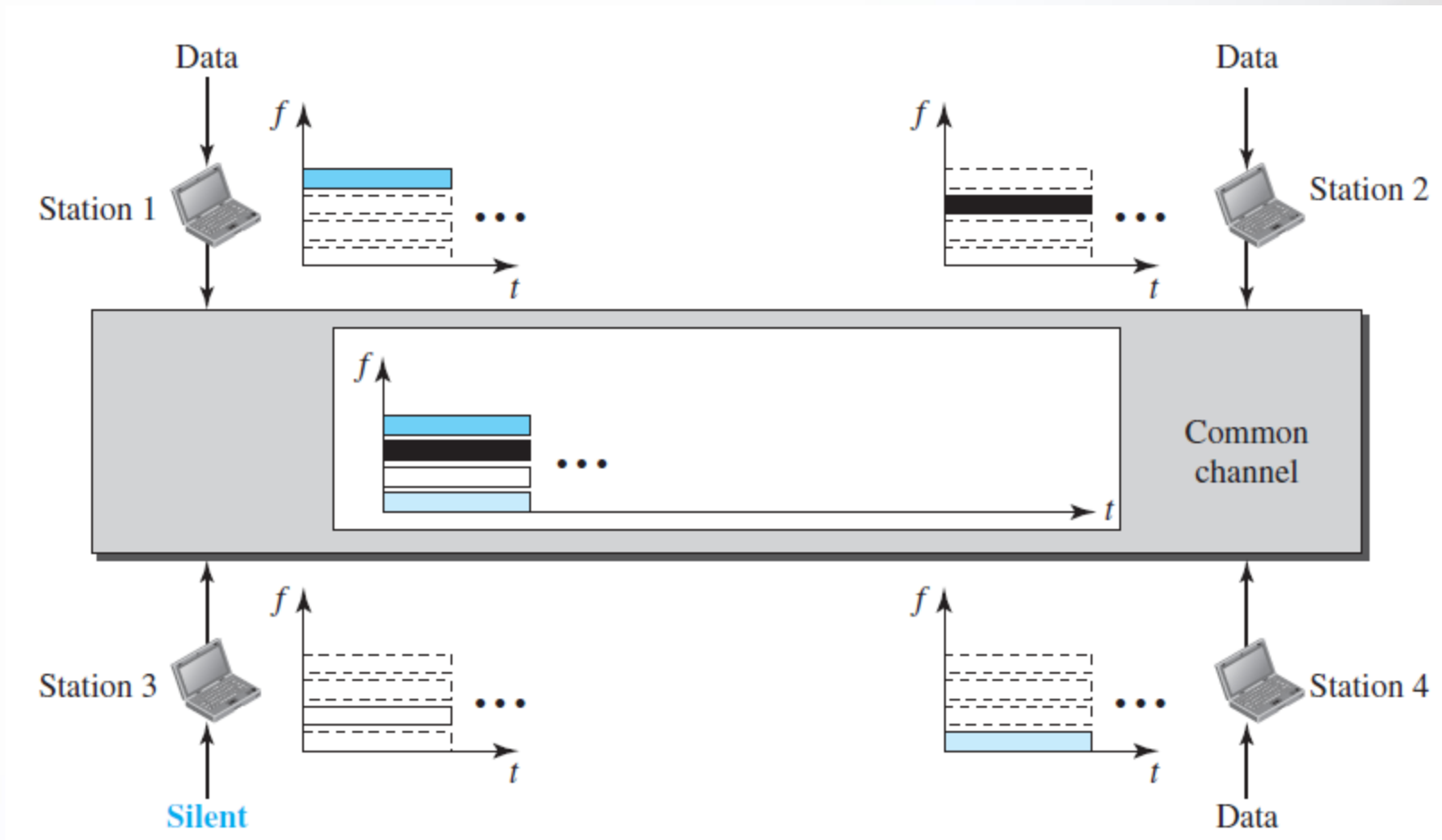
d. Star ring

CHANNELIZATION

- **Channelization** (or *channel partition*) is a multiple-access method in which the available bandwidth of a link is shared among different stations in
 - Time
 - Frequency (or)
 - Through code

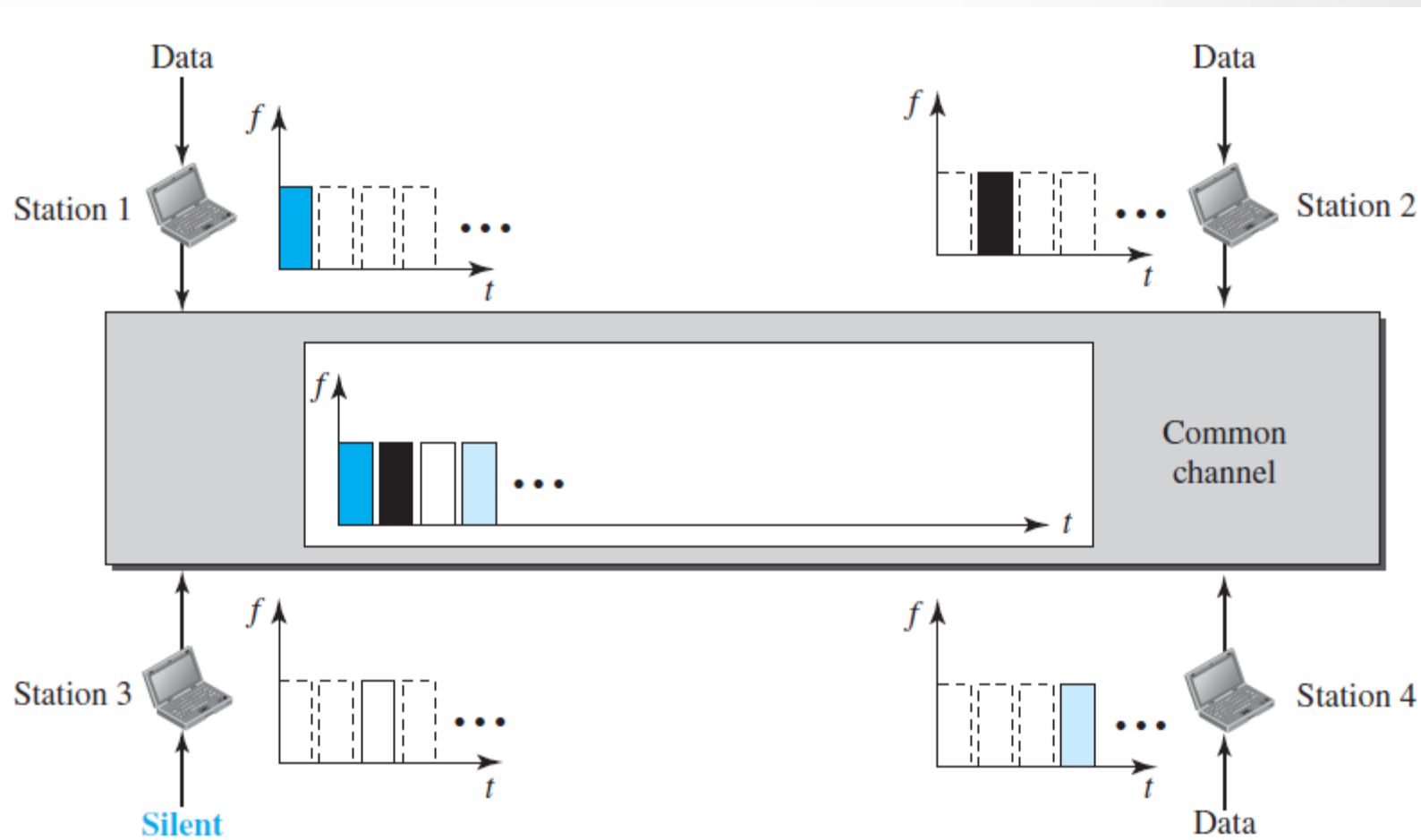
FDMA

- Frequency Division Multiple Access (FDMA)



TDMA

- Time-Division Multiple Access (TDMA)



TDMA

- Each station needs to know the beginning of its slot and the location of its slot.
- This may be difficult because of propagation delays introduced in the system if the stations are spread over a large area.
- To compensate for the delays, we can insert **guard times**.
- Synchronization is normally accomplished by having some **synchronization bits** (normally referred to as preamble bits) at the beginning of each slot.

CDMA

- Code-Division Multiple Access
- CDMA differs from FDMA in that only one channel occupies the entire bandwidth of the link.
- It differs from TDMA in that all stations can send data simultaneously; there is no timesharing.
- We assume that the assigned codes have two properties.
 1. If we multiply each code by another, we get 0.
 2. If we multiply each code by itself, we get 4 (the number of stations).

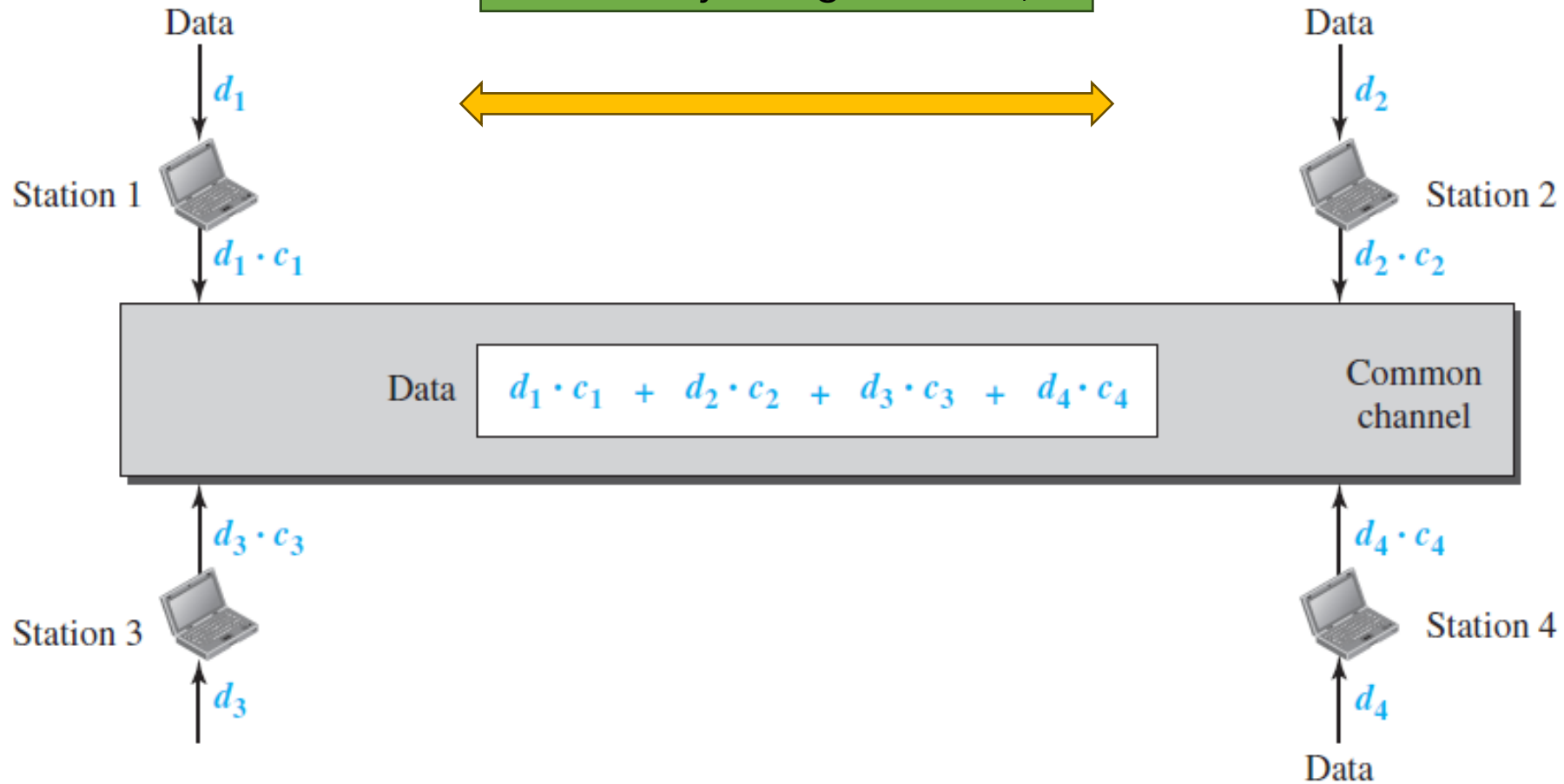
CDMA

$$c_i \cdot c_j = 0 \text{ if } i \neq j$$

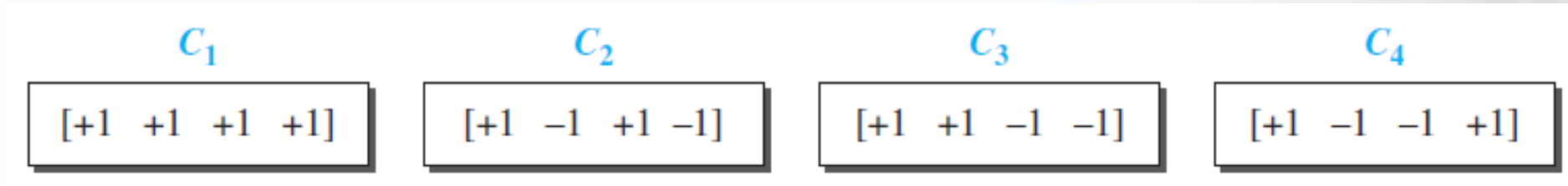
$$c_i \cdot c_i = N \text{ where } N = \text{code length, here 4}$$

$$\begin{aligned} \text{data} &= (d_1 \cdot c_1 + d_2 \cdot c_2 + d_3 \cdot c_3 + d_4 \cdot c_4) \cdot c_1 \\ &= d_1 \cdot c_1 \cdot c_1 + d_2 \cdot c_2 \cdot c_1 + d_3 \cdot c_3 \cdot c_1 + d_4 \cdot c_4 \cdot c_1 = 4 \times d_1 \end{aligned}$$

Divide by 4 to get Data d_1



CHIPS



- CDMA is based on coding theory.
- Each station is assigned a code, which is a sequence of numbers called *chips*.
- The sequences are not random; they were carefully selected; they are called ***orthogonal sequences***.

C_1	C_2	C_3	C_4
$[+1 \ +1 \ +1 \ +1]$	$[+1 \ -1 \ +1 \ -1]$	$[+1 \ +1 \ -1 \ -1]$	$[+1 \ -1 \ -1 \ +1]$

1. Each sequence is made of N elements, where N is the number of stations.
2. If we multiply a sequence by a number, every element in the sequence is multiplied by that element. This is called **multiplication of a sequence by a scalar**. For example,

$$2 \bullet [+1 \ +1 \ -1 \ -1] = [+2 \ +2 \ -2 \ -2]$$

c_1	c_2	c_3	c_4
$[+1 \ +1 \ +1 \ +1]$	$[+1 \ -1 \ +1 \ -1]$	$[+1 \ +1 \ -1 \ -1]$	$[+1 \ -1 \ -1 \ +1]$

3. If we multiply two equal sequences, element by element, and add the results, we get N , where N is the number of elements in each sequence. This is called the **inner product of two equal sequences**. For example,

$$[+1 \ +1 \ -1 \ -1] \bullet [+1 \ +1 \ -1 \ -1] = 1 + 1 + 1 + 1 = 4$$

c_1	c_2	c_3	c_4
$[+1 \ +1 \ +1 \ +1]$	$[+1 \ -1 \ +1 \ -1]$	$[+1 \ +1 \ -1 \ -1]$	$[+1 \ -1 \ -1 \ +1]$

4. If we multiply two different sequences, element by element, and add the results, we get 0. This is called the **inner product of two different sequences**. For example,

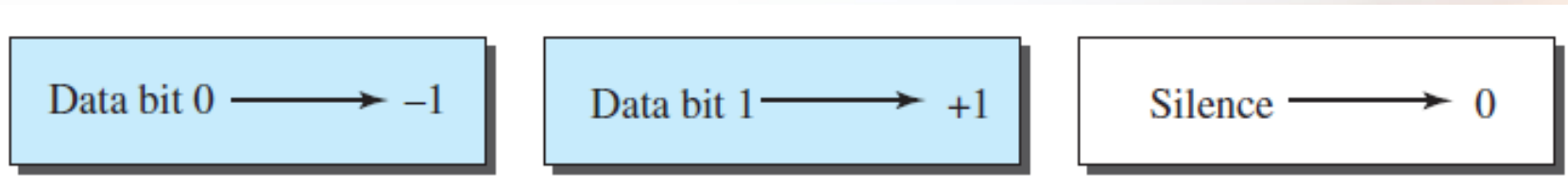
$$[+1 \ +1 \ -1 \ -1] \bullet [+1 \ +1 \ +1 \ +1] = 1 + 1 - 1 - 1 = 0$$

c_1	c_2	c_3	c_4
[+1 +1 +1 +1]	[+1 -1 +1 -1]	[+1 +1 -1 -1]	[+1 -1 -1 +1]

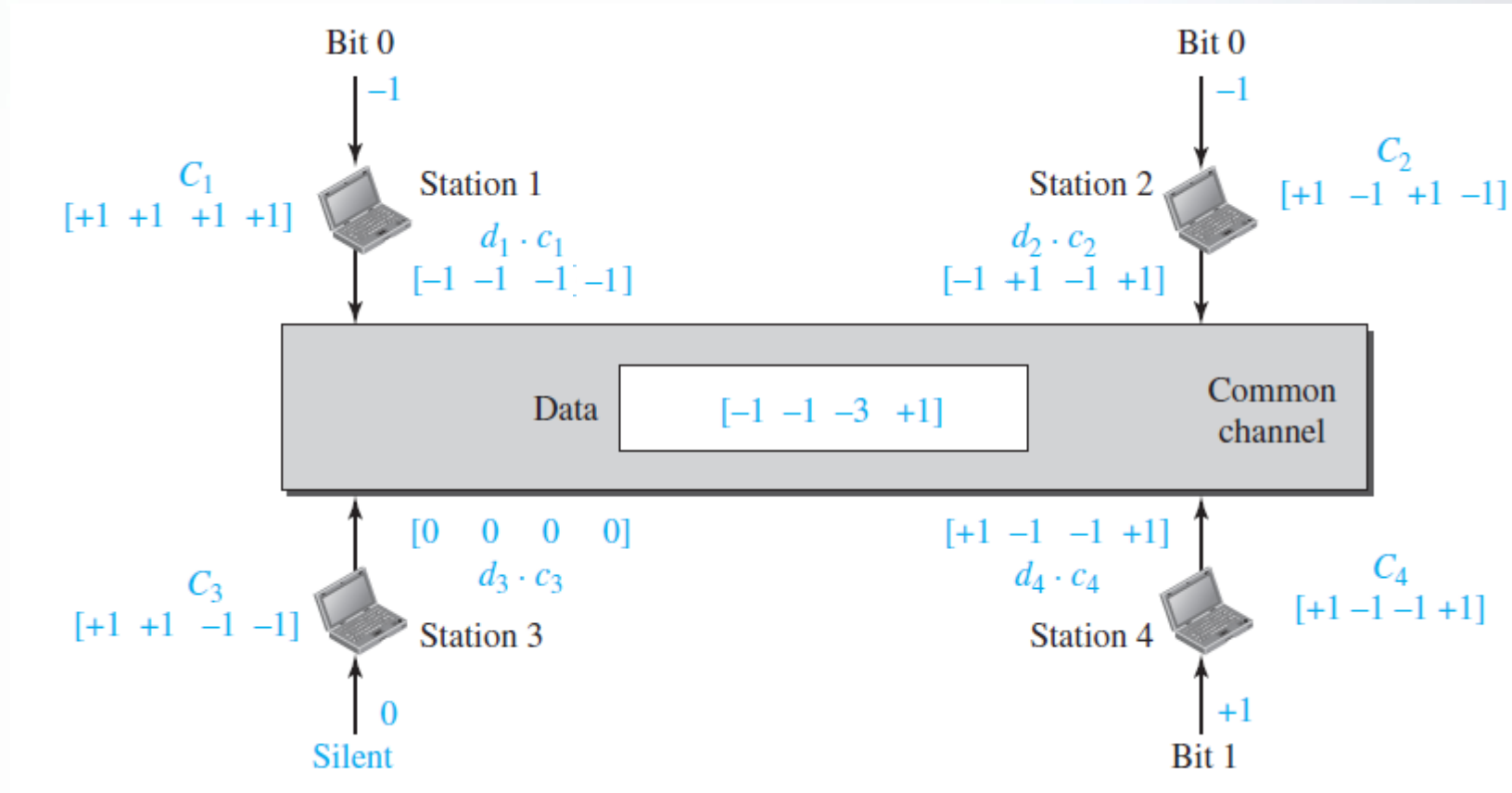
- Adding two sequences means adding the corresponding elements.
The result is another sequence. For example,

$$[+1 +1 -1 -1] + [+1 +1 +1 +1] = [+2 +2 0 0]$$

DATA REPRESENTATION FOR ENCODING

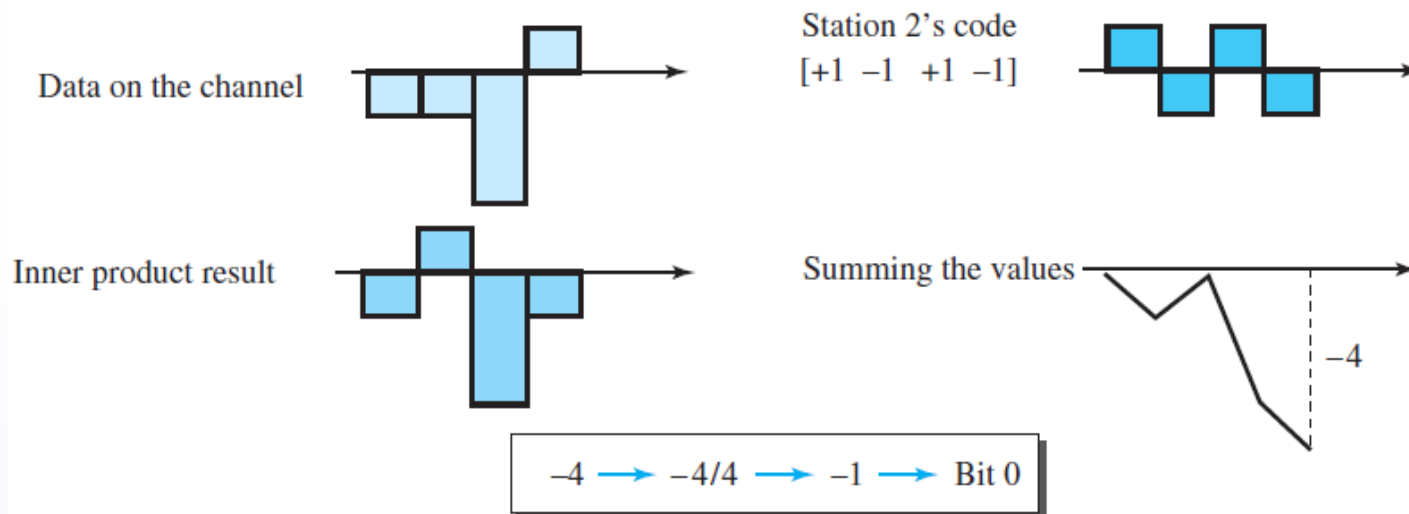
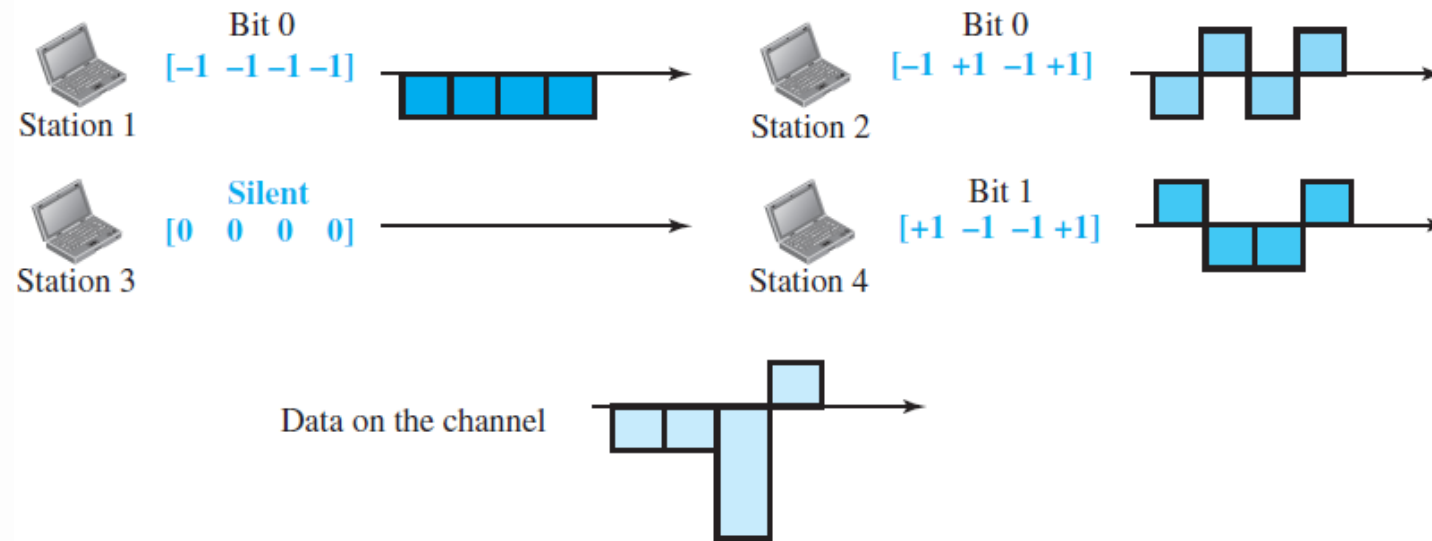


ENCODING AND DECODING



$$[-1, -1, -3, +1] \cdot [+1, -1, +1, -1] = -4/4 = -1 \rightarrow \text{bit 0}$$

SIGNAL LEVEL



SEQUENCE GENERATION

- To generate chip sequences, we use a **Walsh table**, which is a two-dimensional table with an equal number of rows and columns.
- Number of sequence always a power of 2, i.e. $N = 2^m$.

$$W_1 = \begin{bmatrix} +1 \end{bmatrix} \quad W_{2N} = \begin{bmatrix} W_N & W_N \\ W_N & \overline{W_N} \end{bmatrix}$$

a. Two basic rules

$$W_2 = \begin{bmatrix} +1 & +1 \\ +1 & -1 \end{bmatrix} \quad W_4 = \begin{bmatrix} \begin{bmatrix} +1 & +1 \\ +1 & -1 \end{bmatrix} & \begin{bmatrix} +1 & +1 \\ +1 & -1 \end{bmatrix} \\ \begin{bmatrix} +1 & +1 \\ +1 & -1 \end{bmatrix} & \begin{bmatrix} -1 & -1 \\ -1 & +1 \end{bmatrix} \end{bmatrix}$$

b. Generation of W_2 and W_4

ETHERNET

WHAT IS ETHERNET?



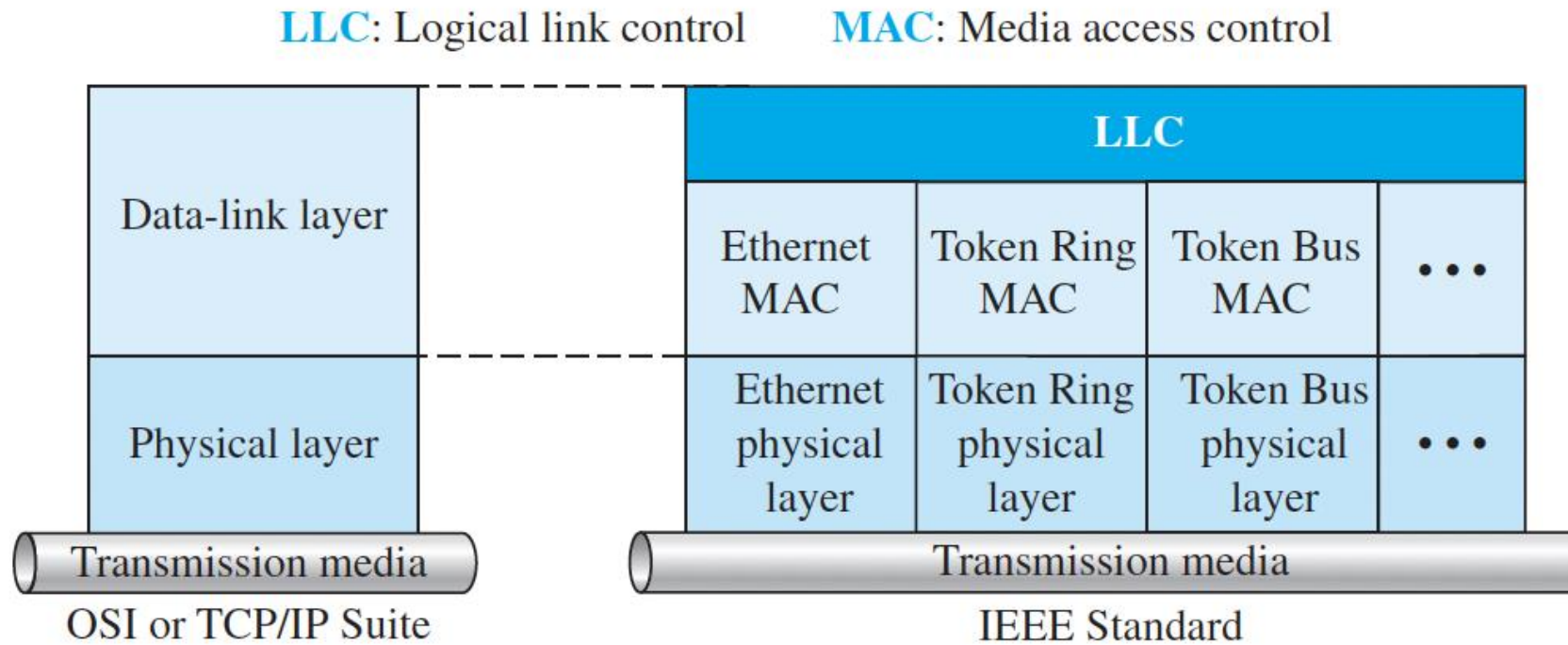
WIRED LAN'S - ETHERNET

- Local Area Network
- Most LAN's are linked to WAN and Internet
- Many LAN technologies exists - Ethernet, Token Ring, Token Bus, FDDI, ATM LAN

IEEE STANDARDS

- Project 802 in 1985
- IEEE Standards to enable interconnection among equipment from a variety of manufacturers.
- Later adopted by ANSI (American National Standards Institute)
- 1987 - ISO approved it as an international standard as ISO 8802 (IEEE 802.2)

IEEE STANDARDS



DATA LINK LAYER

- Divided into 2 sub-layers
 - Logical Link Control (LLC)
 - Media Access Control (MAC)

LOGICAL LINK CONTROL

- It handles
 - Flow Control
 - Error Control
 - Part of Framing (Shared by MAC)
- One single datalink control for all IEEE LANs
- Different from MAC which provides different protocols for different LANs

MEDIA ACCESS CONTROL

- Specific access methods for specific LANS
- For example, it defines CSMA/CD as the media access method for Ethernet LANs and defines the token-passing method for Token Ring and Token Bus LANs.

ETHERNET EVOLUTION

- Originally created in 1970s
- 4 Generations:
 1. Standard Ethernet (10 Mbps)
 2. Fast Ethernet (100 Mbps)
 3. Gigabit Ethernet (1 Gbps)
 4. Ten-Gigabit Ethernet (10 Gbps)